

Codeforces Round 1074 (Div. 4)

A. Perfect Root

1 second, 256 megabytes

A positive integer x is a perfect root if there exists an integer y such that $\sqrt{y} = x$. For example, 5 is a perfect root because $\sqrt{25} = 5$.

For each test case, output n distinct perfect roots. Note that the values only need to be distinct within each test case; you can use the same value in different test cases.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 20$) — the number of test cases.

The only line of each test case contains an integer n ($1 \leq n \leq 20$) — the number of perfect roots to output.

Output

For each test case, output n distinct perfect roots. Each perfect root x must be in the range $1 \leq x \leq 10^9$.

input
3 1 2 5
output
1 2 4 2 102 43 1 21

For the first test case:

- 1 is a perfect root because $\sqrt{1} = 1$.

For the second test case:

- 2 is a perfect root because $\sqrt{4} = 2$.
- 4 is a perfect root because $\sqrt{16} = 4$.

B. Prefix Max

1.5 seconds, 256 megabytes

You are given an array of n integers $a_1, a_2, \dots, a_n$.

The value of an array is the sum of the maximums of each prefix of the array. More formally, the value of an array a is $\sum_{i=1}^n \max(a_1, \dots, a_i)$. For example, the value of the array $[1, 2, 1]$ is $\max(1) + \max(1, 2) + \max(1, 2, 1) = 1 + 2 + 2 = 5$.

You can choose two indices i and j and swap elements a_i and a_j ; this operation can be applied **at most** one time.

Find the maximum possible value of the array a after at most one operation.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 100$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \leq n \leq 50$) — the length of the array a .

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^4$) — the array a .

Output

For each test case, output the maximum possible value of the array a after the swap has been performed.

input
4 5 2 1 4 5 3 2 5 1 3 3 2 1 2 6 7
output
25 10 9 14

For the first test case, we can swap a_1 with a_4 to get the array $[5, 1, 4, 2, 3]$, which has a value of $\max(5) + \max(5, 1) + \max(5, 1, 4) + \max(5, 1, 4, 2) + \max(5, 1, 4, 2, 3)$.

For the second test case, the current value of the array is $\max(5) + \max(5, 1) = 10$. If we were to swap a_1 and a_2 , a would be equal to $[1, 5]$, which has a value of $\max(1) + \max(1, 5) = 6$, meaning the best option is to not perform a swap.

C. Shifted MEX

2 seconds, 256 megabytes

You are given an array of n integers $a_1, a_2, \dots, a_n$. You are allowed to perform the following operation once.

- Select an integer x (which may be negative), and for each value i ($1 \leq i \leq n$), set $a_i = a_i + x$.

For example, if $a = [1, 3, 4, 2]$, and you perform the operation with $x = 3$, a is now equal to $[4, 6, 7, 5]$.

Output the maximum possible value of $\text{MEX}(a)^*$ after the operation is performed.

* $\text{MEX}(a)$ is defined as the smallest non-negative integer that is not present in the array. For example, $\text{MEX}([1, 2, 0, 5])$ is 3, and $\text{MEX}([1, 2, 4, 9])$ is 0.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 1000$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 3000$) — the length of array a .

The second line contains n integers $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$) — the array a .

It is guaranteed that the sum of n over all test cases does not exceed 3000.

Output

For each test case, output the maximum possible value of $\text{MEX}(a)$ after the operation has been performed.

input
6 1 4 5 0 1 1 2 3 2 1 1 4 4 2 3 6 5 2 4 1 0 -1 6 -1 1 2 3 5 6

output
1 4 1 3 4 3

For the first test case, performing the operation with $x = -4$ makes $a = [0]$, and $\text{MEX}([0]) = 1$.

For the second test case, the **MEX** is already 4, which is the highest possible, so we can perform the operation with $x = 0$, which will not change the array.

D. OutOfMemoryError

2 seconds, 256 megabytes

Bessie has an array of n integers $a_1, a_2, \dots, a_n$. Bessie performs m operations on the array. The i -th operation sets $a_{b_i} = a_{b_i} + c_i$.

Unfortunately, due to rising RAM costs, Bessie's computer has limited memory, and whenever any element of the array is greater than h , her computer crashes, and every element in the array is reset to its original value.

After all operations have been performed, output the array a .

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains three integers n, m, h ($1 \leq n, m \leq 2 \cdot 10^5, 1 \leq h \leq 10^9$) — the length of array a , the number of operations performed, and the maximum value that Bessie's computer can store without crashing.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq h$) — the array a .

The next m lines contain two integers b_i, c_i ($1 \leq b_i \leq n, 0 \leq c_i \leq 10^9$) — the operations that Bessie performs on the array.

It is guaranteed that the sum of n and the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the array a after all operations have been performed.

input
3 3 4 5 1 2 1 1 4 2 4 3 3 2 0 5 3 1 1 1 1 1 1 1 1 1 1 2 1 4 4 1 1 0 0 0 1 1 4 4 3 3 4 4
output
1 2 4 1 1 1 1 1 1 0 0 0

For the first test case, the array a is changed as follows:

- Before all operations, $a = [1, 2, 1]$.
- After the first operation, $a = [5, 2, 1]$.
- After the second operation, $a = [5, 6, 1]$, but since $6 > h$, the computer crashes, and $a = [1, 2, 1]$.
- After the third operation, $a = [1, 2, 4]$.
- After the fourth operation, $a = [1, 2, 4]$.

For the third test case, each operation causes the computer to crash, so $a = [1, 0, 0, 0]$.

E. The Robotic Rush

3 seconds, 256 megabytes

There exists an infinitely long number line.

On the number line, there are n robots and m spikes, each located at a specific point on the number line. The i -th robot is located at position a_i , and the i -th spike is located at position b_i . If a robot touches a spike, it dies.

k instructions are transmitted to the robots, with each instruction being to either move left by one unit or to move right by one unit.

For each value i ($1 \leq i \leq k$), output how many robots are still alive after the first i instructions have been processed.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains three integers n, m, k ($1 \leq n, m, k \leq 2 \cdot 10^5$) — the number of robots, spikes, and instructions respectively.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$) — the locations of the robots. It is guaranteed that all elements of a are distinct.

The third line contains m integers $b_1, b_2, \dots, b_m$ ($0 \leq b_i \leq 10^9$) — the locations of the spikes. It is guaranteed that all elements of b are distinct.

The fourth line contains a string of length k — the instructions transmitted to the robots. Each character is either **L**, representing an instruction to move to the left, or **R**, representing an instruction to move to the right.

It is guaranteed that the sum of each of n, m, k over all test cases does not exceed $2 \cdot 10^5$.

Additional constraint: it is guaranteed that there are no robots and spikes at the same position.

Output

Output k integers, where the i -th integer indicates how many robots are alive after the first i instructions have been processed.

For Python users, make sure to select PyPy3 / PyPy2 (depending on which version of Python you're using) instead of Python3 or Python2 when submitting.

input
3 2 1 3 0 1 2 LRR 2 3 3 2 4 1 3 5 LRL 3 2 3 1 3 7 9 6 RRL
output
2 2 1 0 0 0 3 2 2

For the first test case:

- The first robot will move to positions $0 \rightarrow -1 \rightarrow 0 \rightarrow 1$, so it will not die.
- The second robot will move to positions $1 \rightarrow 0 \rightarrow 1 \rightarrow 2$, so it will die after processing the third instruction, since there is a spike at position 2.

For the second test case, both robots will die after moving once.

F. BattleCows

2 seconds, 256 megabytes

Farmer John is sending a cow to compete in the International Bovine Olympiad, but he's too lazy to write problems for a contest, so he does the next most reasonable thing: a fighting tournament!

Farmer John has 2^n cows standing in a line, with the i -th cow having a skill level of a_i . Each cow starts in a stack containing only itself, and the skill level of the stack is equal to the XOR-sum* of the skill levels of all the cows in it. For example, if the stack consists of cows 1, 3, 9 from bottom to top, then the skill level of the stack is $1 \oplus 3 \oplus 9 = 11$.

The following process repeats until there is only one stack.

- Every stack in an odd position (1st, 3rd, etc.) starts a fight with the stack to its right. For example, the 1st stack will fight with the 2nd stack, the 3rd stack will fight with the 4th stack, etc.
- The stack with the higher skill level will win the match, with the left-most stack winning in case of a tie.
- The winning stack will jump on top of the losing stack and shift itself over such that there are no gaps left by the defeated stacks.

In order to make this more exciting, Farmer John created q potions, such that the i -th potion sets the skill level of the cow who drinks it to c_i . Farmer John wants to test his potions on the cows, and so he gives the i -th potion to cow b_i and then runs the tournament. For each trial, Farmer John wants to know how many cows are above the cow that was given the potion in the final stack.

Farmer John's potions wear off quickly, so when the tournament ends, the cow that was given the potion will **have its skill level return to what it originally was**. In other words, all queries are independent.

*The XOR-sum of an array $x_1, x_2, \dots, x_y$ is equal to $x_1 \oplus x_2 \oplus x_3 \dots x_{y-1} \oplus x_y$ where $\oplus$ denotes the bitwise XOR operation.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and q ($1 \leq n \leq 18, 1 \leq q \leq 2 \cdot 10^5$) — where 2^n is equal to the number of cows in the tournament and q is the number of potions.

The second line contains 2^n integers $a_1, a_2, \dots, a_{2^n}$ ($1 \leq a_i \leq 2^{30}$) — the skill levels of the cows.

The next q lines contain two integers b_i and c_i ($1 \leq b_i \leq 2^n, 1 \leq c_i \leq 2^{30}$) — the index of the cow that is given the potion and the new skill level of the cow.

It is guaranteed that the sum of 2^n over all test cases does not exceed $2^{18} = 262144$ and that the sum of q over all test cases does not exceed $2 \cdot 10^5$.

Output

For each trial, output how many cows are above the cow that was given the potion in the final line.

input
4
2 2
1 3 5 7
1 1
4 8
1 2
1 3
1 4
1 2
3 4
1 8 3 10 2 5 7 1
5 3
8 12
1 9
2 1
2 1
1 2 3 4
3 1

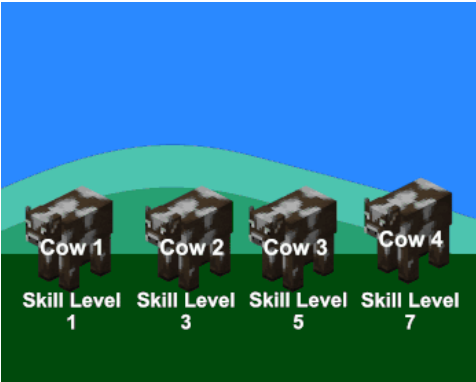
output
1
0
0
1
5
0
2
3
1

For the first round of the first test case, the skill levels of the cows don't change, and the matches go as follows:

- Cows 1 and 2 fight. Since cow 2's skill level is higher than cow 1's skill level, cow 2 wins the fight, so its stack has cows 1, 2 in that order, and the stack's skill level is now $3 \oplus 1 = 2$.
- Cows 3 and 4 fight. Since cow 4's skill level is higher than cow 3's skill level, cow 4 wins the fight, so its stack has cows 3, 4 in that order, and the stack's skill level is now $7 \oplus 5 = 2$.
- The two remaining stacks fight. Since the first stack's skill level is equal to the second stack's skill level, the first stack wins the fight, since its index is lower than the second stack's, so the final stack has cows 3, 4, 1, 2 in that order, and its skill level is now $2 \oplus 2 = 0$.

Since we gave the potion to cow 1, our answer is 1, since there is one cow above cow 1.

A visualization of the round is shown below.



For the second round of the first test case, cow 4's skill level is now 8, and the matches go as follows:

- Cows 1 and 2 fight. Since cow 2's skill level is higher than cow 1's skill level, cow 2 wins the fight, so its stack has cows 1, 2 in that order, and the stack's skill level is now $3 \oplus 1 = 2$.
- Cows 3 and 4 fight. Since cow 4's skill level is higher than cow 3's skill level, cow 4 wins the fight, so its stack has cows 3, 4 in that order, and the stack's skill level is now $8 \oplus 5 = 13$.
- The two remaining stacks fight. Since the second stack's skill level is higher than the first stack's skill level, the second stack wins the fight, so the final stack has cows 1, 2, 3, 4 in that order, and its skill level is now $2 \oplus 13 = 15$.

Since we gave the potion to cow 4, our answer is 0, since there are no cows above cow 4.

G. Mixing MEXes

2 seconds, 256 megabytes

You are given n arrays $a_1, a_2, \dots, a_n$.

The following operation is performed exactly once:

- Choose any array among $a_1, a_2, \dots, a_n$. Suppose you've chosen array a_i ($1 \leq i \leq n$).
- Choose any element in array a_i . Suppose you've chosen the j -th element of a_i , denoted by $a_{i,j}$ ($1 \leq j \leq |a_i|$, where $|a_i|$ denotes the length of array a_i).
- Choose any other array among $a_1, a_2, \dots, a_n$ that is not a_i . Suppose you've chosen a_k ($1 \leq k \leq n, k \neq i$).
- Add $a_{i,j}$ to the back of array a_k . Then, remove $a_{i,j}$ from a_i .
- The value of this operation (i, j, k) is defined as the sum of each array's MEX after the operation is performed. More formally, the

value of an operation after the operation is performed is $\sum_{i=1}^n \text{MEX}(a_i)$.

Evaluate the sum of the values of all possible distinct independent operations. Two operations are distinct if the ordered triple of integers (i, j, k) is different.

$\text{MEX}(a)$ is defined as the smallest non-negative integer that is not present in the array. For example, $\text{MEX}([1, 2, 0, 5])$ is 3, and $\text{MEX}([1, 2, 4, 9])$ is 0.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \leq n \leq 2 \cdot 10^5$) — the number of arrays.

The next n lines start with l_i ($1 \leq l_i \leq 10^5$) — the length of the i th array — then contain l_i integers $a_1, a_2, \dots, a_{l_i}$ ($0 \leq a_{i_j} \leq 10^6$) — the array a_i .

It is guaranteed that the sum of l_i over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the sum of the values of all possible distinct operations.

input
6 2 1 0 2 1 2 3 1 1 2 2 3 3 4 5 6 5 4 1 7 8 10 2 5 6 2 0 7 2 6 6 2 6 8 2 1 3 3 0 1 2 2 6 0 0 1 2 2 3 3 0 2 3 10 1 0 9 7 8 0 1 5 6 4 3 2 8 4 3 8 6 2 5 0 1 7 2 3 0 1 0 4 0 2 3 1 9 2 0 5 4 1 3 0 0 0 7 6 3 2 4 1 8 0 5 3 2 4 1 0 4 0 3 1 1 3 0 3 2
output
6 0 50 8 43 19202

For the first test case, we have 3 possible distinct operations:

- $i = 1, j = 1, k = 2$: The arrays are now [] and [0, 1, 2], which have a mex of 0 and 3 respectively, so the value of the operation is 3.
- $i = 2, j = 1, k = 1$: The arrays are now [0, 1] and [2], which have a mex of 2 and 0 respectively, so the value of the operation is 2.
- $i = 2, j = 2, k = 1$: The arrays are now [0, 2] and [1], which have a mex of 1 and 0 respectively, so the value of the operation is 1.

For the second test case, since no array contains a zero, the value of all operations will be 0.

H. BattleCows 2

3 seconds, 256 megabytes

Farmer John wants to host another tournament with n cows, where the i -th cow has a skill level of a_i . The following process repeats until there is only one cow in the line.

- The first cow in the line fights the second cow in the line, and the cow with the higher skill level wins. If there is a tie, the first cow wins.
- The winning cow's skill level is set to $x + y$, where x is the skill level of the winning cow and y is the skill level of the losing cow.
- The losing cow leaves the line.

However, to maintain accuracy to the real USACOW competition, a cow is able to cheat up to k times. This means that even if it loses the match, Farmer John will treat it as if the losing cow won the match, meaning that the losing cow's skill level will be set to $x + y$, where x is the skill level of the winning cow and y is the skill level of the losing cow, and the winning cow will leave the line.

A position x is good for a cow i if cow i can be removed from its original position and inserted at index x **without changing the order of the other cows** and be the only cow remaining in the line once the tournament has ended, assuming no other cow cheats.

For each cow in the line, calculate the number of good positions for that cow.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and k ($2 \leq n \leq 2 \cdot 10^5, 0 \leq k < n$) — the number of cows and the number of cheats a cow can use.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the skill levels of the cows.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n integers, where the i -th integer denotes the number of good positions for cow i .

input
7 2 0 2 1 2 0 1 1 3 1 1 1 3 3 0 2 1 1 3 1 1 3 1 4 1 1 2 1 3 7 2 1 3 3 17 39 3 12
output
2 0 1 1 2 2 3 2 0 0 3 3 2 4 4 4 4 4 6 6 7 7 5 7

For the first test case, cow 2 would lose no matter what it was positioned, and cow 1 would win no matter where it was positioned.

For the second test case, both cows could win at position 1, but could not win at position 2, meaning that they each have 1 good position.

For the third test case, let's look at cow 1.

- If cow 1 were at position 1, then it would win its first match, making its skill level equal to 2, and could cheat to win its second match.
- If cow 1 were at position 2, then it would have to cheat to win its first match, but would then lose its second match.
- If cow 1 were at position 3, then it would have to cheat to win its first match, and would have no more matches.

Hints: Codeforces Round 1074 (Div. 4)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Perfect Root (official hints)

Hint 1: Fix an integer x , how could you find out the value of y ?

Hint 2: Square both sides of the equation, for what values of x is x^2 an integer?

B. Prefix Max (official hints)

Hint 1: If we were to change the problem such that you could perform as many swaps as you wanted, the answer would remain the same.

Hint 2: Once our running sum encounters the maximum element of the array, the max of that prefix will always be the maximum element of the array.

C. Shifted MEX (official hints)

Solution 1: Hint 1: Duplicate values don't matter, nor does the order of the elements.

Solution 1: Hint 2: Let's consider the inverse problem, where we have a array that is equal to $[0, 1, 2, \dots, k - 1]$ and we can add x to each value in the array. No matter which value of x we choose, the values in a are consecutive.

Solution 2: Hint 1: In order for the MEX to be greater than 0, 0 has to be contained in the array.

Solution 2: Hint 2: Since there are at most 3000 elements in a , there are 3000 possible values of x that have a MEX greater than 0.

D. OutOfMemoryError (official hints)

Hint 1: Clearly, resetting all of the values is too slow, but we don't have to update all the elements at the same time.

Hint 2: If an element hasn't been modified after the last crash, it's value in the final array is the same as its value in the original array.

E. The Robotic Rush (official hints)

Hint 1: Each robot's movements is independent of another robot's movements.

Hint 2: We only need to care about the spikes directly to the right and left of the robot, if they exist.

Hint 3: To figure out when a robot dies, we don't need to care about the position of the robot, only the distance to its left and right spikes.

Hint 4: We only care about the left-most a robot has been and the right-most a robot has been.

F. BattleCows (official hints)

Hint 1: A single cow will only face n matches.

Hint 2: XOR is cumulative, meaning the order of the cows in a stack doesn't change its skill level.

Hint 3: Similarly, the order of the cows in the stack doesn't change the final position of the cow who was given the potion in the final line.

Hint 4: You can calculate range XOR queries in $O(1)$ using prefix sum.

G. Mixing MEXes (official hints)

Hint 1: The only time adding an element changes the MEX of an array is if the added element is equal to the MEX of the array.

Hint 2: For a given element, the element is removed $n - 1$ times, and the element is added to another array $n - 1$ times.

Hint 3: We can treat removing an element from the array and adding it to another array as two separate interactions.

H. BattleCows 2 (official hints)

Hint 1: I had originally proposed this problem with a subtask to solve for Cow 0 only, try that.

Hint 2: We can simulate each cow as if it started as position 0.

Hint 3: A cow only needs cheat at most $\log(\max(a_i))$ times, since its skill level is (at least) double after each cheat.

Hint 4: Prefix sums, binary search, and helper methods are your friend.

Source: <https://codeforces.com/blog/entry/150288>

[Codeforces](https://codeforces.com/) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform

Codeforces Round 1062 (Div. 4)

A. Square?

1 second, 256 megabytes

You are given 4 sticks of lengths a , b , c , and d . You can not break or bend them.

Determine whether it is possible to form a square* using the given sticks.

*A square is defined as a polygon consisting of 4 vertices, of which all sides have equal length and all inner angles are equal. No two edges of the polygon may intersect each other.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The only line of each test case contains four integers a , b , c , and d ($1 \leq a, b, c, d \leq 10$) — the lengths of the sticks.

Output

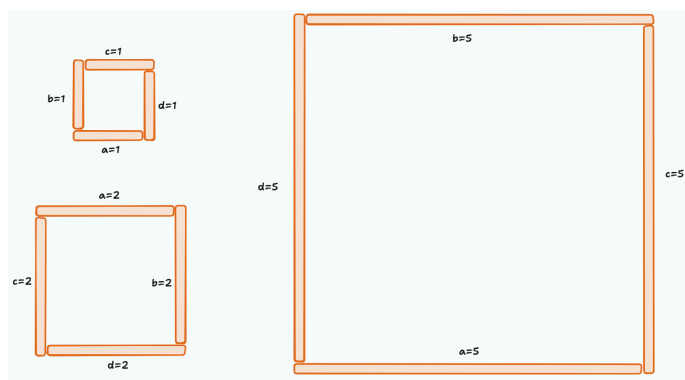
For each test case, print "YES" if it is possible to form a square using the given sticks, and "NO" otherwise.

You may print each letter in any case (uppercase or lowercase). For example, the strings "yEs", "yes", "Yes", and "YES" will all be recognized as a positive answer.

input
7
1 2 3 4
1 1 1 1
2 2 2 2
1 2 1 2
1 1 5 5
5 5 5 5
4 10 5 9
output
NO
YES
YES
NO
NO
YES
NO

In the first test case, we can prove that we can't make a square.

In the second, third, and sixth test cases, we can make a square like this:



B. Your Name

1 second, 256 megabytes

khba is writing his girlfriend's name. He has n cubes, each with one lowercase Latin letter written on it. They are arranged in a row, forming a string s . His girlfriend's name is also a string t , consisting of n lowercase Latin letters.

To prove his love, he must check whether it is possible to rearrange the letters of string s so that it becomes her name t .

Input

The first line contains an integer q ($1 \leq q \leq 1000$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 20$).

The second line of each test case contains two distinct strings s and t , each consisting of n lowercase Latin letters.

Output

For each test case, output "YES" if the letters of s can be arranged to form t ; otherwise, output "NO".

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes" and "YES" will be recognized as positive responses.

input
5
7
humitsa mitsuha
4
orhi hori
6
aakima makima
6
nezuqo nezuko
6
misaka mikasa
output
YES
YES
NO
NO
YES

In the first example, the initial string is "humitsa", and the following operations can be performed:

- swap the first and third characters, resulting in "muhitsa"
- swap the second and fourth characters, resulting in "mihutsa"
- swap the third and fifth characters, resulting in "mithusa"
- swap the fourth and sixth characters, resulting in "mitsuha"

In the second example, the initial string is "orhi", and the following operations can be performed:

- swap the second and third characters, resulting in "ohri"
- swap the first and second characters, resulting in "hori"

C. Isamatdin and His Magic Wand!

2 seconds, 256 megabytes

Isamatdin has n toys arranged in a row. The i -th toy has an integer a_i . He wanted to sort them because otherwise, his mother would scold him.

However, Isamatdin never liked arranging toys in order, so his friend JahonaliX gave him a magic wand to help. Unfortunately, JahonaliX made a small mistake while creating the wand.

But Isamatdin couldn't wait any longer and decided to use the broken wand anyway. The wand can only swap two toys if their integers have **different parity** (one is even, the other is odd). In other words, you can swap toys in positions (i, j) only if $a_i \bmod 2 \neq a_j \bmod 2$, where **mod** — is the remainder of integer division.

Now he wants to know the **lexicographically smallest*** arrangement he can achieve using this broken wand.

* A sequence p is *lexicographically smaller* than a sequence q if there exists an index i such that $p_j = q_j$ for all $j < i$, and $p_i < q_i$.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of toys.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the integers of the toys.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n integers — the lexicographically smallest sequence that can be obtained using the described operation.

input
7 4 2 3 1 4 5 3 2 1 3 4 4 3 7 5 1 2 1000000000 2 3 1 3 5 5 2 5 3 1 7 4 2 4 8 6
output
1 2 3 4 1 2 3 3 4 3 7 5 1 1000000000 2 1 3 5 1 2 3 5 7 2 4 8 6

In the first test case, we can swap positions (1, 3) and then (2, 3).
In the second test case, we can swap positions (1, 2), (1, 3), and then (2, 3).
In the third and fourth test cases, we can't swap any positions because all toy integers have the same parity.

D. Yet Another Array Problem

2 seconds, 256 megabytes

You are given an integer n and an array a of length n .

Find the **smallest** integer x ($2 \leq x \leq 10^{18}$) such that there exists an index i ($1 \leq i \leq n$) with $\gcd^*(a_i, x) = 1$. If no such x exists within the range $[2, 10^{18}]$, output -1 .

* $\gcd(x, y)$ denotes the **greatest common divisor (GCD)** of integers x and y .

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

Each of the following t test cases consists of two lines:

The first line contains a single integer n ($1 \leq n \leq 10^5$) — the length of the array.

The second line contains n space-separated integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^{18}$).

It is guaranteed that the total sum of n across all test cases does not exceed 10^5 .

Output

For each test case, output a single integer: the smallest x ($2 \leq x \leq 10^{18}$) such that there exists an index i with $\gcd(a_i, x) = 1$. If there is no such x in the range $[2, 10^{18}]$, print -1 .

input
4 1 1 4 6 6 12 12 3 24 120 210 4 2 4 6 10
output
2 5 5 3

In the first test case, $\gcd(2, 1) = 1$, which is the smallest number satisfying the condition.
In the second test case:

- $\gcd(2, 6) = 2$, $\gcd(2, 12) = 2$, so 2 cannot be the answer.
- $\gcd(3, 6) = 3$, $\gcd(3, 12) = 3$, so 3 cannot be the answer.
- $\gcd(4, 6) = 2$, $\gcd(4, 12) = 4$, so 4 cannot be the answer.
- $\gcd(5, 6) = 1$, so the answer is 5.

In the third test case:

- $\gcd(2, 24) = 2$, $\gcd(2, 120) = 2$, $\gcd(2, 210) = 2$, so 2 cannot be the answer.
- $\gcd(3, 24) = 3$, $\gcd(3, 120) = 3$, $\gcd(3, 210) = 3$, so 3 cannot be the answer.
- $\gcd(4, 24) = 4$, $\gcd(4, 120) = 4$, $\gcd(4, 210) = 2$, so 4 cannot be the answer.
- $\gcd(5, 24) = 1$, so the answer is 5.

In the fourth test case:

- $\gcd(2, 2) = 2$, $\gcd(2, 4) = 2$, $\gcd(2, 6) = 2$, $\gcd(2, 10) = 2$, so 2 cannot be the answer.
- $\gcd(3, 2) = 1$, so the answer is 3.

E. khba Loves to Sleep!

3 seconds, 256 megabytes

khba has n friends, each standing on a line at position a_i , and each of them is in the range $[0, x]$.

They all want to come to him. One of his friends, Isamatdin, gave him k teleports. Each friend will walk to the nearest teleport (choosing the shortest distance). Once a friend reaches a teleport, khba and the friend can instantly meet.

But khba is so tired that he'll be sleeping while his friends are walking toward him. Now he wants to choose k teleport positions so that their positions are distinct and lie within the range $[0, x]$, in order to maximize the time it takes for the first friend who reaches a teleport to reach it. Assume that friends move at the same speed.

Since khba isn't good at calculations, you should output the k chosen teleport positions.

Input

Each test contains multiple test cases. The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains three integers n , k , and x ($1 \leq n, k \leq 2 \cdot 10^5, k - 1 \leq x \leq 10^9$) — the number of friends, the number of teleports, and the range of possible positions for the teleports.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq x$) — the positions of khba's friends.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

It is guaranteed that the sum of k over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single line containing k integers — the k chosen teleport positions. The positions must be distinct and lie within the range $[0, x]$. The positions may be output in any order.

If there are multiple optimal choices, output any of them.

input
10 4 1 4 1 0 2 4 5 5 4 0 1 2 3 4 2 1 4 4 0 3 4 6 2 4 3 3 2 12 6 12 0 4 3 12 8 12 0 4 1 1 1000000000 0 1 1 1000000000 1000000000 3 4 9 8 7 9 3 4 9 2 0 1
output
3 0 1 2 3 4 2 0 1 5 6 3 9 2 6 10 1000000000 0 0 1 2 3 6 7 8 9

Sample 1. Friends at positions $[1, 0, 2, 4]$. Chosen teleport position: $[3]$.

Nearest teleport for each friend is in 3 , minimal times for friends are $[2, 3, 1, 1]$, respectively.

Sample 2. Friends at positions $[0, 1, 2, 3, 4]$. Chosen teleport positions: $[0, 1, 2, 3, 4]$.

Minimal times for friends are $[0, 0, 0, 0, 0]$, respectively.

Sample 3. Friends at positions $[4, 0]$. Chosen teleport position: $[2]$.

Minimal times for friends are $[2, 2]$, respectively.

Sample 4. Friends at positions $[2, 4, 3]$. Chosen teleport positions: $[0, 1, 5, 6]$.

Minimal times for friends are $[1, 1, 2]$, respectively.

Sample 5. Friends at positions $[6, 12, 0]$. Chosen teleport positions: $[3, 9]$.

Minimal times for friends are $[3, 3, 3]$, respectively.

F. Tree, TREE!!!

2 seconds, 256 megabytes

Roots change, but the tree stands strong — so should your logic.

Behruzбек received a tree* with n nodes. For a chosen root† r , Behruzбек wants to find *cuteness* of the tree.

Consider every set of k distinct nodes of the tree. For each such set, compute its *lowest common ancestor (LCA)* in the tree when it is rooted at r . Let S_r be the set of all distinct nodes obtained this way; then *cuteness* of the tree is $|S_r|$, where $|S|$ means the number of distinct elements.

After discovering the *cuteness* of trees, Behruzбек became interested in finding the *kawaiiiness* of the tree! *Kawaiiiness* is defined as:

$$\sum_{r=1}^n |S_r| = |S_1| + |S_2| + \dots + |S_n|$$

Unfortunately, Behruzбек is feeling sleepy now. Please help Behruzбек by finding the *kawaiiiness* of the tree!

* A tree is a connected graph without cycles.

† A rooted tree is a tree where one vertex is special and called the root.

Input

The first line contains the number of test cases t ($1 \leq t \leq 10^4$).

The first line of each test case contains two integers n and k ($2 \leq k \leq n \leq 2 \cdot 10^5$) — the number of vertices in the tree and the number of distinct integers to be chosen.

The following $n - 1$ lines of each test case describe the tree. Each of the lines contains two integers u and v ($1 \leq u, v \leq n, u \neq v$) that indicate an edge between vertex u and v . It is guaranteed that these edges form a tree.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output one integer — the value of $\sum_{r=1}^n |S_r|$.

input
4 2 2 1 2 5 3 1 2 1 3 1 4 1 5 6 3 1 2 1 3 2 4 2 5 3 6 10 5 5 6 4 9 3 9 2 6 2 8 8 9 6 10 1 6 4 7
output
2 9 17 35

Let $f(i) = |S_i|$

For the third example:

- Root is 1, only 1 and 2 nodes can be obtained. For example, we can choose: $LCA(4, 5, 6) = 1$ and $LCA(2, 4, 5) = 2$. As a result, $f(1) = 2$.
- Root is 2, only 1 and 2 nodes can be obtained. For example, we can choose: $LCA(1, 3, 6) = 1$ and $LCA(1, 4, 5) = 2$. As a result, $f(2) = 2$.
- Root is 3, $f(3) = 3$. For example, node 3 can be obtained by choosing: $LCA(2, 4, 6) = 3$.
- Root is 4, $f(4) = 3$. For example, node 2 can be obtained by choosing: $LCA(1, 3, 5) = 2$.
- Root is 5, $f(5) = 3$. For example, node 2 can be obtained by choosing: $LCA(3, 4, 6) = 2$.
- Root is 6, $f(6) = 4$. For example, node 3 can be obtained by choosing: $LCA(3, 4, 5) = 2$.

Overall, $2 + 2 + 3 + 3 + 3 + 4 = 17$.

G. Mukhammadali and the Smooth Array

1 second, 256 megabytes

Muhammadali has an integer array $a_1, \dots, a_n$. He can change (replace) any subset of positions; changing position i costs c_i and replaces a_i with any integer of his choice. The positions that he does not change must retain their original values.

After all changes, we call an index i ($1 \leq i < n$) a *drop* if the final value at position i is strictly greater than the final value at position $i + 1$. Muhammadali wants the final array to contain no *drops*.

Find the minimum cost of changes required to ensure that there are no *drops* in the array.

Input

The first line contains an integer t ($1 \leq t \leq 5000$) — the number of test cases.

Each test case consists of three lines:

The first line contains a single integer n ($1 \leq n \leq 8000$) — the length of the arrays.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the elements of the array.

The third line contains n integers $c_1, c_2, \dots, c_n$ ($1 \leq c_i \leq 10^9$) — the costs of changes.

It is guaranteed that the sum of n across all test cases does not exceed 8000.

Output

For each test case, output a single integer — the minimum possible total cost required to eliminate all *drops*.

input
10 1 10 5 4 1 2 2 3 5 6 7 8 4 4 3 2 1 1 1 1 1 3 3 1 2 100 1 1 5 5 5 5 5 5 10 1 10 1 10 5 1 3 2 2 4 100 1 1 1 100 6 10 9 8 7 6 5 1 100 1 100 1 100 5 100 1 100 100 100 1 100 1 1 1 4 2 1 2 1 5 4 3 2 7 1 5 3 4 2 6 7 10 1 10 1 10 1 10
output
0 0 3 2 0 1 203 1 6 11

In the first and second examples, the array already has no *drops*, so no changes are needed.

In the third example, one of the optimal arrays is: $[2, 3, 5, 6]$; to achieve this, all elements except the second need to be replaced, so the answer is $c_1 + c_3 + c_4 = 3$.

Hints: Codeforces Round 1062 (Div. 4)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Square? (editorial, first line)

A square has four equal sides.

B. Your Name (editorial, first line)

Notice that we can rearrange one string into another only if each letter appears the same number of times in both strings.

C. Isamatdin and His Magic Wand! (editorial, first line)

First of all, notice that if all elements have the same parity, we cannot perform any swaps.

D. Yet Another Array Problem (official hints)

Hint 1: Answer is always prime number, try to prove why.

Hint 2: Notice how equation works if x does not satisfy property.

Hint 3: It means that for each i , $a_i \equiv 0 \pmod{x}$

E. khba Loves to Sleep! (editorial, first line)

Let the minimum time until the first arrival (that is, the minimum distance among all friends to the nearest teleport) be m .

F. Tree, TREE!!! (official hints)

Hint 1: We don't need to know *lowest common ancestor (LCA)* to solve this problem. Think of subtree sizes to replace *LCA*.

Hint 2: Let sz_i = subtree size of i when root is r . In order for i to be able to be in the S_r : $sz_i \geq k$ must hold.

Hint 3: Notice how instead of calculating the answer for each root, we can calculate contribution of i when root is $1 \dots n$.

Hint for bonus 1: Let us have the answer for dp_v = answer when root is v , let u be the directly connected node to v . How dp_v changes when we make root u (dp_u)?

G. Mukhammadali and the Smooth Array (official hints)

Hint 1: Think of longest non-decreasing sequence

Hint 2: In finding simple LNDS, all elements have the same "weights". But here we are given weights.

Source: <https://codeforces.com/blog/entry/147869>

[Codeforces](https://codeforces.com/) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform

Codeforces Round 1050 (Div. 4)

A. Sublime Sequence

1 second, 256 megabytes

Farmer John has an integer x . He creates a sequence of length n by alternating integers x and $-x$, starting with x .

For example, if $n = 5$, the sequence looks like: $x, -x, x, -x, x$.

He asks you to find the sum of all integers in the sequence.

Input

The first line contains an integer t ($1 \leq t \leq 100$) — the number of test cases.

The only line of input for each test case is two integers x and n ($1 \leq x, n \leq 10$).

Output

For each test case, output the sum of all integers in the sequence.

input
4
1 4
2 5
3 6
4 7
output
0
2
0
4

B. Lasers

2 seconds, 256 megabytes

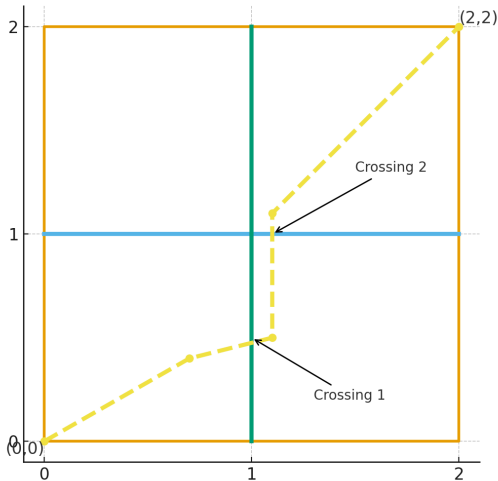
There is a 2D-coordinate plane that ranges from $(0, 0)$ to (x, y) . You are located at $(0, 0)$ and want to head to (x, y) .

However, there are n horizontal lasers, with the i -th laser continuously spanning $(0, a_i)$ to (x, a_i) . Additionally, there are also m vertical lasers, with the i -th laser continuously spanning $(b_i, 0)$ to (b_i, y) .

You may move in any direction to reach (x, y) , but your movement must be a continuous curve that lies inside the plane. Every time you cross a vertical or a horizontal laser, it counts as one crossing. Particularly, if you pass through an intersection point between two lasers, it counts as **two crossings**.

For example, if $x = y = 2, n = m = 1, a = [1], b = [1]$, the movement can be as follows:

Minimum crossings = 2 (must cross $x=1$ and $y=1$ once each)



What is the minimum number of crossings necessary to reach (x, y) ?

Input

The first line contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains four integers n, m, x , and y ($1 \leq n, m \leq 2 \cdot 10^5, 2 \leq x, y \leq 10^9$).

The following line contains n integers $a_1, a_2, \dots, a_n$ ($0 < a_i < y$) — the y-coordinates of the horizontal lasers. It is guaranteed that $a_i > a_{i-1}$ for all $i > 1$.

The following line contains m integers $b_1, b_2, \dots, b_m$ ($0 < b_i < x$) — the x-coordinates of the vertical lasers. It is guaranteed that $b_i > b_{i-1}$ for all $i > 1$.

It is guaranteed that the sum of n and m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the minimum number of crossings necessary to reach (x, y) .

input
2
1 1 2 2
1
1
2 1 100000 100000
42 58
32
output
2
3

C. Pacer

2 seconds, 256 megabytes

The FitnessGram Pacer Test is a multistage aerobic capacity test that progressively gets more difficult as it continues. The 20 meter pacer test will begin in 30 seconds. Line up at the start. A single lap should be completed every time you hear this sound. Ding! Remember to run in a straight line and run as long as possible. The test will begin on the word start. On your mark. Get ready!...

Farmer John is running the FitnessGram Pacer Test! Farmer John takes **one minute** to run to the other side of the gym. Therefore, at the start of each minute, FJ can choose to either run to the other side of the gym or stay in place. If he chooses to run to the other side of the gym, he gains **one point**.

FJ will run the Pacer Test until the start of the m -th minute. Initially (at the start of the 0-th minute), FJ is at the starting side of the gym, which we will denote as side 0. The opposite side of the gym is denoted side 1.

The pacer test audio plays n times. At the start of the a_i -th minute, FJ must be at the b_i -th side of the gym.

What is the maximum number of points FJ can acquire while ensuring that he meets the audio's requirements?

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \leq n \leq 2 \cdot 10^5, n \leq m \leq 10^9$) — the number of requirements and the number of total minutes.

The following n lines contain two integers a_i and b_i ($1 \leq a_i \leq m, b_i \in \{0, 1\}$) — the i -th requirement by the audio. It is guaranteed that $a_i > a_{i-1}$ over all $i > 1$.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the maximum number of points that FJ can acquire.

input
3 2 4 2 1 4 0 2 7 1 1 4 0 4 9 1 0 2 0 6 1 9 0
output
2 7 6

- For the first sample test case,
- During minute 0, FJ can stay at side 0.
 - During minute 1, FJ can run to side 1 and gain 1 point.
 - Right before minute 2, the audio requires FJ to be at side 1. Here, FJ is indeed at side 1.
 - During minute 2, FJ can run to side 0 and gain 1 point.
 - During minute 3, FJ can stay at side 0.
 - Right before minute 4, the audio requires FJ to be at side 0. Here, FJ is indeed at side 0.
 - Since the start of minute 4 has reached, the Pacer Test ends. His total score is 2.

Relevant illustration of the statement:



D. Destruction of the Dandelion Fields

2 seconds, 256 megabytes

Farmer John has a lawnmower, initially turned off. He also has n fields, with the i -th field having a_i dandelions. He will visit all the fields in any order he wants, and each field **exactly once**.

FJ's lawnmower seems to have a mind of its own. Right before visiting a field, it checks if the field has an even or odd number of dandelions. If it has an odd number, then the lawnmower toggles its state (if it is off, it turns on; if it is on, it turns off). Then, if the lawnmower is on, it will cut all dandelions in that field. Otherwise, if the lawnmower is off, then FJ will simply visit the field and cut no dandelions.

If FJ visits the n fields in optimal order, what is the maximum total number of dandelions he can cut?

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of fields.

The following line contains n space-separated integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the number of dandelions in each field.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output an integer on a new line: maximum dandelions FJ can cut if he visits all n fields in optimal order.

input
3 3 2 4 6 4 4 2 1 6 4 1000000000 999999999 1000000000 999999999
output
0 13 2999999999

For the first test case, since there is no field with an odd number of dandelions, FJ can never turn his lawnmower on. Since his lawnmower is always off, he can never cut any dandelions, so the answer is 0.

For the second test case, FJ can visit the third field first; then his lawnmower will turn on. Then he can visit the other fields in any order. Since his lawnmower is always on, dandelions in every field can be cut.

For the third test case, FJ can visit the fields in the following order: field 2, field 1, field 3, then field 4.

E. Split

2 seconds, 256 megabytes

Farmer John has an array a containing n positive integers and an integer k .

Let $a[l, r]$ be a subarray* of a . He performs the following procedure to independently determine if subarray $a[l, r]$ is awesome:

- Initially, FJ has k empty **multisets**, numbered from 1 to k .
- Then, for each element a_i ($1 \leq i \leq n$) in a :
 - If $l \leq i \leq r$ (that is, a_i is in the subarray $a[l, r]$), he places a_i in multiset 1,
 - Otherwise, he places a_i into any multiset he wants (**which may be multiset 1**).
- Subarray $a[l, r]$ is awesome if there is some way for him to place elements such that, for every value v , all multisets contain the same number of elements with value v . In other words, he wants to make all multisets contain the exact same elements (ignoring ordering).

Output the number of awesome subarrays.

*For array a of size n and integers $1 \leq l \leq r \leq n$, the subarray $a[l, r]$ denotes the array consisting of the elements $a_l, \dots, a_r$, in order.

Input

The first line contains an integer t ($1 \leq t \leq 1000$) — the number of test cases.

The first line of each test case contains two integers n and k ($2 \leq k \leq n \leq 2 \cdot 10^5$).

The following line contains n space-separated integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each testcase, output one integer on a new line: the number of awesome subarrays.

input
4 3 2 1 1 1 4 2 1 2 1 2 8 2 3 3 3 3 2 2 2 2 6 3 1 1 1 1 1 1
output
0 7 18 11

Test case 1: $n = 3, a = [1, 1, 1]$.

For $k = 2$, you cannot finish with the same number of 1's in both multisets, so no subarray works.

Test case 2: $n = 4, a = [1, 2, 1, 2]$.

For $k = 2$, the final state must give each multiset exactly one 1 and one 2. Thus a valid subarray can contain at most one 1 and at most one 2.

F. Gravity Falls

2 seconds, 256 megabytes

Farmer John has n arrays $a_1, a_2, \dots, a_n$ that may have different lengths. He will stack the arrays on top of each other, resulting in a grid with n rows. The arrays are left-aligned and can be stacked in any order he desires.

Then, gravity will take place. Any cell that is not on the bottom row and does not have an element beneath it will fall down a row. This process will be repeated until there are no cells that satisfy the aforementioned constraint.

Over all possible ways FJ can stack the arrays, output the lexicographically minimum bottom row after gravity takes place.

Input

The first line contains t ($1 \leq t \leq 1000$) — the number of test cases.

The first line of each test case contains n ($1 \leq n \leq 2 \cdot 10^5$).

For the following n lines, the first integer k_i ($1 \leq k_i \leq 2 \cdot 10^5$) denotes the length of a_i .

Then, k_i space-separated integers $a_{i_1}, a_{i_2}, \dots, a_{i_{k_i}}$ follow ($1 \leq a_{i_j} \leq 2 \cdot 10^5$).

It is guaranteed that the sum of n and the sum of all k_i over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the lexicographically minimum bottom row on a new line.

input
4 1 3 5 2 7 2 2 2 9 3 3 1 4 3 1 5 2 5 1 2 5 2 3 3 4 4 9 7 7 6 5 4 3 2 1 4 2 4 5 1
output
5 2 7 2 9 4 5 1 2 4 5 1 3 2 1

Demonstration for test case 2:

Initial stacking		
3	1	4
2	9	

After gravity		
3	1	
2	9	4

Bottom row: 2 9 4 (lexicographically minimum)

Demonstration for test case 4:

Initial stacking (top→bottom: 0, 1, 2)						
4	4	9				
7	6	5	4	3	2	1
2	4	5	1			

After gravity						
4	4	9				
7	6	5	4			
2	4	5	1	3	2	1

Bottom row: 2 4 5 1 3 2 1 (lexicographically minimum)

G. Farmer John's Last Wish

3 seconds, 256 megabytes

Bessie has found an array a of length n on the floor. There appears to be a handwritten note lying next to the array, seemingly written by Farmer John. The note reads:

Help me, dear Bessie! Let $f(a)$ denote the maximum integer k in the range $[1, n]$ such that $\gcd(a_1, a_2, \dots, a_k) > \gcd(a_1, a_2, \dots, a_{k+1})$, or 0 if no such k exists.

Bessie decides to help FJ. She defines $g(a)$ to represent the maximum value of $f(a)$ over all possible reorderings of a .

Bessie decides to not only find $g(a)$, but also the value of $g(p)$ for all prefixes p of a . Output n integers, the i 'th of which is $g([a_1, a_2, \dots, a_i])$.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$).

The following line contains n space-separated integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n integers on a new line: the i 'th of which should be $g([a_1, a_2, \dots, a_i])$.

input
3 8 2 4 3 6 5 7 8 6 6 6 6 6 6 6 9 8 4 2 6 3 9 5 7 8
output
0 1 2 3 3 3 4 5 0 0 0 0 0 0 0 1 2 2 4 4 4 4 5

Hints: Codeforces Round 1050 (Div. 4)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Sublime Sequence (editorial, first line)

Notice that if we pair each odd-indexed element with an even-indexed element, it sums to 0.

B. Lasers (editorial, first line)

Since our path is continuous, whatever route we take will always pass through all vertical and horizontal lasers in the region from $(0, 0)$ to (x, y) since our movement is bounded inside the region.

C. Pacer (editorial, first line)

In this problem, we want to maximize the amount of points between consecutive requirements.

D. Destruction of the Dandelion Fields (editorial, first line)

Firstly, if there is no odd farm, then the answer is 0 by default since we can never turn the lawnmower on.

E. Split (editorial, first line)

First, let's highlight the important parts of the statement.

F. Gravity Falls (editorial, first line)

Firstly, note the last row must have a length of $\max_k$, the maximum k among all arrays.

G. Farmer John's Last Wish (editorial, first line)

Let's start by understanding how $f(a)$ works.

Source: <https://codeforces.com/blog/entry/146112>

[Codeforces](https://codeforces.com/) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform

Codeforces Round 1017 (Div. 4)

A. Trippi Troppi

1 second, 256 megabytes

Trippi Troppi resides in a strange world. The ancient name of each country consists of three strings. The first letter of each string is concatenated to form the country's modern name.

Given the country's ancient name, please output the modern name.

Input

The first line contains an integer t – the number of independent test cases ($1 \leq t \leq 100$).

The following t lines each contain three space-separated strings. Each string has a length of no more than 10, and contains only lowercase Latin characters.

Output

For each test case, output the string formed by concatenating the first letter of each word.

input
7 united states america oh my god i cant lie binary indexed tree believe in yourself skibidi slay sigma god bless america
output
usa omg icl bit biy sss gba

B. Bobritto Bandito

1 second, 256 megabytes

In Bobritto Bandito's home town of residence, there are an infinite number of houses on an infinite number line, with houses at $\dots, -2, -1, 0, 1, 2, \dots$. On day 0, he started a plague by giving an infection to the unfortunate residents of house 0. Each succeeding day, the plague spreads to **exactly one** healthy household that is next to an infected household. It can be shown that each day the infected houses form a continuous segment.

Let the segment starting at the l -th house and ending at the r -th house be denoted as $[l, r]$. You know that after n days, the segment $[l, r]$ became infected. Find any such segment $[l', r']$ that could have been infected on the m -th day ($m \leq n$).

Input

The first line contains an integer t ($1 \leq t \leq 100$) – the number of independent test cases.

The only line of each test case contains four integers n, m, l , and r ($1 \leq m \leq n \leq 2000, -n \leq l \leq 0 \leq r \leq n, r - l = n$).

Output

For each test case, output two integers l' and r' on a new line. If there are multiple solutions, output any.

input
4 4 2 -2 2 4 1 0 4 3 3 -1 2 9 8 -6 3
output
-1 1 0 1 -1 2 -5 3

In the first test case, it is possible that on the 1-st, 2-nd, and 3-rd days the interval of houses affected is $[-1, 0]$, $[-1, 1]$, $[-2, 1]$. Therefore, $[-1, 1]$ is a valid output.

C. Brr Brrr Patapim

2 seconds, 256 megabytes

Brr Brrr Patapim is trying to learn of Tiramisù's secret passcode, which is a permutation* of $2 \cdot n$ elements. To help Patapim guess, Tiramisù gave him an $n \times n$ grid G , in which $G_{i,j}$ (or the element in the i -th row and j -th column of the grid) contains p_{i+j} , or the $(i + j)$ -th element in the permutation.

Given this grid, please help Patapim crack the forgotten code. It is guaranteed that the permutation exists, and it can be shown that the permutation can be determined uniquely.

*A permutation of m integers is a sequence of m integers which contains each of $1, 2, \dots, m$ exactly once. For example, $[1, 3, 2]$ and $[2, 1]$ are permutations, while $[1, 2, 4]$ and $[1, 3, 2, 3]$ are not.

Input

The first line contains an integer t — the number of test cases ($1 \leq t \leq 200$).

The first line of each test case contains an integer n ($1 \leq n \leq 800$).

Each of the following n lines contains n integers, giving the grid G . The first of these lines contains $G_{1,1}, G_{1,2}, \dots, G_{1,n}$; the second of these lines contains $G_{2,1}, G_{2,2}, \dots, G_{2,n}$, and so on. ($1 \leq G_{i,j} \leq 2 \cdot n$).

It is guaranteed that the grid encodes a valid permutation, and the sum of n over all test cases does not exceed 800.

Output

For each test case, please output $2n$ numbers on a new line: $p_1, p_2, \dots, p_{2n}$.

input
3 3 1 6 2 6 2 4 2 4 3 1 1 2 2 3 3 4
output
5 1 6 2 4 3 2 1 1 2 3 4

D. Tung Tung Sahur

2 seconds, 256 megabytes

You have two drums in front of you: a left drum and a right drum. A hit on the left can be recorded as "L", and a hit on the right as "R".

The strange forces that rule this world are fickle: sometimes, a blow sounds once, and sometimes it sounds twice. Therefore, a hit on the left drum could have sounded as either "L" or "LL", and a hit on the right drum could have sounded as either "R" or "RR".

The sequence of hits made is recorded in the string p , and the sounds heard are in the string s . Given p and s , determine whether it is true that the string s could have been the result of the hits from the string p .

For example, if $p = \text{"LR"}$, then the result of the hits could be any of the strings "LR", "LRR", "LLR", and "LLRR", but the strings "LLLR" or "LRL" cannot.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) – the number of independent test cases.

The first line of each test case contains the string p ($1 \leq |p| \leq 2 \cdot 10^5$) consisting only of the characters "R" and "L", where $|p|$ denotes the length of the string p .

The second line of each test case contains the string s ($1 \leq |p| \leq |s| \leq 2 \cdot 10^5$) consisting only of the characters "R" and "L".

It is guaranteed that the sum of $|s|$ does not exceed $2 \cdot 10^5$ across all test cases.

Output

For each set of input data, output "YES" if s can be the heard sound, and "NO" otherwise. You may output in any case.

input
5 R RR LRLR LRLR LR LLLR LLLLLRL LLLLRRLL LLRLRLRRLL LLLRLRRLLRRRL
output
YES YES NO NO YES

E. Boneca Ambalabu

2 seconds, 256 megabytes

Boneca Ambalabu gives you a sequence of n integers $a_1, a_2, \dots, a_n$.

Output the maximum value of $(a_k \oplus a_1) + (a_k \oplus a_2) + \dots + (a_k \oplus a_n)$ among all $1 \leq k \leq n$. Note that $\oplus$ denotes the bitwise XOR operation.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) – the number of independent test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) – the length of the array.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 2^{30}$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the maximum value on a new line.

input
5 3 18 18 18 5 1 2 4 8 16 5 8 13 4 5 15 6 625 676 729 784 841 900 1 1
output
0 79 37 1555 0

In the first test case, the best we can do is $(18 \oplus 18) + (18 \oplus 18) + (18 \oplus 18) = 0$.

In the second test case, we choose $k = 5$ to get $(16 \oplus 1) + (16 \oplus 2) + (16 \oplus 4) + (16 \oplus 8) + (16 \oplus 16) = 79$.

F. Trulimero Trulicina

2 seconds, 256 megabytes

Trulicina gives you integers n , m , and k . It is guaranteed that $k \geq 2$ and $n \cdot m \equiv 0 \pmod k$.

Output a n by m grid of integers such that each of the following criteria hold:

- Each integer in the grid is between 1 and k , inclusive.
- Each integer from 1 to k appears an equal number of times.
- No two cells that share an edge have the same integer.

It can be shown that such a grid always exists. If there are multiple solutions, output any.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) – the number of test cases.

The first line of each test case contains three integers n , m , and k ($2 \leq n \cdot m \leq 2 \cdot 10^5, 2 \leq k \leq n \cdot m, n \cdot m \equiv 0 \pmod k$).

It is guaranteed that the sum of $n \cdot m$ over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n lines, each containing m integers that satisfy the criteria. If there are multiple solutions, output any.

input
3 2 2 2 3 4 6 5 5 25
output
1 2 2 1 1 6 1 6 2 5 2 5 3 4 3 4 17 2 12 25 14 3 1 6 19 11 8 20 23 24 4 9 10 5 13 21 22 7 15 18 16

G. Chimpanzini Bananini

2 seconds, 256 megabytes

Chimpanzini Bananini stands on the brink of a momentous battle—one destined to bring finality.

For an arbitrary array b of length m , let's denote the *rizziness* of the array to be $\sum_{i=1}^m b_i \cdot i = b_1 \cdot 1 + b_2 \cdot 2 + b_3 \cdot 3 + \dots + b_m \cdot m$.

Chimpanzini Bananini gifts you an empty array. There are three types of operations you can perform on it.

- 1. Perform a cyclic shift on the array. That is, the array $[a_1, a_2, \dots, a_n]$ becomes $[a_n, a_1, a_2, \dots, a_{n-1}]$.
- 2. Reverse the entire array. That is, the array $[a_1, a_2, \dots, a_n]$ becomes $[a_n, a_{n-1}, \dots, a_1]$.
- 3. Append an element to the end of the array. The array $[a_1, a_2, \dots, a_n]$ becomes $[a_1, a_2, \dots, a_n, k]$ after appending k to the end of the array.

After each operation, you are interested in calculating the *rizziness* of your array.

Note that all operations are **persistent**. This means that each operation modifies the array, and subsequent operations should be applied to the current state of the array after the previous operations.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer q ($1 \leq q \leq 2 \cdot 10^5$) — the number of operations you perform on your array.

The following q lines first contain a single integer s ($1 \leq s \leq 3$) — the operation type.

- If $s = 1$, then the cyclic shift operation should be performed.
- If $s = 2$, then the reversal operation should be performed.
- If $s = 3$, then the line will contain an additional integer k ($1 \leq k \leq 10^6$), denoting the element appended to the back of the array.

It is guaranteed that the sum of q will not exceed $2 \cdot 10^5$ over all test cases. Additionally, it is guaranteed that the first operation on each test case will be one with $s = 3$.

Output

For each test case, output q lines, outputting the *rizziness* of your array after each operation.

input
1
13
3 1
3 2
3 3
1
3 4
2
3 5
1
3 6
2
3 7
2
1

output
1
5
14
11
27
23
48
38
74
73
122
102
88

The first six states of the array:

- $[1]$
- $[1, 2]$
- $[1, 2, 3]$
- $[3, 1, 2]$
- $[3, 1, 2, 4]$
- $[4, 2, 1, 3]$

H. La Vaca Saturno Saturnita

4 seconds, 256 megabytes

Saturnita's mood depends on an array a of length n , which only he knows the meaning of, and a function $f(k, a, l, r)$, which only he knows how to compute. Shown below is the pseudocode for his function $f(k, a, l, r)$.

```
function f(k, a, l, r):
    ans := 0
    for i from l to r (inclusive):
        while k is divisible by a[i]:
            k := k/a[i]
            ans := ans + k
    return ans
```

You are given q queries, each containing integers k, l , and r . For each query, please output $f(k, a, l, r)$.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and q ($1 \leq n \leq 10^5, 1 \leq q \leq 5 \cdot 10^4$).

The following line contains n integers $a_1, a_2, \dots, a_n$ ($2 \leq a_i \leq 10^5$).

The following q lines each contain three integers k, l , and r ($1 \leq k \leq 10^5, 1 \leq l \leq r \leq n$).

It is guaranteed that the sum of n does not exceed 10^5 over all test cases, and the sum of q does not exceed $5 \cdot 10^4$ over all test cases.

Output

For each query, output the answer on a new line.

input
2
5 3
2 3 5 7 11
2 1 5
2 2 4
2310 1 5
4 3
18 12 8 9
216 1 2
48 2 4
82944 1 4

output
5 6 1629 13 12 520

Hints: Codeforces Round 1017 (Div. 4)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Trippi Troppi (editorial, first line)

This can be solved by printing the zero-th index of each string in sequence.

B. Bobritto Bandito (official hint)

Hint: What is the condition for l' and r' to be valid?

C. Brr Brr Patapim (official hint)

Hint: How can we find p_i for $2 \leq i \leq n$? How can we find p_i for $n+1 \leq i \leq 2n$?

D. Tung Tung Sahur (official hint)

Hint: What would happen if s_1 consists of only L ?

E. Boneca Ambalabu (official hints)

Hint 1: Consider each bit independently.

Hint 2: Suppose we fix k . How can we compute the desired sum quickly?

Note that this may require preprocessing.

F. Trulimero Trulicina (official hint)

Hint: What would happen if we tried outputting the numbers $1, \dots, k, 1, \dots, k, \dots$ in the natural reading order? For example, for $n = 3$, $m = 4$, and $k = 6$, we would output the following:

1234

5612

3456

In which cases does this fail?

G. Chimpanzini Bananini (official hints)

Hint 1: Note that reversing an array then pushing an element to the back is similar to simply pushing an element to the front. Thus, we would like to push elements to both ends of the array. What data structure supports this?

Hint 2: What other values do we need to maintain in order to keep track of score?

H. La Vaca Saturno Saturnita (official hints)

Hint 1: What must the value of $a[i]$ be in order for k to change?

Hint 2: Can we use this to bound the number of times k changes?

Source: <https://codeforces.com/blog/entry/141796>

[Codeforces](https://codeforces.com/) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform

Codeforces Round 1003 (Div. 4)

A. Skibidus and Amog'u

1 second, 256 megabytes

Skibidus lands on a foreign planet, where the local Amog tribe speaks the Amog'u language. In *Amog'u*, there are two forms of nouns, which are *singular* and *plural*.

Given that the root of the noun is transcribed as S , the two forms are transcribed as:

- Singular: $S + \text{"us"}$
- Plural: $S + \text{"i"}$

Here, $+$ denotes *string concatenation*. For example, $abc + def = abcdef$.

For example, when S is transcribed as "amog", then the singular form is transcribed as "amogus", and the plural form is transcribed as "amogi". Do note that *Amog'u* nouns can have an **empty** root — in specific, "us" is the singular form of "i" (which, on an unrelated note, means "imposter" and "imposters" respectively).

Given a transcribed *Amog'u* noun in singular form, please convert it to the transcription of the corresponding plural noun.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). The description of the test cases follows.

The only line of each test case contains a string W , which is a transcribed *Amog'u* noun in *singular* form. It is guaranteed that W consists of only lowercase English letters, has a length of at most 10, and ends with "us".

Output

For each test case, output the transcription of the corresponding plural noun on a separate line.

input
9 us sus fungus cactus sussus amogus chungus ntarsus skibidus
output
i si fungi cacti sussi amogi chungi ntarsi skibidi

B. Skibidus and Ohio

1 second, 256 megabytes

Skibidus is given a string s that consists of lowercase Latin letters. If s contains more than 1 letter, he can:

- Choose an index i ($1 \leq i \leq |s| - 1$, $|s|$ denotes the current length of s) such that $s_i = s_{i+1}$. Replace s_i with any lowercase Latin letter of his choice. Remove s_{i+1} from the string.

Skibidus must determine the minimum possible length he can achieve through any number of operations.

Input

The first line contains an integer t ($1 \leq t \leq 100$) — the number of test cases.

The only line of each test case contains a string s ($1 \leq |s| \leq 100$). It is guaranteed s only contains lowercase Latin letters.

Output

For each test case, output an integer on the new line, the minimum achievable length of s .

input
4 baa skibidus cc ohio
output
1 8 1 4

In the first test case, Skibidus can:

- Perform an operation on $i = 2$. He replaces s_2 with b and removes s_3 from the string. Then, s becomes bb.
- Perform an operation on $i = 1$. He replaces s_1 with b and removes s_2 from the string. Then, s becomes b.
- Because s only contains 1 letter, Skibidus cannot perform any more operations.

Therefore, the answer is 1 for the first test case.

In the second test case, he cannot perform an operation on any index. Therefore, the answer is still the length of the initial string, 8.

C1. Skibidus and Fanum Tax (easy version)

2 seconds, 256 megabytes

This is the easy version of the problem. In this version, $m = 1$.

Skibidus has obtained two arrays a and b , containing n and m elements respectively. For **each** integer i from 1 to n , he is allowed to perform the operation **at most once**:

- Choose an integer j such that $1 \leq j \leq m$. Set $a_i := b_j - a_i$. Note that a_i may become non-positive as a result of this operation.

Skibidus needs your help determining whether he can sort a in non-decreasing order* by performing the above operation some number of times.

* a is sorted in non-decreasing order if $a_1 \leq a_2 \leq \dots \leq a_n$.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \leq n \leq 2 \cdot 10^5$, $m = 1$).

The following line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

The following line of each test case contains m integers $b_1, b_2, \dots, b_m$ ($1 \leq b_i \leq 10^9$).

It is guaranteed that the sum of n and the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, if it is possible to sort a in non-decreasing order, print "YES" on a new line. Otherwise, print "NO" on a new line.

You can output the answer in any case. For example, the strings "yEs", "yes", and "Yes" will also be recognized as positive responses.

input
5 1 1 5 9 3 1 1 4 3 3 4 1 1 4 2 5 6 4 1 5 4 10 5 4 3 1 9 8 7 8
output
YES NO YES NO YES

In the first test case, $[5]$ is already sorted.

In the second test case, it can be shown that it is impossible.

In the third test case, we can set $a_3 := b_1 - a_3 = 6 - 2 = 4$. The sequence $[1, 4, 4, 5]$ is in nondecreasing order.

In the last case, we can apply operations on each index. The sequence becomes $[-1, 0, 1]$, which is in nondecreasing order.

C2. Skibidus and Fanum Tax (hard version)

2 seconds, 256 megabytes

This is the hard version of the problem. In this version, $m \leq 2 \cdot 10^5$.

Skibidus has obtained two arrays a and b , containing n and m elements respectively. For **each** integer i from 1 to n , he is allowed to perform the operation **at most once**:

- Choose an integer j such that $1 \leq j \leq m$. Set $a_i := b_j - a_i$. Note that a_i may become non-positive as a result of this operation.

Skibidus needs your help determining whether he can sort a in non-decreasing order* by performing the above operation some number of times.

* a is sorted in non-decreasing order if $a_1 \leq a_2 \leq \dots \leq a_n$.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \leq n \leq 2 \cdot 10^5, 1 \leq m \leq 2 \cdot 10^5$).

The following line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

The following line of each test case contains m integers $b_1, b_2, \dots, b_m$ ($1 \leq b_i \leq 10^9$).

It is guaranteed that the sum of n and the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, if it is possible to sort a in non-decreasing order, print "YES" on a new line. Otherwise, print "NO" on a new line.

You can output the answer in any case. For example, the strings "yEs", "yes", and "Yes" will also be recognized as positive responses.

input
5 1 3 5 9 1 1000000000 3 2 1 4 3 3 4 4 3 2 4 6 5 6 1 8 5 2 6 4 5 4 5 4 1000 3 1 9 8 7 8
output
YES NO YES NO YES

In the first test case, $[5]$ is already sorted.

In the second test case, it can be shown that it is impossible.

In the third test case, we can set $a_2 := b_1 - a_2 = 6 - 4 = 2$ and $a_3 := b_3 - a_3 = 8 - 6 = 2$. The sequence $[2, 2, 2, 5]$ is in nondecreasing order.

In the last case, we can apply operations on each index. The sequence becomes $[-1, 0, 1]$, which is in nondecreasing order.

D. Skibidus and Sigma

2 seconds, 256 megabytes

Let's denote the score of an array b with k elements as $\sum_{i=1}^k \left(\sum_{j=1}^i b_j \right)$. In other words, let S_i denote the sum of the first i elements of b . Then, the score can be denoted as $S_1 + S_2 + \dots + S_k$.

Skibidus is given n arrays $a_1, a_2, \dots, a_n$, each of which contains m elements. Being the sigma that he is, he would like to concatenate them in **any order** to form a single array containing $n \cdot m$ elements. Please find the maximum possible score Skibidus can achieve with his concatenated array!

Formally, among all possible permutations* p of length n , output the maximum score of $a_{p_1} + a_{p_2} + \dots + a_{p_n}$, where $+$ represents concatenation†.

*A permutation of length n contains all integers from 1 to n exactly once.

†The concatenation of two arrays c and d with lengths e and f respectively (i.e. $c + d$) is $c_1, c_2, \dots, c_e, d_1, d_2, \dots, d_f$.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \leq n \cdot m \leq 2 \cdot 10^5$) — the number of arrays and the length of each array.

The i 'th of the next n lines contains m integers $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ($1 \leq a_{i,j} \leq 10^6$) — the elements of the i 'th array.

It is guaranteed that the sum of $n \cdot m$ over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the maximum score among all possible permutations p on a new line.

input
3 2 2 4 4 6 1 3 4 2 2 2 2 3 2 1 2 4 1 2 1 2 3 3 4 5 1 1 9
output
41 162 72

For the first test case, there are two possibilities for p :

- $p = [1, 2]$. Then, $a_{p_1} + a_{p_2} = [4, 4, 6, 1]$. Its score is $4 + (4 + 4) + (4 + 4 + 6) + (4 + 4 + 6 + 1) = 41$.
- $p = [2, 1]$. Then, $a_{p_1} + a_{p_2} = [6, 1, 4, 4]$. Its score is $6 + (6 + 1) + (6 + 1 + 4) + (6 + 1 + 4 + 4) = 39$.

The maximum possible score is 41.

In the second test case, one optimal arrangement of the final concatenated array is $[4, 1, 2, 1, 2, 2, 2, 2, 3, 2, 1, 2]$. We can calculate that the score is 162.

E. Skibidus and Rizz

1.5 seconds, 256 megabytes

With the approach of Valentine's Day, Skibidus desperately needs a way to rizz up his crush! Fortunately, he knows of just the way: creating the perfect Binary String!

Given a binary string* t , let x represent the number of 0 in t and y represent the number of 1 in t . Its **balance-value** is defined as the value of $\max(x - y, y - x)$.

Skibidus gives you three integers n , m , and k . He asks for your help to construct a binary string s of length $n + m$ with exactly n 0's and m 1's such that the maximum **balance-value** among all of its substrings[†] is **exactly** k . If it is not possible, output -1.

*A binary string only consists of characters 0 and 1.

†A string a is a substring of a string b if a can be obtained from b by the deletion of several (possibly, zero or all) characters from the beginning and several (possibly, zero or all) characters from the end.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first and only line of each test case contains three integers n , m , and k ($0 \leq n, m \leq 2 \cdot 10^5, 1 \leq k \leq n + m, n + m \geq 1$).

It is guaranteed that the sum of n and the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, if it is possible to construct s , output it on a new line. If there are multiple possible s , output any. Otherwise, output -1 on a new line.

input
6 1 2 1 4 3 2 2 4 3 8 3 2 5 0 4 5 0 5
output
101 0100101 011011 -1 -1 00000

In the first test case, we must construct s such that it contains one 0, two 1, and a maximum balance of 1 among all of its substrings. One possible valid s is 101 because:

- Consider the substring bounded by indices $[1, 1]$. Its **balance-value** is $\max(0 - 1, 1 - 0) = 1$.
- Consider the substring bounded by indices $[1, 2]$. Its **balance-value** is $\max(1 - 1, 1 - 1) = 0$.
- Consider the substring bounded by indices $[1, 3]$. Its **balance-value** is $\max(1 - 2, 2 - 1) = 1$.
- Consider the substring bounded by indices $[2, 2]$. Its **balance-value** is $\max(1 - 0, 0 - 1) = 1$.
- Consider the substring bounded by indices $[2, 3]$. Its **balance-value** is $\max(1 - 1, 1 - 1) = 0$.
- Consider the substring bounded by indices $[3, 3]$. Its **balance-value** is $\max(0 - 1, 1 - 0) = 1$.

Among all possible substrings, the maximum **balance-value** is 1.

In the second test case, the substring with the maximum **balance-value** is 0100, which has a balance of $\max(3 - 1, 1 - 3) = 2$.

F. Skibidus and Slay

4 seconds, 512 megabytes

Let's define the *majority* of a sequence of k elements as the unique value that appears strictly more than $\lfloor \frac{k}{2} \rfloor$ times. If such a value does not exist, then the sequence does **not** have a majority. For example, the sequence $[1, 3, 2, 3, 3]$ has a majority 3 because it appears $3 > \lfloor \frac{5}{2} \rfloor = 2$ times, but $[1, 2, 3, 4, 5]$ and $[1, 3, 2, 3, 4]$ do not have a majority.

Skibidus found a tree* of n vertices and an array a of length n . Vertex i has the value a_i written on it, where a_i is an integer in the range $[1, n]$.

For each i from 1 to n , please determine if there exists a non-trivial simple path[†] such that i is the *majority* of the **sequence of integers written on the vertices** that form the path.

*A tree is a connected graph without cycles.

†A sequence of vertices $v_1, v_2, \dots, v_m$ ($m \geq 2$) forms a non-trivial simple path if v_i and v_{i+1} are connected by an edge for all $1 \leq i \leq m - 1$ and all v_i are pairwise distinct. **Note that the path must consist of at least 2 vertices.**

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \leq n \leq 5 \cdot 10^5$) — the number of vertices.

The second line of each test case contains $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) — the integers written on the vertices.

Each of the next $n - 1$ lines contains two integers u_i and v_i , denoting the two vertices connected by an edge ($1 \leq u_i, v_i \leq n, u_i \neq v_i$).

It is guaranteed that the given edges form a tree.

It is guaranteed that the sum of n over all test cases does not exceed $5 \cdot 10^5$.

Output

For each test case, output a binary string s of length n on a separate line. s_i should be computed as follows:

- If there is a non-trivial path containing i as the majority, s_i is '1';
- Otherwise, s_i is '0'.

input
4 3 1 2 3 1 3 2 3 4 3 1 1 3 1 2 2 3 4 2 4 2 4 4 2 1 2 2 3 3 4 13 1 4 4 7 4 7 1 1 7 11 11 11 11 1 2 2 3 3 4 4 5 4 6 2 7 7 8 2 9 6 10 5 11 11 12 10 13
output
000 1010 0001 1001001000100

In the first test case, there is no non-trivial path with 1, 2, or 3 as a majority, so the binary string outputted is "000".

In the second test case, $1 \rightarrow 2 \rightarrow 4$ is a non-trivial path with 3 as a majority.

G. Skibidus and Capping

2 seconds, 256 megabytes

Skibidus was abducted by aliens of Amog! Skibidus tries to talk his way out, but the Amog aliens don't believe him. To prove that he is not totally capping, the Amog aliens asked him to solve this task:

An integer x is considered a *semi-prime* if it can be written as $p \cdot q$ where p and q are (not necessarily distinct) prime numbers. For example, 9 is a *semi-prime* since it can be written as $3 \cdot 3$, and 3 is a prime number.

Skibidus was given an array a containing n integers. He must report the number of pairs (i, j) such that $i \leq j$ and $\text{lcm}(a_i, a_j)^*$ is *semi-prime*.

*Given two integers x and y , $\text{lcm}(x, y)$ denotes the least common multiple of x and y .

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($2 \leq n \leq 2 \cdot 10^5$).

The following line contains n integers $a_1, a_2, \dots, a_n$ ($2 \leq a_i \leq n$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the number of ordered pairs of indices on a new line.

input
3 4 2 2 3 4 6 2 2 3 4 5 6 9 2 2 4 5 7 8 9 3 5
output
5 12 18

In the first test case, the 5 pairs of indices are (1, 3), (1, 4), (2, 3), (2, 4), and (4, 4).

H. Bro Thinks He's Him

3 seconds, 256 megabytes

Skibidus thinks he's Him! He proved it by solving this difficult task. Can you also prove yourself?

Given a binary string* t , $f(t)$ is defined as the minimum number of contiguous substrings, each consisting of identical characters, into which t can be partitioned. For example, $f(00110001) = 4$ because t can be partitioned as [00] [11] [000] [1] where each bracketed segment consists of identical characters.

Skibidus gives you a binary string s and q queries. In each query, a single character of the string is flipped (i.e. 0 changes to 1 and 1 changes to 0); changes are saved after the query is processed. After each query, output the sum over all $f(b)$ where b is a non-empty subsequence† of s , modulo 998 244 353.

*A binary string consists of only characters 0 and 1.

†A subsequence of a string is a string which can be obtained by removing several (possibly zero) characters from the original string.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a binary string s ($1 \leq |s| \leq 2 \cdot 10^5$).

The following line of each test case contains an integer q ($1 \leq q \leq 2 \cdot 10^5$) — the number of queries.

The following line contains q integers $v_1, v_2, \dots, v_q$ ($1 \leq v_i \leq |s|$), denoting s_{v_i} is flipped for the i 'th query.

It is guaranteed that the sum of $|s|$ and the sum of q over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output q integers on a single line — the answer after each query modulo 998 244 353.

input
3 101 2 1 3 10110 3 1 2 3 101110101 5 7 2 4 4 1

output

```
10 7
61 59 67
1495 1169 1417 1169 1396
```

In the first test case, s becomes `001` after the first query. Let's calculate the answer for each subsequence:

- $f(s_1) = f(0) = 1$
- $f(s_2) = f(0) = 1$
- $f(s_3) = f(1) = 1$
- $f(s_1 s_2) = f(00) = 1$
- $f(s_1 s_3) = f(01) = 2$
- $f(s_2 s_3) = f(01) = 2$
- $f(s_1 s_2 s_3) = f(001) = 2$

The sum of these values is 10, modulo 998 244 353.

Hints: Codeforces Round 1003 (Div. 4)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Skibidus and Amog'u (editorial, first line)

Let n be the length of the string.

B. Skibidus and Ohio (editorial, first line)

Note that if you can ever do an operation, the answer is 1.

C1. Skibidus and Fanum Tax (easy version) (editorial, first line)

The overall idea for both subtasks is that you need to iterate from left to right, and upon each element, pick the operation such that you obtain the smallest value of a_i that is greater than a_{i-1} .

C2. Skibidus and Fanum Tax (hard version) (editorial, first line)

The overall idea for both subtasks is that you need to iterate from left to right, and upon each element, pick the operation such that you obtain the smallest value of a_i that is greater than a_{i-1} .

D. Skibidus and Sigma (editorial, first line)

It is clear that the score of an array b of length n can be expressed as

$$n * b_1 + (n - 1) * b_2 + (n - 2) * b_3 \dots + 1 * b_n.$$

E. Skibidus and Rizz (editorial, first line)

Claim 1: It is impossible when $k < |n - m|$ Proof 1: The entire string will have a balance value greater than k .

F. Skibidus and Slay (editorial, first line)

Assume that a sequence a of length n contains two types of elements, 1 denoting majority and 0 denoting "not majority".

G. Skibidus and Capping (editorial, first line)

Let $a_i = x$ and $a_j = y$ for any pair (i, j) that we count.

H. Bro Thinks He's Him (editorial, first line)

A brute attempt at a solution would be to go through each non-empty subset of $1, 2, \dots, n$ and, for every pair of adjacent indices (i, j) in the subset, increase ans if $s[i] = s[j]$...

Source: <https://codeforces.com/blog/entry/139178>

[Codeforces](https://codeforces.com/) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform

Codeforces Round 993 (Div. 4)

A. Easy Problem

1 second, 256 megabytes

Cube is given an integer n . She wants to know how many ordered pairs of positive integers (a, b) there are such that $a = n - b$. Since Cube is not very good at math, please help her!

Input

The first line contains an integer t ($1 \leq t \leq 99$) — the number of test cases.

The only line of each test case contains an integer n ($2 \leq n \leq 100$).

Output

For each test case, output the number of ordered pairs (a, b) on a new line.

input
3 2 4 6
output
1 3 5

In the first test case, the only ordered pair that works is $(a, b) = (1, 1)$.
In the second test case, the three ordered pairs of (a, b) that work are $(3, 1), (2, 2), (1, 3)$.

B. Normal Problem

1 second, 256 megabytes

A string consisting of only characters 'p', 'q', and 'w' is painted on a glass window of a store. Ship walks past the store, standing directly in front of the glass window, and observes string a . Ship then heads inside the store, looks directly at the same glass window, and observes string b .

Ship gives you string a . Your job is to find and output b .

Input

The first line contains an integer t ($1 \leq t \leq 100$) — the number of test cases.

The only line of each test case contains a string a ($1 \leq |a| \leq 100$) — the string Ship observes from outside the store. It is guaranteed that a only contains characters 'p', 'q', and 'w'.

Output

For each test case, output string b , the string Ship observes from inside the store, on a new line.

input
5 qwq ppppp pppwwqqq wqpqwpqwwqp pqpqpqpq
output
pwp qqqqq pppwwqqq qpwwpqpqpqp pqpqpqpq

C. Hard Problem

1 second, 256 megabytes

Ball is the teacher in Paperfold University. The seats of his classroom are arranged in 2 rows with m seats each.

Ball is teaching $a + b + c$ monkeys, and he wants to assign as many monkeys to a seat as possible. Ball knows that a of them only want to sit in row 1, b of them only want to sit in row 2, and c of them have no preference. Only one monkey may sit in each seat, and each monkey's preference must be followed if it is seated.

What is the maximum number of monkeys that Ball can seat?

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

Each test case contains four integers m, a, b , and c ($1 \leq m, a, b, c \leq 10^8$).

Output

For each test case, output the maximum number of monkeys you can seat.

input
5 10 5 5 10 3 6 1 1 15 14 12 4 1 1 1 1 420 6 9 69
output
20 5 30 2 84

In the second test case, 6 monkeys want to sit in the front row, but only 3 seats are available. The monkeys that have no preference and the monkeys who prefer sitting in the second row can sit in the second row together. Thus, the answer is $3 + 2 = 5$.

D. Harder Problem

2 seconds, 256 megabytes

Given a sequence of positive integers, a positive integer is called a *mode* of the sequence if it occurs the maximum number of times that any positive integer occurs. For example, the mode of $[2, 2, 3]$ is 2. Any of 9, 8, or 7 can be considered to be a mode of the sequence $[9, 9, 8, 8, 7, 7]$.

You gave UFO an array a of length n . To thank you, UFO decides to construct another array b of length n such that a_i is a mode of the sequence $[b_1, b_2, \dots, b_i]$ for all $1 \leq i \leq n$.

However, UFO doesn't know how to construct array b , so you must help her. Note that $1 \leq b_i \leq n$ must hold for your array for all $1 \leq i \leq n$.

Input

The first line contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of a .

The following line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n numbers $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq n$) on a new line. It can be shown that b can always be constructed. If there are multiple possible arrays, you may print any.

input
4 2 1 2 4 1 1 1 2 8 4 5 5 5 1 1 2 1 10 1 1 2 2 1 1 3 3 1 1
output
1 2 1 1 2 2 4 5 5 1 1 2 2 3 1 8 2 2 1 3 3 9 1 1

Let's verify the correctness for our sample output in test case 2.

- At $i = 1$, 1 is the only possible mode of $[1]$.
- At $i = 2$, 1 is the only possible mode of $[1, 1]$.
- At $i = 3$, 1 is the only possible mode of $[1, 1, 2]$.
- At $i = 4$, 1 or 2 are both modes of $[1, 1, 2, 2]$. Since $a_i = 2$, this array is valid.

E. Insane Problem

2 seconds, 256 megabytes

Wave is given five integers k, l_1, r_1, l_2 , and r_2 . Wave wants you to help her count the number of ordered pairs (x, y) such that all of the following are satisfied:

- $l_1 \leq x \leq r_1$.
- $l_2 \leq y \leq r_2$.
- There exists a non-negative integer n such that $\frac{y}{x} = k^n$.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The only line of each test case contains five integers k, l_1, r_1, l_2 , and r_2 ($2 \leq k \leq 10^9, 1 \leq l_1 \leq r_1 \leq 10^9, 1 \leq l_2 \leq r_2 \leq 10^9$).

Output

For each test case, output the number of matching ordered pairs (x, y) on a new line.

input
5 2 2 6 2 12 2 1 1000000000 1 1000000000 3 5 7 15 63 1000000000 1 5 6 1000000000 15 17 78 2596 20914861
output
12 19999999987 6 1 197

In the third test case, the matching ordered pairs are the following:

- (5, 15)
- (5, 45)
- (6, 18)
- (6, 54)
- (7, 21)
- (7, 63)

In the fourth test case, the only valid ordered pair is (1, 1 000 000 000)

F. Easy Demon Problem

4 seconds, 256 megabytes

For an arbitrary grid, Robot defines its *beauty* to be the sum of elements in the grid.

Robot gives you an array a of length n and an array b of length m . You construct a n by m grid M such that $M_{i,j} = a_i \cdot b_j$ for all $1 \leq i \leq n$ and $1 \leq j \leq m$.

Then, Robot gives you q queries, each consisting of a single integer x . For each query, determine whether or not it is possible to perform the following operation **exactly** once so that M has a *beauty* of x :

1. Choose integers r and c such that $1 \leq r \leq n$ and $1 \leq c \leq m$
2. Set $M_{i,j}$ to be 0 for all ordered pairs (i, j) such that $i = r, j = c$, or both.

Note that queries are **not persistent**, meaning that you do not actually set any elements to 0 in the process — you are only required to output if it is possible to find r and c such that if the above operation is performed, the *beauty* of the grid will be x . Also, note that you must perform the operation for each query, even if the beauty of the original grid is already x .

Input

The first line contains three integers n, m , and q ($1 \leq n, m \leq 2 \cdot 10^5, 1 \leq q \leq 5 \cdot 10^4$) — the length of a , the length of b , and the number of queries respectively.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq |a_i| \leq n$).

The third line contains m integers $b_1, b_2, \dots, b_m$ ($0 \leq |b_i| \leq m$).

The following q lines each contain a single integer x ($1 \leq |x| \leq 2 \cdot 10^5$), the beauty of the grid you wish to achieve by setting all elements in a row and a column to 0.

Output

For each testcase, output "YES" (without quotes) if there is a way to perform the aforementioned operation such that the beauty is x , and "NO" (without quotes) otherwise.

You can output "YES" and "NO" in any case (for example, strings "yES", "yes" and "Yes" will be recognized as a positive response).

input
3 3 6 -2 3 -3 -2 2 -1 -1 1 -2 2 -3 3
output
NO YES NO NO YES NO

input
5 5 6 1 -2 3 0 0 0 -2 5 0 -3 4 -3 5 2 -1 2
output
YES YES YES YES NO YES

In the second example, the grid is

0 -2 5 0 -3

0 4 -10 0 6

0 -6 15 0 -9

0 0 0 0 0

0 0 0 0 0

By performing the operation with $r = 4$ and $c = 2$, we create the following grid:

0 0 5 0 -3

0 0 -10 0 6

0 0 15 0 -9

0 0 0 0 0

0 0 0 0 0

which has *beauty* 4. Thus, we output YES.

In the second query, selecting $r = 3$ and $c = 5$ creates a grid with *beauty* -3 .

In the third query, selecting $r = 3$ and $c = 3$ creates a grid with *beauty* 5.

G1. Medium Demon Problem (easy version)

2 seconds, 256 megabytes

This is the easy version of the problem. The key difference between the two versions is highlighted in bold.

A group of n spiders has come together to exchange plushies. Initially, each spider has 1 plushie. Every year, if spider i has at least one plushie, he will give exactly one plushie to spider r_i . Otherwise, he will do nothing. Note that all plushie transfers happen at the same time. **In this version, if any spider has more than 1 plushie at any point in time, they will throw all but 1 away.**

The process is *stable* in the current year if each spider has the same number of plushies (before the current year's exchange) as he did the previous year (before the previous year's exchange). Note that year 1 can never be *stable*.

Find the first year in which the process becomes *stable*.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($2 \leq n \leq 2 \cdot 10^5$) — the number of spiders.

The following line contains n integers $r_1, r_2, \dots, r_n$ ($1 \leq r_i \leq n, r_i \neq i$) — the recipient of the plushie of each spider.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output an integer on a new line, the first year in which the process becomes *stable*.

input
5
2
2 1
5
2 3 4 5 1
5
2 1 4 2 3
5
4 1 1 5 4
10
4 3 9 1 6 7 9 10 10 3
output
2
2
5
4
5

For the second test case:

- At year 1, the following array shows the number of plushies each spider has: [1, 1, 1, 1, 1]. Then, year 1's exchange happens.

- At year 2, the following array shows the number of plushies each spider has: [1, 1, 1, 1, 1]. Since this array is the same as the previous year, this year is *stable*.

For the third test case:

- At year 1, the following array shows the number of plushies each spider has: [1, 1, 1, 1, 1]. Then, year 1's exchange happens.
- At year 2, the following array shows the number of plushies each spider has: [1, 1, 1, 1, 0]. Then, year 2's exchange happens. Note that even though two spiders gave spider 2 plushies, spider 2 may only keep one plushie.
- At year 3, the following array shows the number of plushies each spider has: [1, 1, 0, 1, 0]. Then, year 3's exchange happens.
- At year 4, the following array shows the number of plushies each spider has: [1, 1, 0, 0, 0]. Then, year 4's exchange happens.
- At year 5, the following array shows the number of plushies each spider has: [1, 1, 0, 0, 0]. Since this array is the same as the previous year, this year is *stable*.

G2. Medium Demon Problem (hard version)

2 seconds, 256 megabytes

This is the hard version of the problem. The key difference between the two versions is highlighted in bold.

A group of n spiders has come together to exchange plushies. Initially, each spider has 1 plushie. Every year, if spider i has at least one plushie, he will give exactly one plushie to spider r_i . Otherwise, he will do nothing. Note that all plushie transfers happen at the same time. **In this version, each spider is allowed to have more than 1 plushie at any point in time.**

The process is *stable* in the current year if each spider has the same number of plushies (before the current year's exchange) as he did the previous year (before the previous year's exchange). Note that year 1 can never be *stable*.

Find the first year in which the process becomes *stable*.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($2 \leq n \leq 2 \cdot 10^5$) — the number of spiders.

The following line contains n integers $r_1, r_2, \dots, r_n$ ($1 \leq r_i \leq n, r_i \neq i$) — the recipient of the plushie of each spider.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output an integer on a new line, the first year in which the process becomes *stable*.

input
5
2
2 1
5
2 3 4 5 1
5
2 1 4 2 3
5
4 1 1 5 4
10
4 3 9 1 6 7 9 10 10 3
output
2
2
5
5
5

For the second test case:

- At year 1, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 1]$. Then, year 1's exchange happens.
- At year 2, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 1]$. Since this array is the same as the previous year, this year is *stable*.

For the third test case:

- At year 1, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 1]$. Then, year 1's exchange happens.
- At year 2, the following array shows the number of plushies each spider has: $[1, 2, 1, 1, 0]$. Then, year 2's exchange happens.
- At year 3, the following array shows the number of plushies each spider has: $[1, 3, 0, 1, 0]$. Then, year 3's exchange happens.
- At year 4, the following array shows the number of plushies each spider has: $[1, 4, 0, 0, 0]$. Then, year 4's exchange happens.
- At year 5, the following array shows the number of plushies each spider has: $[1, 4, 0, 0, 0]$. Since this array is the same as the previous year, this year is *stable*.

input
2
4 3
1 5 2 4
4 9 5 3
4 5 2 3
1 5 5 2
1 1 4 4
2 2 3 3
1 2 4 3
3 3
1 2 3
4 5 6
7 8 9
1 1 1 3
1 3 3 3
2 2 2 2
output
500 42 168
14 42 5

In the second query of the first test case, $A = [9, 5, 5, 2]$. Therefore, the sum is $1 \cdot 9 + 2 \cdot 5 + 3 \cdot 5 + 4 \cdot 2 = 42$.

H. Hard Demon Problem

3.5 seconds, 512 megabytes

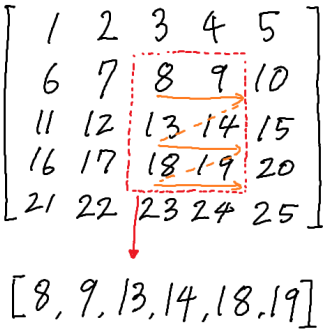
Swing is opening a pancake factory! A good pancake factory must be good at flattening things, so Swing is going to test his new equipment on 2D matrices.

Swing is given an $n \times n$ matrix M containing positive integers. He has q queries to ask you.

For each query, he gives you four integers x_1, y_1, x_2, y_2 and asks you to flatten the submatrix bounded by (x_1, y_1) and (x_2, y_2) into an array A .

Formally,
 $A = [M_{(x_1,y_1)}, M_{(x_1,y_1+1)}, \dots, M_{(x_1,y_2)}, M_{(x_1+1,y_1)}, M_{(x_1+1,y_1+1)}, \dots, M_{(x_2,y_2)}]$

The following image depicts the flattening of a submatrix bounded by the red dotted lines. The orange arrows denote the direction that the elements of the submatrix are appended to the back of A , and A is shown at the bottom of the image.



Afterwards, he asks you for the value of $\sum_{i=1}^{|A|} A_i \cdot i$ (sum of $A_i \cdot i$ over all i).

Input

The first line contains an integer t ($1 \leq t \leq 10^3$) — the number of test cases.

The first line of each test contains two integers n and q ($1 \leq n \leq 2000, 1 \leq q \leq 10^6$) — the length of M and the number of queries.

The following n lines contain n integers each, the i 'th of which contains $M_{(i,1)}, M_{(i,2)}, \dots, M_{(i,n)}$ ($1 \leq M_{(i,j)} \leq 10^6$).

The following q lines contain four integers x_1, y_1, x_2 , and y_2 ($1 \leq x_1 \leq x_2 \leq n, 1 \leq y_1 \leq y_2 \leq n$) — the bounds of the query.

It is guaranteed that the sum of n over all test cases does not exceed 2000 and the sum of q over all test cases does not exceed 10^6 .

Output

For each test case, output the results of the q queries on a new line.

Hints: Codeforces Round 993 (Div. 4)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Easy Problem (editorial, first line)

For any n , Cube can set a = any integer between 1 and $n - 1$ inclusive, and set $b = n - a$.

B. Normal Problem (editorial, first line)

The letters she reads that comprise string b are just the letters that comprise string a , flipped left-to-right.

C. Hard Problem (editorial, first line)

Let A , B , C be three sets of monkeys, such that monkeys in A can only sit in row 1, B in row 2, and C can sit anywhere.

D. Harder Problem (editorial, first line)

Observe that if you have an array where all elements are unique, they will all have frequency 1, therefore they can all be classified as the mode.

E. Insane Problem (editorial, first line)

Clearly, trying to bruteforce over all possible values of x or y is too slow, because the bounds are $1 \leq l_1 \leq r_1 \leq 10^9$.

F. Easy Demon Problem (editorial, first line)

Let's denote the beauty of the matrix as B , and denote SumA as the sum of all the elements in the array a , and SumB as the sum of all the elements in the array b .

G1. Medium Demon Problem (easy version) (editorial, first line)

This problem deals with a specific subclass of graphs called "functional graphs", also known as "successor graphs".

G2. Medium Demon Problem (hard version) (editorial, first line)

Note that similarly to G1, once all plushies end up in the hands of spiders who are in a loop, the process becomes stable.

H. Hard Demon Problem (editorial, first line)

Consider translating the sum back onto the matrix.

Source: <https://codeforces.com/blog/entry/137306>

[Codeforces](https://codeforces.com/) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform

Codeforces Round 952 (Div. 4)

A. Creating Words

1 second, 256 megabytes

Matthew is given two strings a and b , both of length 3. He thinks it's particularly funny to create two new words by swapping the first character of a with the first character of b . He wants you to output a and b after the swap.

Note that the new words may not necessarily be different.

Input

The first line contains t ($1 \leq t \leq 100$) — the number of test cases.

The first and only line of each test case contains two space-separated strings, a and b , both of length 3. The strings only contain lowercase Latin letters.

Output

For each test case, after the swap, output a and b , separated by a space.

input
6 bit set cat dog hot dog uwu owo cat cat zzz zzz
output
sit bet dat cog dot hog owu uwo cat cat zzz zzz

B. Maximum Multiple Sum

1 second, 256 megabytes

Given an integer n , find an integer x such that:

- $2 \leq x \leq n$.
- The sum of multiples of x that are less than or equal to n is maximized. Formally, $x + 2x + 3x + \dots + kx$ where $kx \leq n$ is maximized over all possible values of x .

Input

The first line contains t ($1 \leq t \leq 100$) — the number of test cases.

Each test case contains a single integer n ($2 \leq n \leq 100$).

Output

For each test case, output an integer, the optimal value of x . It can be shown there is only one unique answer.

input
2 3 15
output
3 2

For $n = 3$, the possible values of x are 2 and 3. The sum of all multiples of 2 less than or equal to n is just 2, and the sum of all multiples of 3 less than or equal to n is 3. Therefore, 3 is the optimal value of x .

For $n = 15$, the optimal value of x is 2. The sum of all multiples of 2 less than or equal to n is $2 + 4 + 6 + 8 + 10 + 12 + 14 = 56$, which can be proven to be the maximal over all other possible values of x .

C. Good Prefixes

2 seconds, 256 megabytes

Alex thinks some array is *good* if there exists some element that can be represented as the sum of all **other** elements (the sum of all other elements is 0 if there are no other elements). For example, the array $[1, 6, 3, 2]$ is good since $1 + 3 + 2 = 6$. Furthermore, the array $[0]$ is also good. However, the arrays $[1, 2, 3, 4]$ and $[1]$ are not good.

Alex has an array $a_1, a_2, \dots, a_n$. Help him count the number of good non-empty prefixes of the array a . In other words, count the number of integers i ($1 \leq i \leq n$) such that the length i prefix (i.e. $a_1, a_2, \dots, a_i$) is good.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of elements in the array.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$) — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer — the number of good non-empty prefixes of the array a .

input
7 1 0 1 1 4 1 1 2 0 5 0 1 2 1 4 7 1 1 0 3 5 2 12 7 1000000000 1000000000 1000000000 1000000000 1000000000 1000000000 294967296 10 0 1000000000 1000000000 1000000000 1000000000 1000000000 1000000000 1000000000 1000000000 589934592
output
1 0 3 3 4 1 2

In the fourth test case, the array has five prefixes:

- prefix $[0]$ is a good array, as mentioned in the statement;
- prefix $[0, 1]$ is not a good array, since $0 \neq 1$;
- prefix $[0, 1, 2]$ is not a good array, since $0 \neq 1 + 2$, $1 \neq 0 + 2$ and $2 \neq 0 + 1$;
- prefix $[0, 1, 2, 1]$ is a good array, since $2 = 0 + 1 + 1$;
- prefix $[0, 1, 2, 1, 4]$ is a good array, since $4 = 0 + 1 + 2 + 1$.

As you can see, three of them are good, so the answer is 3.

D. Manhattan Circle

2 seconds, 256 megabytes

Given a n by m grid consisting of '.' and '#' characters, there exists a whole manhattan circle on the grid. The top left corner of the grid has coordinates $(1, 1)$, and the bottom right corner has coordinates (n, m) .

Point (a, b) belongs to the manhattan circle centered at (h, k) if $|h - a| + |k - b| < r$, where r is a positive constant.

On the grid, the set of points that are part of the manhattan circle is marked as '#'. Find the coordinates of the center of the circle.

Input

The first line contains t ($1 \leq t \leq 1000$) — the number of test cases.

The first line of each test case contains n and m ($1 \leq n \cdot m \leq 2 \cdot 10^5$) — the height and width of the grid, respectively.

The next n lines contains m characters '.' or '#'. If the character is '#', then the point is part of the manhattan circle.

It is guaranteed the sum of $n \cdot m$ over all test cases does not exceed $2 \cdot 10^5$, and there is a whole manhattan circle on the grid.

Output

For each test case, output the two integers, the coordinates of the center of the circle.

input
6 5 5#.. 5 5 ..#.. .###. ##### .###. ..#.. 5 6#..... ###... .#..... 1 1 # 5 6 ...#.. ..###. .##### ..###. ...#.. 2 10#.....
output
3 3 3 3 4 2 1 1 3 4 2 4

E. Secret Box

1 second, 256 megabytes

Ntarsis has a box B with side lengths x , y , and z . It lies in the 3D coordinate plane, extending from $(0, 0, 0)$ to (x, y, z) .

Ntarsis has a secret box S . He wants to choose its dimensions such that all side lengths are positive integers, and the volume of S is k . He can place S somewhere within B such that:

- S is parallel to all axes.
- every corner of S lies on an integer coordinate.

S is magical, so when placed at an integer location inside B , it will not fall to the ground.

Among all possible ways to choose the dimensions of S , determine the **maximum** number of distinct locations he can choose to place his secret box S inside B . Ntarsis does not rotate S once its side lengths are selected.

Input

The first line consists of an integer t , the number of test cases ($1 \leq t \leq 2000$). The description of the test cases follows.

The first and only line of each test case contains four integers x, y, z and k ($1 \leq x, y, z \leq 2000, 1 \leq k \leq x \cdot y \cdot z$).

It is guaranteed the sum of all x , sum of all y , and sum of all z do not exceed 2000 over all test cases.

Note that k may not fit in a standard 32-bit integer data type.

Output

For each test case, output the answer as an integer on a new line. If there is no way to select the dimensions of S so it fits in B , output 0.

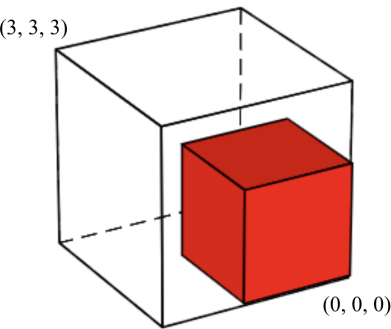
input
7 3 3 3 8 3 3 3 18 5 1 1 1 2 2 2 7 3 4 2 12 4 3 1 6 1800 1800 1800 4913000000
output
8 2 5 0 4 4 1030301

For the first test case, it is optimal to choose S with side lengths 2, 2, and 2, which has a volume of $2 \cdot 2 \cdot 2 = 8$. It can be shown there are 8 ways to put S inside B .

The coordinate with the least x , y , and z values for each possible arrangement of S are:

- (0, 0, 0)
- (1, 0, 0)
- (0, 1, 0)
- (0, 0, 1)
- (1, 0, 1)
- (1, 1, 0)
- (0, 1, 1)
- (1, 1, 1)

The arrangement of S with a coordinate of $(0, 0, 0)$ is depicted below:



For the second test case, S with side lengths 2, 3, and 3 are optimal.

F. Final Boss

2 seconds, 256 megabytes

You are facing the final boss in your favorite video game. The boss enemy has h health. Your character has n attacks. The i 'th attack deals a_i damage to the boss but has a cooldown of c_i turns, meaning the next time you can use this attack is turn $x + c_i$ if your current turn is x . Each turn, you can use all attacks that are not currently on cooldown, **all at once**. If all attacks are on cooldown, you do nothing for the turn and skip to the next turn.

Initially, all attacks are not on cooldown. How many turns will you take to beat the boss? The boss is beaten when its health is 0 or less.

Input

The first line contains t ($1 \leq t \leq 10^4$) – the number of test cases.

The first line of each test case contains two integers h and n ($1 \leq h, n \leq 2 \cdot 10^5$) – the health of the boss and the number of attacks you have.

The following line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 2 \cdot 10^5$) – the damage of your attacks.

The following line of each test case contains n integers $c_1, c_2, \dots, c_n$ ($1 \leq c_i \leq 2 \cdot 10^5$) – the cooldown of your attacks.

It is guaranteed that the sum of h and n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output an integer, the minimum number of turns required to beat the boss.

input
8
3 2
2 1
2 1
5 2
2 1
2 1
2 1
50 3
5 6 7
5 6 7
50 3
2 2 2
3 3 3
90000 2
200000 200000
1 1
100000 1
1
200000
6 7
3 2 3 2 3 1 2
6 5 9 5 10 7 7
21 6
1 1 1 1 1 1
5 5 8 10 7 6

output
1
3
15
25
1
19999800001
1
21

For the first test case, you can use attacks 1 and 2 on the first turn, dealing 3 damage in total, and slaying the boss.

For the second case, you can beat the boss in 3 turns by using the following attacks:

Turn 1: Use attacks 1 and 2, dealing 3 damage to the boss. The boss now has 2 health left.

Turn 2: Use attack 2, dealing 1 damage to the boss. The boss now has 1 health left.

Turn 3: Use attack 1, dealing 2 damage to the boss. The boss now has -1 health left. Since its health is less than or equal to 0, you beat the boss.

For the sixth test case: remember to use 64-bit integers as the answer can get large.

G. D-Function

2 seconds, 256 megabytes

Let $D(n)$ represent the sum of digits of n . For how many integers n where $10^l \leq n < 10^r$ satisfy $D(k \cdot n) = k \cdot D(n)$? Output the answer modulo $10^9 + 7$.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) – the number of test cases.

Each test case contains three integers l, r , and k ($0 \leq l < r \leq 10^9$, $1 \leq k \leq 10^9$).

Output

For each test case, output an integer, the answer, modulo $10^9 + 7$.

input
6
0 1 4
0 2 7
1 2 1
1 2 3
582 74663 3
0 3 1
output
2
3
90
12
974995667
999

For the first test case, the only values of n that satisfy the condition are 1 and 2.

For the second test case, the only values of n that satisfy the condition are 1, 10, and 11.

For the third test case, all values of n between 10 inclusive and 100 exclusive satisfy the condition.

H1. Maximize the Largest Component (Easy Version)

2 seconds, 512 megabytes

Easy and hard versions are actually different problems, so read statements of both problems completely and carefully. The only difference between the two versions is the operation.

Alex has a grid with n rows and m columns consisting of '.' and '#' characters. A set of '#' cells forms a connected component if from any cell in this set, it is possible to reach any other cell in this set by only moving to another cell in the set that shares a **common side**. The size of a connected component is the number of cells in the set.

In one operation, Alex selects any row r ($1 \leq r \leq n$) or any column c ($1 \leq c \leq m$), then sets every cell in row r or column c to be '#'. Help Alex find the maximum possible size of the largest connected component of '#' cells that he can achieve after performing the operation **at most once**.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \leq n \cdot m \leq 10^6$) — the number of rows and columns of the grid.

The next n lines each contain m characters. Each character is either '.' or '#'.

It is guaranteed that the sum of $n \cdot m$ over all test cases does not exceed 10^6 .

Output

For each test case, output a single integer — the maximum possible size of a connected component of '#' cells that Alex can achieve.

input
6 1 1 . 4 2 .. #. #. .# 3 5 .#.#. ..#.. .#.#. 5 5 #...## #...### 6 6 .#...# #...# .#...# #...# .#...# ###...# 6 8 ..#...# ####.# ###...# ...#...# ...#...# #...#...#
output
1 6 9 11 15 30

In the second test case, it is optimal for Alex to set all cells in column 2 to be '#'. Doing so will lead to the largest connected component of '#' having a size of 6.

In the third test case, it is optimal for Alex to set all cells in row 2 to be '#'. Doing so will lead to the largest connected component of '#' having a size of 9.

In the fourth test case, it is optimal for Alex to set all cells in row 4 to be '#'. Doing so will lead to the largest connected component of '#' having a size of 11.

H2. Maximize the Largest Component (Hard Version)

2 seconds, 512 megabytes

Easy and hard versions are actually different problems, so read statements of both problems completely and carefully. The only difference between the two versions is the operation.

Alex has a grid with n rows and m columns consisting of '.' and '#' characters. A set of '#' cells forms a connected component if from any cell in this set, it is possible to reach any other cell in this set by only moving to another cell in the set that shares a **common side**. The size of a connected component is the number of cells in the set.

In one operation, Alex selects any row r ($1 \leq r \leq n$) and any column c ($1 \leq c \leq m$), then sets every cell in row r and column c to be '#'. Help Alex find the maximum possible size of the largest connected component of '#' cells that he can achieve after performing the operation **at most once**.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \leq n \cdot m \leq 10^6$) — the number of rows and columns of the grid.

The next n lines each contain m characters. Each character is either '.' or '#'.

It is guaranteed that the sum of $n \cdot m$ over all test cases does not exceed 10^6 .

Output

For each test case, output a single integer — the maximum possible size of a connected component of '#' cells that Alex can achieve.

input
6 1 1 . 4 2 .. #. #. .# 3 5 .#.#. ..#.. ..#.. .#.. 5 5 #...# ...# #...### 6 6 .#...#. #...#.. .#...# #.#.#. .#.##. ###...# 6 8 ..#....# .####.#. ###.#. .# .##.#.## .#.##.## #..##.#.
output
1 7 11 16 22 36

In the fourth test case, it is optimal for Alex to set all cells in row 4 and column 2 to be '#'. Doing so will lead to the largest connected component of '#' having a size of 16.

In the fifth test case, it is optimal for Alex to set all cells in row 2 and column 4 to be '#'. Doing so will lead to the largest connected component of '#' having a size of 22.

Hints: Codeforces Round 952 (Div. 4)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Creating Words (editorial, first line)

To swap the first character of the strings, you can use the built-in method `std::swap` in C++, or for each string, separate the first character from the rest of the string and concatenate it with the other string.

B. Maximum Multiple Sum (editorial, first line)

To maximize the number of multiples of x less than n , it optimal to choose a small x , in this case, 2.

C. Good Prefixes (editorial, first line)

The only element that can be the sum of all other elements is the maximum element, since all elements are positive.

D. Manhattan Circle (editorial, first line)

Note that the manhattan circle is always in a diamond shape, symmetric from the center.

E. Secret Box (editorial, first line)

Since the side lengths of S has to multiply to k , all three side lengths of S has to be divisors of k .

F. Final Boss (editorial, first line)

Since the sum of h is bounded by $2 \cdot 10^5$, and each attack deals at least 1 damage.

G. D-Function (editorial, first line)

To satisfy $D(k \cdot n) = k \cdot D(n)$, each digit d in n must become $k \cdot d$ after multiplying n by k .

H1. Maximize the Largest Component (Easy Version) (editorial, first line)

Let's first solve the problem if we can only select and fill rows.

H2. Maximize the Largest Component (Hard Version) (editorial, first line)

For each row r and column c , we need to find the size of the component formed by filling both row r and column c (i.e. the size of the component containing row r and column c ...

Source: <https://codeforces.com/blog/entry/129620>

[Codeforces](https://codeforces.com/) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform

Codeforces Round 1122 (Div. 3)

A. Good Contest

1 second, 256 megabytes

The next programming contest has three problems and n participants.

Problem 1 is easy, problem 2 is medium, and problem 3 is hard.

A participant is called *weak* if they did not solve all three problems.

Unfortunately, the scoreboard was lost. The only remaining information is an array a of length 3, where a_i is the number of participants who solved problem i .

Among all scoreboards consistent with this information, find the minimum possible number of weak participants.

Input

The first line contains an integer t ($1 \leq t \leq 3000$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 9$) — the number of participants.

The second line of each test case contains three integers a_1, a_2, a_3 ($0 \leq a_i \leq n$), where a_i is the number of participants who solved problem i .

Output

For each test case, print a single integer — the minimum possible number of weak participants.

input
6 3 3 3 3 4 4 4 3 1 1 1 1 9 9 8 9 5 0 5 5 6 4 3 2
output
0 1 0 1 5 4

In the first test case, all 3 participants can have solved all three problems, so the answer is 0.

In the second test case, participant 1 could have solved only problems 1 and 2, while participants 2, 3, and 4 solved all three problems. Therefore, participant 1 is the only participant who is weak, so the answer is 1. It can be shown that this is minimal.

B. Three Piles

1 second, 256 megabytes

Alice and Bob are playing a game with three piles of stones. Initially, Alice has a stones, Bob has b stones, and the third pile contains c stones.

Alice and Bob take turns, with Alice going first. On each turn, the current player may take any number of stones from the third pile, possibly zero, and add them to their own pile.

If both players take zero stones on two consecutive turns, the game ends.

Let A and B be the final numbers of stones Alice and Bob have, respectively. The score of the game is $|A - B|$.

Alice wants to maximize the score, while Bob wants to minimize it. Assuming both players play optimally, find the final score.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

Each test case contains three integers a, b , and c ($0 \leq a, b, c \leq 10^9$) — the initial numbers of stones Alice has, Bob has, and the third pile has, respectively.

Output

For each test case, output one integer — the final score if both players play optimally.

It is important to use a 64-bit integer type, such as `long long` in C++.

input
5 3 6 3 3 6 10 5 5 4 2 5 6 67676767 41414141 998244353
output
3 7 4 3 1024506979

In the first test case, Alice can choose to take 0 stones on her first turn. Bob can then also take 0 stones, so the game ends with piles of sizes 3 and 6. Therefore, the score can be 3. It can be shown that Alice cannot achieve a larger score, and Bob cannot achieve a smaller score.

In the second test case, Alice can take all 10 stones from the third pile on her first turn. The game then ends with piles of sizes 13 and 6, so the score is 7.

C. AND, OR, Sort!

2 seconds, 256 megabytes

You are given a **binary string*** s of length n .

You may perform the following operation any number of times (possibly zero):

- choose an integer i ($1 \leq i \leq n$), and replace s_i with either the **bitwise AND** or the **bitwise OR** of $s_1, s_2, \dots, s_i$.

Note that the bitwise AND or bitwise OR of a single element is equal to the element itself.

Your goal is to make s sorted in non-decreasing order†.

Find the minimum number of operations required to sort s in non-decreasing order.

*A binary string only contains characters 0 and 1.

† If s is in non-decreasing order, then $s_1 \leq s_2 \leq \dots \leq s_n$.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \leq n \leq 2 \cdot 10^5$) — the length of the binary string s .

The second line of each test case contains the binary string s of length n . Each character of s is either 0 or 1.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer — the minimum number of operations required to sort s in non-decreasing order.

input
6 4 0011 4 1000 5 01000 8 01001101 7 0101010 7 0111101
output
0 3 1 2 3 1

In the first test case, the string is already sorted, so no operations are required.

In the second test case, we can use bitwise OR to change the last three characters to 1, obtaining 1111 in 3 operations.

D. Falling Concrete

2 seconds, 256 megabytes

Vihaan is repairing a road consisting of n sections. The height of the i -th section is a_i .

He has a forklift which can move road sections. In one operation, Vihaan chooses two indices i and j ($1 \leq i < j \leq n$), picks up the j -th section, and moves it backwards to position i .

As the section is carried backwards, one unit of concrete falls from it onto each section it passes over. Then, the carried section is inserted at position i .

More formally, the subarray

$[a_i, a_{i+1}, \dots, a_{j-1}, a_j]$

is replaced with

$[a_j - (j - i), a_i + 1, a_{i+1} + 1, \dots, a_{j-1} + 1].$

A part of the road is called *flat* if it is a contiguous segment of sections with equal heights.

Vihaan may perform any number of operations, possibly zero.

Find the maximum possible length of a flat part of the road.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of sections of the road.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($n \leq a_i \leq 10^9$) — the initial heights of the sections.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

It can be shown that under the given constraints, the height of every section remains positive after any sequence of operations.

Output

For each test case, print a single integer — the maximum possible length of a flat part of the road.

input
8 5 5 5 9 8 6 6 6 6 6 6 5 5 6 7 8 9 5 9 7 12 10 12 4 4 7 5 8 7 14 9 14 12 8 11 12 5 1000000000 1000000000 1000000000 1000000000 1000000000 8 8 8 12 8 14 10 15 13
output
4 6 1 5 4 2 5 6

In the first test case, Vihaan can move the section at position 4 to position 1:

$[5, 5, 5, 9, 8] \rightarrow [6, 6, 6, 6, 8].$

Thus, a flat part of length 4 can be created.

In the fourth test case, Vihaan can perform the following operations:

- 1. Move the section at position 3 to position 1:

$[9, 7, 12, 10, 12] \rightarrow [10, 10, 8, 10, 12].$

- 2. Move the section at position 4 to position 3:

$[10, 10, 8, 10, 12] \rightarrow [10, 10, 9, 9, 12].$

- 3. Move the section at position 5 to position 3:

$[10, 10, 9, 9, 12] \rightarrow [10, 10, 10, 10, 10].$

Thus, the entire road can be made flat.

In the fifth test case, Vihaan can first move the section at position 2 to position 1:

$[4, 7, 5, 8] \rightarrow [6, 5, 5, 8].$

Then, he can move the section at position 4 to position 2:

$[6, 5, 5, 8] \rightarrow [6, 6, 6, 6].$

Thus, the entire road can be made flat.

E. Prime Destruction

2 seconds, 256 megabytes

You are given a **multiset** a consisting of n positive integers.

You may perform the following operation any number of times (possibly zero):

- Choose an integer $x > 1$ from the multiset and a prime divisor p of x . Remove one occurrence of x from the multiset and add p copies of $\frac{x}{p}$.

You are also given an integer k ($1 \leq k \leq n$). Let $f(k)$ be the minimum number of operations required, starting from the original multiset, until every integer in the multiset is at most k .

Find $f(k)$.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and k ($1 \leq k \leq n \leq 2 \cdot 10^5$) — the initial size of the multiset and the given integer, respectively.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) — the elements of the multiset.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer $f(k)$.

input
6 1 1 1 6 2 6 6 4 3 2 1 8 1 8 6 4 3 2 1 8 6 12 3 12 10 9 8 7 6 5 4 3 2 1 12 10 9 10 9 8 7 6 5 4 3 2 1 5 5 5 4 3 2 1
output
0 4 25 15 1 0

In the first test case, the only element is already at most k , so no operations are needed.

In the second test case, we can perform the following operations:

- $[6, 6, 4, 3, 2, 1] \rightarrow [2, 2, 2, 6, 4, 3, 2, 1]$,
 $[2, 2, 2, 6, 4, 3, 2, 1] \rightarrow [2, 2, 2, 2, 2, 2, 4, 3, 2, 1]$,
 $[2, 2, 2, 2, 2, 2, 4, 3, 2, 1] \rightarrow [2, 2, 2, 2, 2, 2, 2, 2, 3, 2, 1]$,
 $[2, 2, 2, 2, 2, 2, 2, 2, 3, 2, 1] \rightarrow [2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 2, 1]$.

Thus, 4 operations are sufficient. It can be shown that no sequence with strictly fewer operations exists.

F. MEX Replacement

2 seconds, 256 megabytes

You are given a multiset of non-negative integers in compressed form. That is, for each value x_i in the multiset, you know the number of its occurrences y_i .

In one operation, you may choose any non-empty collection of elements currently in the multiset, remove all chosen elements, and insert one element equal to their MEX*.

You may perform the operation any number of times, possibly zero.

Find the maximum integer that can appear in the multiset at any moment.

*The minimum excluded (MEX) of a collection of integers $c_1, c_2, \dots, c_k$ is the smallest non-negative integer x that does not occur in the collection.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of distinct integers in the initial multiset.

Each of the next n lines contains two integers x_i and y_i ($0 \leq x_i \leq 10^9$, $1 \leq y_i \leq 10^9$), indicating that the integer x_i occurs exactly y_i times in the initial multiset.

The integers $x_1, x_2, \dots, x_n$ are pairwise distinct.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output one integer — the maximum integer that can appear in the multiset at any moment.

input
10 4 2 2 0 3 4 1 1 2 1 10 1 1 2 3 3 0 1 2 2 3 1 3 0 4 1 2 2 1 1 0 100 2 2 7 3 1 2 0 1 3 3 3 0 1 1 1 59 1 4 0 5 1 1 2 3 3 1
output
5 10 3 4 4 7 4 3 59 5

In the first test case, the initial multiset is $\{0, 0, 0, 1, 1, 2, 2, 4\}$.

We can replace $\{0, 1, 2\}$ with 3, obtaining $\{0, 0, 1, 2, 3, 4\}$. Then, we can replace $\{0, 1, 2, 3, 4\}$ with 5.

It can be shown that no integer greater than 5 can appear, so the answer is 5.

In the second test case, 10 already appears in the initial multiset, and no greater integer can appear. Thus, the answer is 10.

G. Modular Tree

3 seconds, 256 megabytes

Vihaan has a rooted tree* with n nodes. The tree is rooted at node 1.

Each node i has an initial value a_i and a modulus b_i . Let x_i denote the **current** value of node i . Initially, $x_i = a_i$.

Vihaan may perform the following operation any number of times:

- Choose a node u . Let s be the sum of the **current** values of all direct children of u , then replace x_u with $(x_u + s) \bmod b_u$.

After performing any number of operations, Vihaan wants to maximize the sum of the values of all nodes.

Determine the maximum possible sum.

*A tree is an undirected connected graph in which there are no cycles.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of nodes in the tree.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < b_i$) — the initial values of the nodes.

The third line contains n integers $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$) — the moduli of the nodes.

Each of the next $n - 1$ lines contains two integers u and v ($1 \leq u, v \leq n$) — an edge between nodes u and v .

It is guaranteed that the given edges form a tree.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print one integer — the maximum possible sum of the values of all nodes after performing any number of operations.

input
8 1 3 7 2 0 3 5 4 1 2 3 0 2 3 7 3 4 1 2 2 3 3 0 0 1 5 2 2 1 2 2 3 4 1 2 3 4 10 3 4 5 1 2 1 3 1 4 3 0 1 3 10 2 4 1 2 1 3 5 0 0 1 2 3 12 6 9 3 4 1 2 1 3 2 4 3 5 4 0 999999999 999999999 999999999 1000000000 1000000000 1000000000 1000000000 1 2 1 3 1 4
output
3 7 11 6 18 12 27 3999999996

In the first test case, node 1 has no children, so performing an operation on it does not change its value. Thus, the maximum possible sum is 3.

In the third test case, Vihaan can perform the operation on node 1 three times:

- $[0, 2, 3] \rightarrow [2, 2, 3]$.
- $[2, 2, 3] \rightarrow [4, 2, 3]$.
- $[4, 2, 3] \rightarrow [6, 2, 3]$.

The resulting sum is $6 + 2 + 3 = 11$. It can be shown that no sequence of operations can obtain a larger sum.

In the fourth test case, Vihaan can perform the following operations:

- Perform the operation on node 2:

$[0, 0, 1] \rightarrow [0, 1, 1]$.

- Perform the operation on node 1 four times:

$[0, 1, 1] \rightarrow [1, 1, 1] \rightarrow [2, 1, 1] \rightarrow [3, 1, 1] \rightarrow [4, 1, 1]$.

The resulting sum is $4 + 1 + 1 = 6$. It can be shown that no sequence of operations can obtain a larger sum.

H. Deque Malfunction

3 seconds, 256 megabytes

Vihaan built a **deque** on his own, but it sometimes malfunctions.

Vihaan wants the final contents of the deque, from front to back, to be $[1, 2, \dots, n]$.

You are given an array b of length m . The elements arrive one at a time in the order $b_1, b_2, \dots, b_m$.

Whenever an element b_i arrives, Vihaan must perform exactly one of the following operations:

- push b_i to the front of the deque.
- push b_i to the back of the deque.
- make the deque malfunction and insert b_i at any position of his choice in the deque.

Vihaan may also perform either of the following operations at **any** time:

- remove the element at the front of the deque.
- remove the element at the back of the deque.

After all m elements have arrived and all operations have been performed, the deque must contain exactly $[1, 2, \dots, n]$ in that order.

Determine the minimum number of times the deque must malfunction to create $[1, 2, \dots, n]$, or determine that it is impossible.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \leq n \leq m \leq 2 \cdot 10^5$) — the length of the desired deque and the number of arriving elements.

The second line contains m integers $b_1, b_2, \dots, b_m$ ($1 \leq b_i \leq n$) — the order in which the elements arrive.

It is guaranteed that the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print one integer — the minimum number of times the deque must malfunction.

If it is impossible to obtain the desired deque, print -1 .

input
9
1 3
1 1 1
3 4
2 1 1 2
4 4
1 4 2 3
4 5
4 3 4 2 1
5 8
1 3 2 5 4 3 1 2
6 6
1 6 5 4 3 2
4 6
1 3 2 3 4 3
6 8
2 6 3 5 2 1 4 2
6 8
1 6 2 5 1 5 3 4

output
0
-1
2
0
1
4
0
2
3

In the first test case, Vihaan can perform the following operations:

1. Push the first 1 to the front:

$[] \rightarrow [1]$.

Then, immediately pop it:

$[1] \rightarrow []$.

2. Push the second 1 to the front:

$[] \rightarrow [1]$.

Then, immediately pop it:

$[1] \rightarrow []$.

3. Push the last 1 to the back:

$[] \rightarrow [1]$.

Thus, no malfunctions are required.

In the second test case, the value 3 never appears in b , so it is impossible to obtain the desired deque.

In the third test case, Vihaan can perform the following operations:

1. Push 1 to the back:

$[] \rightarrow [1]$.

2. Push 4 to the back:

$[1] \rightarrow [1, 4]$.

3. Make the deque malfunction and insert 2 between 1 and 4:

$[1, 4] \rightarrow [1, 2, 4]$.

4. Make the deque malfunction and insert 3 between 2 and 4:

$[1, 2, 4] \rightarrow [1, 2, 3, 4]$.

It can be shown that no sequence of operations using fewer than 2 malfunctions can obtain the desired deque. Thus, the minimum is 2. In the fourth test case, Vihaan can perform the following operations:

1. Push the first occurrence of 4 to the front:

$[] \rightarrow [4]$.

Then, immediately pop it:

$[4] \rightarrow []$.

2. Push 3 to the back:

$[] \rightarrow [3]$.

3. Push the second occurrence of 4 to the back:

$[3] \rightarrow [3, 4]$.

4. Push 2 to the front:

$[3, 4] \rightarrow [2, 3, 4]$.

5. Push 1 to the front:

$$[2, 3, 4] \rightarrow [1, 2, 3, 4].$$

Thus, no malfunctions are required.

Hints: Codeforces Round 1122 (Div. 3)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Good Contest (official hint)

Hint: How many participants can possibly have solved all three problems?

B. Three Piles (official hints)

Hint 1: Consider the cases $a \geq b$ and $a < b$ separately.

Hint 2: If Alice ever takes stones, she should take all of them.

C. AND, OR, Sort! (official hints)

Hint 1: If a prefix contains both 0 and 1, what are its bitwise AND and bitwise OR?

Hint 2: A sorted binary string is determined by one split point.

D. Falling Concrete (official hints)

Hint 1: Look at how the height and position of each section change after an operation.

Hint 2: A section moving one position to the right gains 1 height, while a section moving one position to the left loses 1 height.

E. Prime Destruction (official hint)

Hint: Use DP. Let dp_i be the minimum number of operations needed to turn one i into elements which are all at most k .

F. MEX Replacement (official hints)

Hint 1: Work backwards from one copy of the value you want to create.

Hint 2: Process the values from large to small, and let q be the number of copies of the current value that you need.

Hint 3: If you need q copies of x but only have $c < q$, the missing $q - c$ copies of x each force another copy of every smaller value.

G. Modular Tree (official hints)

Hint 1: Process the tree from the leaves upward.

Hint 2: A node can always be brought back to its original value after changing it.

Hint 3: For every node i , its reachable values have the form $(a_i + g_i x) \bmod b_i$ for some integer x . Find g_i .

H. Deque Malfunction (official hints)

Hint 1: We only need to keep one occurrence of every value $1, 2, \dots, n$. What can we do with every other occurrence?

Hint 2: Let t_i be the arrival position of the occurrence of i which stays in the final deque. When can i be inserted without a malfunction?

Hint 3: An element can be inserted normally exactly when t_i is a prefix minimum or a suffix minimum of $t_1, t_2, \dots, t_n$.

Hint 4: Fix the position of the global minimum. The prefix minima and suffix minima can be optimized separately.

Source: <https://codeforces.com/blog/entry/156984>

Codeforces Round 1119 (Div. 3)

A. Moo Language School

1 second, 256 megabytes

Farmer John is trying to increase literacy rates in the United Cows of Farmer John (UCFJ). The UCFJ consists of n fields and $\frac{n}{k}$ farms (where n is a multiple of k), with each farm consisting of k consecutive fields. In other words, the i -th field is in the $\lceil \frac{i}{k} \rceil$ -th farm: Fields $1, 2, \dots, k$ are in the first farm, fields $k + 1, k + 2, \dots, 2k$ are in the second farm, etc.

Farmer John wants to build schools such that each farm has at least one school. However, some fields are owned by Farmer Nhoj, who will charge Farmer John extra to build a school there. Farmer John wants to know the minimum number of times that he would have to build a school on Farmer Nhoj's land in order to ensure that each farm has at least one school.

Input

The first line of each input contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and k ($1 \leq k \leq n \leq 20$, n is a multiple of k) — the number of fields and size of each farm.

The second line of each test case contains a binary string s of length n — the fields owned by Farmer Nhoj. If $s_i = 1$, the i -th field is owned by Farmer Nhoj. If $s_i = 0$, the i -th field is not owned by Farmer Nhoj.

Output

For each test case, output a single integer — the minimum number of times that Farmer John must build a school on Farmer Nhoj's land.

input
6 8 2 10011100 5 1 11111 8 4 01111110 5 1 00101 4 4 1101 4 4 1111
output
1 5 0 2 0 1

For the first test case, we can build a school on the 2nd, 3rd, 5th, and 7th fields, and of those, only the 5th field is owned by Farmer Nhoj, meaning our answer is 1. It can be shown that this is the best possible answer.

For the second test case, Farmer Nhoj owns every field, and since we have to build 5 schools, we must build on Farmer Nhoj's land 5 times.

B. Minus Two

2 seconds, 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$. You may perform the following operation:

- For all indices i ($1 \leq i \leq n$), set $a_i = |a_i - 2|$.

Find the maximum possible frequency of any integer in a after performing the operation an arbitrary number (possibly zero) of times.

Input

The first line of each input contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of a .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer denoting the maximum possible frequency of any integer in a after performing the operation an arbitrary number of times.

input
5 2 1 3 4 1 1 1 2 3 6 7 8 4 2 2 2 2 5 1 10 100 1000 100000
output
2 3 1 4 3

In the first test case, we can perform the operation once to have $a = [|1 - 2|, |3 - 2|] = [1, 1]$, which means the maximum possible frequency of any integer in a is 2.

In the second test case, the maximum frequency of an integer in a is 3. It can be shown that, no matter how many operations we perform, the maximum possible frequency of an integer in a will never exceed 3.

C. 101

2 seconds, 256 megabytes

The score of an array b of length m is defined as the maximum length of a subarray of b such that the first and last elements of the subarray are equal to 1 and all other elements in the subarray are equal to 0. Formally, the score of b is equal to the maximum integer k for which there exists an index i such that:

- $1 \leq i \leq m - k + 1$
- $b_i = b_{i+k-1} = 1$
- $b_{i+1} = b_{i+2} = \dots = b_{i+k-2} = 0$

If there is no subarray meeting the requirements, the score of b is 0.

You are given an array $a_1, a_2, \dots, a_n$, such that each element is equal to one of $-1, 0$, or 1 . Replace each -1 with either a 0 or 1 such that the score of a is maximal over all possible ways to replace the -1 s in a .

Input

The first line of each input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($1 \leq n \leq 2 \cdot 10^5$) — the length of a .

The second line of each test case contains $a_1, a_2, \dots, a_n$ ($a_i \in \{-1, 0, 1\}$) — the array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n space separated integers representing a after the -1 s were replaced with 0s or 1s. If there are multiple possible solutions, output any.

input
10 6 1 0 -1 0 0 1 7 0 -1 0 0 1 0 1 5 -1 0 0 -1 0 4 0 0 0 0 1 -1 6 1 0 1 0 0 -1 7 0 1 0 0 0 1 0 6 -1 -1 -1 -1 -1 -1 7 -1 0 1 -1 0 0 1 3 -1 0 0
output
1 0 0 0 0 1 0 1 0 0 1 0 1 1 0 0 1 0 0 0 0 0 1 1 0 1 0 0 1 0 1 0 0 0 1 0 1 0 0 0 0 1 0 0 1 0 0 0 1 1 0 0

In the first test case, we can change the only -1 to a 0 , making $a = [1, 0, 0, 0, 0, 1]$. Since the first and last elements of a are equal to 1 , and all other elements are 0 , the score of a is 6 .

In the third test case, changing both -1 s to 1 s makes $a = [1, 0, 0, 1, 0]$, and the largest subarray that satisfies the conditions in the statement is from the 1 -st index to the 4 -th index.

In the fifth test case, we set the only -1 to 1 , making $a = [1]$, meaning the largest subarray that satisfies the conditions in the statement is the full array.

D. MEX Multiset

2 seconds, 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$. There exist 3 initially empty multisets A, B, C , and for each index i ($1 \leq i \leq n$), you may put a_i into **exactly one** of A, B , or C .

Determine whether it is possible to put the elements into the multisets such that $\text{MEX}(A) + \text{MEX}(B) + \text{MEX}(C) \geq 2 \cdot \max(\text{MEX}(A), \text{MEX}(B), \text{MEX}(C))$.
*. If so, output a construction that achieves this.

* $\text{MEX}(D)$ is defined as the smallest non-negative integer that is not present in the set D . For example, $\text{MEX}([1, 2, 0, 5]) = 3$, and $\text{MEX}([1, 2, 4, 9]) = 0$. The MEX of an empty set is 0 .

Input

The first line of each input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($3 \leq n \leq 2 \cdot 10^5$) — the length of a .

The second line of each test case contains $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$) — the array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

If a valid distribution of elements into the multisets exists, output **YES**. Otherwise, output **NO**.

If the answer is **YES**, output a string s of length n on a new line, such that $s_i = \mathbf{A}$ if the i -th element was put into the multiset A , $s_i = \mathbf{B}$ if the i -th element was put into the multiset B , and $s_i = \mathbf{C}$ if the i -th element was put into the multiset C .

You can output the answer in any case (upper or lower). For example, the strings **YES**, **yes**, **yEs**, and **Yes** will be recognized as positive responses, and the strings **NO**, **no**, **No** will be recognized as negative responses. Additionally, the strings **AABCAAA**, **aabcaaa**, and **aaBcaaa** will be recognized as the same answer.

If there are multiple possible outputs, output any.

input
5 6 1 0 0 1 2 1 4 0 0 0 0 3 0 2 2 4 6 7 6 7 5 0 0 0 1 2
output
YES ABABCA YES ABAC NO YES AAAB YES ABCAB

In the first test case, we can have $A = \{0, 1, 1\}$, $B = \{0, 1\}$, $C = \{2\}$, meaning $\text{MEX}(A) + \text{MEX}(B) + \text{MEX}(C) = 4$, and $2 \cdot \max(\text{MEX}(A), \text{MEX}(B), \text{MEX}(C)) = 2 \cdot \max(2, 2, 0) = 4$.

In the third test case, it can be shown that there are no valid distributions.

E. Treasure Map Destruction (Constructive Version)

2 seconds, 256 megabytes

Bessie the Buccaneer has plundered the six or seven seas for decades, but now she needs to find a place to bury her treasure!

There are n islands in a line, and Bessie buried her treasure on some number of islands. It is guaranteed that Bessie buried treasure on at least one island.

In order to remember where she buried her treasure, she created a treasure map. The treasure map can be represented as an array of n integers, where a_i indicates the minimum number of islands that Bessie would need to travel through to get to an island with treasure if she started at island i , assuming she can only travel to adjacent islands.

However, after Bessie buried her treasure, Elsie the Evil Explorer tried to steal Bessie's treasure map! Thankfully, Elsie was unsuccessful, but in the chaos, parts of Bessie's treasure map got destroyed, meaning that some of the elements of a were set to -1 ! Panicked, Bessie gives you the destroyed array b , and wants you to find any possible set of islands that she could have placed her treasure on. It's possible that Bessie messed up while writing the treasure map, so there could be no possible set of islands that she could have placed her treasure on.

Input

The first line of the input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($1 \leq n \leq 2 \cdot 10^5$) — the number of islands.

The second line contains $b_1, b_2, \dots, b_n$ ($-1 \leq b_i \leq n - 1$) — the destroyed array of distances. If $b_i = -1$, the element was destroyed.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a bitstring of length n , such that the i -th character is **1** if there is a treasure at position i , or **0** if there isn't. If there are multiple possible answers, output any. If there are no possible answers, output -1 . **Your answer must contain at least one treasure.**

input
12 5 0 1 -1 -1 0 3 -1 0 2 5 -1 1 -1 1 -1 5 -1 -1 -1 -1 -1 5 -1 2 -1 3 -1 7 2 1 0 1 0 1 2 1 -1 3 1 -1 1 1 0 4 3 -1 -1 -1 6 -1 -1 0 -1 2 4 10 -1 1 -1 -1 1 1 -1 -1 2 -1
output
10101 -1 10001 11010 -1 0010100 1 010 1 0001 -1 1001001000

In the first test case, Bessie could have placed her treasure at the first, third, and fifth island, making $a = [0, 1, 0, 1, 0]$.

In the fourth test case, since the entire map was destroyed, all $2^5 - 1 = 31$ **non-empty** sets of islands are possible.

F. Binary Bubble Sort Inversions

3 seconds, 256 megabytes

Define performing a bubble on an array $b_1, b_2, \dots, b_m$ to be performing the following action:

- For each integer i ($1 \leq i < m$) **in order**, if $b_i > b_{i+1}$, swap b_i and b_{i+1} .

Define performing a reverse bubble on an array $b_1, b_2, \dots, b_m$ to be performing the following action:

- For each integer i ($1 \leq i < m$) **in reverse order**, if $b_i > b_{i+1}$, swap b_i and b_{i+1} .

For example, if we perform a bubble on $[1, 1, 0]$, we would transform it as follows: $[1, 1, 0] \rightarrow [1, 1, 0] \rightarrow [1, 0, 1]$. If we perform a reverse bubble on $[1, 1, 0]$, we would transform it as follows: $[1, 1, 0] \rightarrow [1, 0, 1] \rightarrow [0, 1, 1]$.

You are given an array $a_1, a_2, \dots, a_n$ consisting of only 0s and 1s and a binary string* s of length n . You perform n operations **in order**, where the i -th operation performs a bubble on a if $s_i = 1$ and performs a reverse bubble on a if $s_i = 0$. For each of the $n + 1$ states of a — before any operations have been performed and after each operation — count the number of inversions[†] in a .

*A binary string is a string that only contains 0s and 1s.

[†]An inversion in an array c of length m is a pair of indices i, j such that $1 \leq i < j \leq m$ and $c_i > c_j$.

Input

The first line of each input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($1 \leq n \leq 5 \cdot 10^5$) — the length of a and s .

The second line of each test case contains $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 1$) — the array a .

The third line of each test case contains s ($s_i \in \{0, 1\}$, $|s| = n$) — the binary string s .

It is guaranteed that the sum of n over all test cases does not exceed $5 \cdot 10^5$.

* $|c|$ denotes the length of a string c .

Output

For each test case, output $n + 1$ space separated integers, the i -th integer denoting the number of inversions in a after the first $i - 1$ operations have been performed.

input
7 4 1 1 0 0 1010 7 0 1 0 1 1 0 0 0101001 4 0 0 1 1 1001 1 1 1 5 0 1 1 0 0 11101 3 1 0 0 000 6 1 0 1 1 0 0 011000
output
4 2 1 0 0 7 4 2 0 0 0 0 0 0 0 0 0 0 0 0 4 2 0 0 0 0 2 1 0 0 7 4 2 1 0 0 0

In the first test case:

- Before any operations: $a = [1, 1, 0, 0]$, which has 4 inversions.
- After the first operation: $a = [1, 0, 0, 1]$, which has 2 inversions.
- After the second operation: $a = [0, 1, 0, 1]$, which has 1 inversion.
- After the third operation: $a = [0, 0, 1, 1]$, which has 0 inversions.
- After the fourth operation: $a = [0, 0, 1, 1]$, which has 0 inversions.

G. Index Removal

2 seconds, 256 megabytes

An array $b_1, b_2, \dots, b_m$ is good if, for each i ($1 \leq i < m$), $b_{i+1} \geq b_i$ and $b_{i+1} - b_i \leq k$. An array of length 1 is always good.

You are given an **initially good** array $a_1, a_2, \dots, a_n$. Solve the following problem for each i ($1 \leq i \leq n$) **independently**:

- The i -th element of a is removed, making $a = [a_1, a_2, \dots, a_{i-1}, a_{i+1}, \dots, a_n]$. In one operation, you are able to subtract 1 from any element of a . What is the minimum number of operations required to make a good again?

Input

The first line of each input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and k ($2 \leq n \leq 2 \cdot 10^5, 1 \leq k \leq 10^9$).

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$). It is guaranteed that a is initially good.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n space separated integers on a single line: the i -th integer denoting the solution to the problem for the i -th index.

input
7 4 2 1 2 4 5 4 1 1 2 3 4 5 7 1 8 9 16 20 5 1000000000 1 6 7 67 6767 6 1 1 1 2 2 3 4 4 1 1 2 3 3 4 2 1 2 4 6
output
0 1 1 0 0 2 1 0 0 2 1 4 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 2 2 0

In the first test case:

- $i = 1, a = [2, 4, 5]$. Since a is still good, we do not need to perform any operations.
- $i = 2, a = [1, 4, 5]$. If we perform the operation once on the 2nd element, $a = [1, 3, 5]$, which is a good array.
- $i = 3, a = [1, 2, 5]$. If we perform the operation once on the 3rd element, $a = [1, 2, 4]$, which is a good array.
- $i = 4, a = [1, 2, 4]$. Since a is still good, we do not need to perform any operations.

In the fourth test case, a will remain good no matter what element we remove, so the answer for each index is 0.

H. Treasure Map Destruction (Counting Version)

2 seconds, 256 megabytes

Bessie the Buccaneer has plundered the six or seven seas for decades, but now she needs to find a place to bury her treasure!

There are n islands in a line, and Bessie buried her treasure on some number of islands. It is guaranteed that Bessie buried treasure on at least one island.

In order to remember where she buried her treasure, she created a treasure map. The treasure map can be represented as an array of n integers, where a_i indicates the minimum number of islands that Bessie would need to travel through to get to an island with treasure if she started at island i , assuming she can only travel to adjacent islands.

However, after Bessie buried her treasure, Elsie the Evil Explorer tried to steal Bessie's treasure map! Thankfully, Elsie was unsuccessful, but in the chaos, parts of Bessie's treasure map got destroyed, meaning that some of the elements of a were set to -1 ! Panicked, Bessie gives you the destroyed array b , and wants to know how many possible distinct sets of islands she could have placed her treasures on. Two sets A, B are distinct if there exists at least one element in A that is not in B or vice versa.

It's possible that Bessie messed up while writing the treasure map, so there could be no possible set of islands on which she could have placed her treasure.

Input

The first line of the input contains t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains n ($1 \leq n \leq 2 \cdot 10^5$) — the number of islands.

The second line contains $b_1, b_2, \dots, b_n$ ($-1 \leq b_i \leq n - 1$) — the destroyed array of distances. If $b_i = -1$, the element was destroyed.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each testcase, output the number of possible distinct sets of islands that Bessie could have placed her treasure on mod $10^9 + 7$.

input
12 5 0 1 -1 -1 0 3 -1 0 2 5 -1 1 -1 1 -1 5 -1 -1 -1 -1 -1 5 -1 2 -1 3 -1 7 2 1 0 1 0 1 2 1 -1 3 1 -1 1 1 0 4 3 -1 -1 -1 6 -1 -1 0 -1 2 4 10 -1 1 -1 -1 1 1 -1 -1 2 -1
output
4 0 5 31 0 1 1 1 1 1 1 0 3

In the first test case, while we know that Bessie placed treasure at the first and fifth islands, Bessie could have also placed her treasure at the third island, fourth island, both the third and fourth islands, or neither the third nor fourth island, leading to 4 possible sets of island locations.

In the fourth test case, since the entire map was destroyed, all $2^5 - 1 = 31$ **non-empty** sets of islands are possible.

Hints: Codeforces Round 1119 (Div. 3)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Moo Language School (editorial, first line)

Farmer John can choose to build on every field that Farmer Nhoj does not own.

B. Minus Two (official hints)

Hint 1: When comparing two numbers that are initially not equal, the only way they can become equal after some number of operations is if at least one of them has an operation performed on it when it is < 2 .

Hint 2: If two elements are equal at some point, they'll be equal no matter how many more operations are performed. Therefore, it suffices to look at the end behavior of each element.

C. 101 (official hints)

Hint 1: If a -1 is in between two 1 s, it would only hurt the answer to make it a 1 .

Hint 2: If a -1 is not in between two 1 s, the -1 would only be included in a subarray satisfying the conditions if it was an endpoint.

D. MEX Multiset (official hints)

Hint 1: If there is only one 0 in the array, then exactly one set will have a mex greater than zero

Hint 2: $MEX(A) = MEX(B)$, $MEX(C) = 0$ satisfies the constraints.

E. Treasure Map Destruction (Constructive Version) (official hints)

Hint 1: If an element x at index i is not equal to -1 , there must be a treasure at either island $i + x$ or $i - x$

Hint 2: If an element x at index i is not equal to -1 , there must no treasures in the exclusive range of indices $(i + x, i - x)$

F. Binary Bubble Sort Inversions (official hints)

Hint 1: Look at performing some bubbles and reverse bubbles on small arrays. How many elements are moved?

Hint 2: A bubble will result is the leftmost 1 being moved all the way to the right. Similarly, a reverse bubble will result in the rightmost 0 being moved all the way to the left.

Hint 3: Since prefix zeroes and suffix ones don't matter, if we continuously remove them, then a bubble removes the last element of the array and a reverse bubble removes the first element of the array. Is there a way to remove the first and last elements of an array in constant time?

G. Index Removal (official hints)

Hint 1: Ignoring indexes 1 and n , if we remove index i , if we perform an operation on index j , all elements between $[i, j]$ will also have an operation performed on them.

Hint 2: If we remove index i , and we have to reduce the element at index $i + 1$, $i + 2$, etc, what will we subtract it to?

H. Treasure Map Destruction (Counting Version) (official hints)

Hint 1: Let's consider all locations that we have not restricted: we can separate them into 3 categories: the position is forced to be a treasure, the position has no undestroyed that depend on it, and all other positions.

Hint 2: The islands that are considered "non-restricted" form a chain of locations, with each non-restricted chain having a treasure between them. What must be true of the locations in this chain such that no condition is violated?

Hint 3: For every pair of adjacent locations, there must be at least one treasure at one of those locations.

Source: <https://codeforces.com/blog/entry/156457>

Codeforces Round 1114 (Div. 3)

A. Riptide

1 second, 256 megabytes

Alice, Bob, and Charlie are playing a game with tokens. They start with a , b , and c tokens, respectively.

The game is played in rounds. Before the beginning of each round, they check the number of tokens everyone has:

- If any two players have the exact same number of tokens, the game immediately ends.
- Otherwise, the round begins, all three players have a strictly different number of tokens. The player with the strictly most tokens gives exactly 1 token to the player with the strictly fewest tokens.

Given the starting tokens a , b , and c , determine exactly how many rounds the game will last before it ends.

Input

The first line contains a single integer t ($1 \leq t \leq 10^3$) — the number of test cases.

Each test case consists of a single line containing three integers a , b , and c ($1 \leq a, b, c \leq 10$).

Output

For each test case, output a single integer — the number of rounds the game will last before it ends.

input
6 1 2 3 4 6 1 3 3 7 1 7 10 6 1 9 1 1 1
output
1 2 0 3 3 0

In the first test case:

- No two players have the same number of tokens.
- Charlie has the most tokens (3 tokens), and Alice has the fewest tokens (1 token). Therefore, Charlie gives Alice a token.
- Now, Alice has 2 tokens, Bob has 2 tokens, and Charlie has 2 tokens. Since there are two players (or more) with the same number of tokens, the game ends.

The game ended after 1 round, so the answer is 1.

In the second test case, the game is played as follows:

- Bob gives Charlie a token, now Alice has 4 tokens, Bob has 5 tokens, and Charlie has 2 tokens.
- Bob gives Charlie a token, now Alice has 4 tokens, Bob has 4 tokens, and Charlie has 3 tokens. Since two players have the same number of tokens, the game ends.

The game lasted 2 rounds.

In the third test case, two players already have the same number of tokens. So the answer is 0 since no rounds were played.

B. Evanescent

2 seconds, 256 megabytes

Let $f(s)$ be the compressed version of a string s , formed by replacing every maximal contiguous block of identical characters with a single copy of that character. For example, $f(\text{"aabbcc"}) = \text{"abc"}$.

Let $|s|$ denote the length of a string s . Following this, $|f(s)|$ denotes the length of the compressed string. For example:

- $|f(\text{"aabbcc"})| = |\text{"abc"}| = 3$
- If the string is empty, its length is 0.

Yousef has given you a string s consisting of n lowercase Latin letters. You must delete **exactly** one character s_i ($2 \leq i \leq n - 1$) to form a new string s' , and then find the minimum possible value of $|f(s')|$.

Note that you **cannot** delete s_1 or s_n .

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($3 \leq n \leq 2 \cdot 10^5$) — the length of the string.

The second line of each test case contains a string s ($|s| = n$), consisting of lowercase Latin letters.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer — the minimum possible length of the resulting compressed string after deleting one character.

input
9 3 abb 3 aab 3 abc 4 abaa 4 abba 5 eeeeee 6 yysssee 7 abacaba 18 goodluckandhavefun
output
2 2 2 1 3 1 3 5 16

In the first test case, we can only delete the character $s_2 = \text{'b'}$, producing a string $s' = \text{"ab"}$, with $|f(s')| = 2$. Therefore, 2 is the minimum length achievable.

In the fourth test case, we can delete the character $s_2 = \text{'b'}$. The resulting string is $s' = \text{"aaa"}$ with $f(s') = \text{"a"}$, so $|f(s')| = 1$.

In the sixth test case, deleting any valid character results in $f(s') = "e"$ and $|f(s')| = 1$.

In the eighth test case, we can delete the character $s_4 = 'c'$. The resulting string is $s' = "abaaba"$ with $f(s') = "ababa"$ and $|f(s')| = 5$.

C1. Marenol (easy version)

2 seconds, 256 megabytes

This is the easy version of the problem. In this version, you are only asked to determine whether string a can be transformed into string b .

Yousef has given you two binary strings, a and b , of the same length n .

You are allowed to perform any of the following operations:

- Choose a substring* in a equal to **001** and replace it with **100**, or vice versa (i.e., **001** $\rightarrow$ **100** or **100** $\rightarrow$ **001**).
- Choose a substring in a equal to **110** and replace it with **011**, or vice versa (i.e., **011** $\rightarrow$ **110** or **110** $\rightarrow$ **011**).

Your task is to determine whether it is possible to transform string a into string b using a finite number of operations.

*A string a is a substring of a string b if a can be obtained from b by deletion of several (possibly zero or all) characters from the beginning and several (possibly zero or all) characters from the end.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of each string.

The second line of each test case contains a binary string a ($|a| = n$), consisting of only characters **0** and/or **1**.

The third line of each test case contains a binary string b ($|b| = n$), consisting of only characters **0** and/or **1**.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output "YES" if the string a can be transformed into string b using a finite number of operations, and "NO" otherwise.

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

input
9
1
0
0
2
01
10
3
001
100
4
1010
0101
4
1100
1000
5
01001
10010
6
110000
000011
6
111000
000111
7
1001100
0000111
output
YES
NO
YES
NO
NO
YES
YES
NO
YES

In the first test case, it already holds that $a = b$. Therefore, the answer is YES.

In the second test case, we cannot perform any operation. Since $a \neq b$, the answer is NO.

In the third test case, we can choose the substring $a[1, 3] = 001$ and replace it with **100**, making $a = b$. Therefore, the answer is YES.

In the seventh test case, we can do the following in order:

- 110000** $\rightarrow$ **100100**
- 100100** $\rightarrow$ **100001**
- 100001** $\rightarrow$ **001001**
- 001001** $\rightarrow$ **000011**

Therefore, the answer is YES.

C2. Marenol (hard version)

2 seconds, 256 megabytes

This is the hard version of the problem. In this version, you are asked to determine the minimum number of operations to transform a into b .

Yousef has given you two binary strings, a and b , of the same length n .

You are allowed to perform any of the following operations:

- Choose a substring* in a equal to **001** and replace it with **100**, or vice versa (i.e. **001** $\rightarrow$ **100** or **100** $\rightarrow$ **001**).
- Choose a substring in a equal to **110** and replace it with **011**, or vice versa (i.e. **011** $\rightarrow$ **110** or **110** $\rightarrow$ **011**).

Your task is to determine the minimum number of operations required to transform string a into string b . If it is impossible to transform a into b using the given operations, output -1 instead.

*A string a is a substring of a string b if a can be obtained from b by deletion of several (possibly zero or all) characters from the beginning and several (possibly zero or all) characters from the end.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of each string.

The second line of each test case contains a binary string a ($|a| = n$), consisting of only characters 0 and/or 1.

The third line of each test case contains a binary string b ($|b| = n$), consisting of only characters 0 and/or 1.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the minimum number of operations required to transform a into b . If it is impossible, output -1 instead.

input
5 4 0100 0001 4 0100 0010 6 110000 000011 8 10101010 10101010 5 01001 10010
output
1 -1 4 0 3

In the first test case, we can choose the substring $a[2, 4] = 100$ and replace it with 001 . This takes exactly 1 operation.

In the second test case, it is impossible to transform a into b , so the answer is -1 .

In the third test case, we can do the following in order:

- 110000 $\rightarrow$ 100100
- 100100 $\rightarrow$ 100001
- 100001 $\rightarrow$ 001001
- 001001 $\rightarrow$ 000011

This takes 4 operations. It can be shown that 4 is the minimum answer.

D. Silhouette

2.5 seconds, 256 megabytes

Yousef has a secret array a of n strictly positive integers. For each element a_i , its shadow b_i is the sum of all elements in a that are strictly smaller than a_i . Formally:

$$b_i = \sum_{\substack{1 \leq j \leq n \\ a_j < a_i}} a_j$$

You are given the shadow array b . Your task is to reconstruct the **lexicographically smallest** valid array a consisting of strictly positive integers that satisfies the above condition. If no such array exists, output -1 .

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the size of the array.

The second line of each test case contains n integers $b_1, b_2, \dots, b_n$ ($0 \leq b_i \leq 2 \cdot 10^{14}$) — the shadow array.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^{18}$) — the lexicographically smallest valid array a that satisfies the condition. If no valid array exists, output -1 instead.

input
8 1 0 5 0 4 0 4 14 3 4 0 0 3 0 0 0 3 0 1 1 4 1 1 1 1 7 0 4 4 4 4 4 9 5 0 0 0 3 3
output
1 2 5 2 5 6 3 2 2 1 1 1 1 2 2 -1 -1 1 1 1 2 2

In the first test case, the answer is $a = [1]$. Since there is only one element, there are no strictly smaller elements, so its shadow is 0. Thus $a = [1]$ is valid. It is also lexicographically smallest, because the only allowed values are positive integers, and 1 is the smallest possible.

In the second test case, the answer is $a = [2, 5, 2, 5, 6]$:

- For each 2, there is no smaller element in the array, so the shadow is 0.
- For each 5, the strictly smaller elements are the two 2's, so the shadow is $2 + 2 = 4$.
- For 6, the strictly smaller elements are two 2's and two 5's, so the shadow is $2 + 2 + 5 + 5 = 14$.

Therefore the shadow array is exactly $b = [0, 4, 0, 4, 14]$.

In the third test case, the shadow array for $a = [3, 2, 2]$ is calculated as follows:

- For $a_1 = 3$, the strictly smaller elements in the array are the two 2s. Their sum is $2 + 2 = 4$. So, $b_1 = 4$.
- For $a_2 = 2$, there are no strictly smaller elements in the array. So, $b_2 = 0$.
- For $a_3 = 2$, there are no strictly smaller elements in the array. So, $b_3 = 0$.

The resulting shadow array is $b = [4, 0, 0]$, which matches the input.

2 seconds, 256 megabytes

E. Chronostasis

2 seconds, 256 megabytes

Yousef has a hidden array a of length n consisting entirely of **strictly positive** integers.

An operation was performed exactly once to create an array b :

- Set $b_1 = a_1$.
- For every i from 2 to n , set $b_i = a_i - a_{i-1}$.
- After this, the elements of b were completely shuffled.

You are given the shuffled array b . Reconstruct the **lexicographically smallest** original array a . If it's impossible for any arrangement of b to produce an array a of strictly positive integers, output -1 .

Input

The first line of input contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the size of the array.

The second line of each test case contains n integers $b_1, b_2, \dots, b_n$ ($-10^9 \leq b_i \leq 10^9$) — the elements of the shuffled array b .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n strictly positive integers $a_1, a_2, \dots, a_n$ ($a_i \geq 1$) — the lexicographically smallest original array a . If it's impossible to create a valid array a , output -1 instead.

input
8 1 5 4 -5 2 1 1 6 -3 4 2 -1 1 0 6 -2 -2 4 1 0 1 7 0 0 -2 3 0 -1 2 8 -1 -1 -1 -1 5 0 0 1 5 1000000000 500000000 750000000 100000000 900000000 10 1000000000 -1000000000 500000000 -500000000 1 1 -1 -1 2 -2
output
5 -1 1 1 3 2 6 3 1 1 2 6 4 2 2 1 1 1 1 4 2 1 1 1 6 5 4 3 2 100000000 600000000 135000000 225000000 325000000 -1

In the first test case, the only valid array is $a = [5]$.

In the second test case, there is no valid arrangement of the elements of b that reconstructs an array a consisting entirely of strictly positive integers. Therefore, the answer is -1 .

In the third test case, one valid arrangement reconstructs the array $a = [1, 1, 3, 2, 6, 3]$. The resulting sequence of differences $[1, 0, 2, -1, 4, -3]$ is a permutation of the given array b , and among all valid reconstructions, this array is lexicographically smallest.

F. Whiplash

Yousef has given you an **even** integer n and two arrays, a and b , both consisting of n non-negative integers.

You can perform the following operation on array a any number of times (possibly zero):

- Choose an index i ($1 \leq i \leq n$).
- For all j such that $1 \leq j \leq n$ and $j \neq i$, replace a_j with $a_j \oplus a_i$. (Here, $\oplus$ denotes the **bitwise XOR operation**).
- The element a_i remains unchanged.

Determine whether it is possible to transform array a into array b using a finite number of operations.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains an **even** integer n ($2 \leq n \leq 2 \cdot 10^5$) — the length of the array.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 2^{30}$) — the elements of the array a .

The third line of each test case contains n integers $b_1, b_2, \dots, b_n$ ($0 \leq b_i < 2^{30}$) — the elements of the array b .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output "YES" if the array a can be transformed into array b using a finite number of operations, and "NO" otherwise.

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

input
6 2 1 2 1 0 4 1 2 4 7 6 7 5 3 4 1 2 4 8 8 4 2 1 4 1 2 3 4 1 2 4 5 4 1 2 0 3 3 3 0 3 6 3 5 6 9 10 12 6 5 3 12 15 9
output
NO YES YES NO NO YES

In the first test case, starting from $[1, 2]$, there is no sequence of operations that ends at $[1, 0]$, so the answer is "NO".

In the second test case, one valid sequence is:

- Choose index 2, $[1, 2, 4, 7] \rightarrow [3, 2, 6, 5]$.
- Choose index 3, $[3, 2, 6, 5] \rightarrow [5, 4, 6, 3]$.
- Choose index 4, $[5, 4, 6, 3] \rightarrow [6, 7, 5, 3]$.

In the third test case, one valid sequence is:

- Choose index 1, [1, 2, 4, 8] → [1, 3, 5, 9].
- Choose index 2, [1, 3, 5, 9] → [2, 3, 6, 10].
- Choose index 3, [2, 3, 6, 10] → [4, 5, 6, 12].
- Choose index 2, [4, 5, 6, 12] → [1, 5, 3, 9].
- Choose index 4, [1, 5, 3, 9] → [8, 12, 10, 9].
- Choose index 1, [8, 12, 10, 9] → [8, 4, 2, 1].

G. Nightcrawler

2.5 seconds, 256 megabytes

Yousef has given you a rooted tree* with n vertices, where the root is vertex 1. Each vertex i is assigned an integer a_i .

You must partition the set of all n vertices into exactly k disjoint subsets $S_1, S_2, \dots, S_k$ (that is, each vertex must belong to exactly one of the k sets) such that the following condition is satisfied:

- For any subset S_i containing two or more vertices, for every pair of vertices $u, v \in S_i$, one must be the ancestor† of the other (i.e., they must all lie on the same path extending from the root toward a leaf).

The score of a subset S_i is defined as the maximum value a_u among all vertices u in that subset. The score of the partition is the sum of the scores of the k subsets. In other words, the score of the partition is equal

to $\sum_{i=1}^k \max_{u \in S_i} a_u$.

For every integer k from 1 to n , calculate the maximum possible score of a partition. If it is impossible to partition the tree into exactly k subsets that satisfy the condition, output -1 .

*A tree is a connected graph without cycles. A rooted tree is a tree where one vertex is special and called the root.

†An ancestor of vertex v is any vertex on the simple path from v to the root, including the root, but not including v . The root has no ancestors.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($3 \leq n \leq 2 \cdot 10^5$) — the number of vertices.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the values of the vertices.

The third line of each test case contains $n - 1$ integers $p_2, p_3, \dots, p_n$ ($1 \leq p_i < i$), where p_i is the parent of the i -th vertex.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

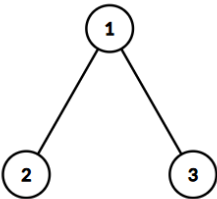
Output

For each test case, output a single line containing n space-separated integers. The k -th integer should represent the maximum possible score for a partition of k subsets. If it is impossible to partition the tree into k subsets, output -1 for that value.

input
7 3 10 20 30 1 1 4 5 10 15 20 1 2 2 4 1 2 3 4 1 1 3 5 1 2 3 4 5 1 2 3 4 9 1 100 1 90 80 1 2 3 4 1 1 2 4 3 3 3 3 6 5 4 10 3 9 1 1 2 1 4 1 4 10 10 20 1 1 2 1
output
-1 50 60 -1 35 45 50 -1 6 9 10 5 9 12 14 15 -1 -1 -1 -1 110 200 280 281 282 -1 -1 24 28 31 32 -1 30 40 41

In the first test case:

- For $k = 1$, we would need all the vertices to be in the same set. However, for vertices 2 and 3, neither of them is the ancestor of the other. Therefore, there is no valid partition.
- For $k = 2$, we can make $S_1 = \{2\}$, $S_2 = \{1, 3\}$. The score of this partition is $\max_{u \in S_1} a_u + \max_{u \in S_2} a_u = 20 + 30 = 50$. It can be shown that this is the maximum score.
- For $k = 3$, we can make $S_1 = \{1\}$, $S_2 = \{2\}$, $S_3 = \{3\}$. The score of this partition is $10 + 20 + 30 = 60$.

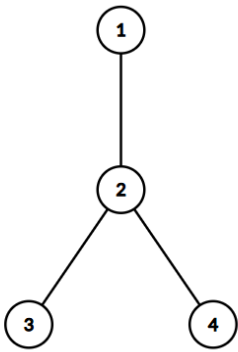


The given tree in the first test case.

In the second test case:

- For $k = 1$, we would need all the vertices to be in the same set. However, vertices 3 and 4 are not on the same root-to-leaf path, so this is impossible.
- For $k = 2$, we can make $S_1 = \{3\}$, $S_2 = \{1, 2, 4\}$. The score of this partition is $\max_{u \in S_1} a_u + \max_{u \in S_2} a_u = 15 + 20 = 35$. It can be shown that this is the maximum score.
- For $k = 3$, we can make $S_1 = \{3\}$, $S_2 = \{4\}$, $S_3 = \{1, 2\}$. The score of this partition is $15 + 20 + 10 = 45$. It can be shown that this is the maximum score.
- For $k = 4$, we can make $S_1 = \{1\}$, $S_2 = \{2\}$, $S_3 = \{3\}$, $S_4 = \{4\}$. The score of this partition is $5 + 10 + 15 + 20 = 50$.

The given tree in the second test case.



Hints: Codeforces Round 1114 (Div. 3)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Riptide (official hint)

Hint: Assume $a \leq b \leq c$, does b ever change?

B. Evanescent (official hint)

Hint: If we perform the operation on any character in a maximal contiguous block of length > 1 , then $f(s') = f(s)$.

C1. Marenol (easy version) (official hints)

Hint 1: The operation is equivalent to freely swapping a_i with a_{i+2} for any valid index i .

Hint 2: After the reduction from Hint 1, we can treat a as two separate strings.

C2. Marenol (hard version) (editorial, first line)

Building off the core observation from the easy version, we know that elements are strictly trapped in their own parity.

D. Silhouette (official hints)

Hint 1: Equal values in a will always have the exact same shadow, and larger elements always have strictly larger shadows. So if you sort the distinct values of b , they correspond in order to the distinct values of a : the group with the i -th smallest value has shadow equal to the i -th smallest distinct value in b .

Hint 2: Let $s_1 < s_2 < \dots < s_m$ be the distinct sorted values of b , and let group i be all elements sharing the value corresponding to s_i . Then s_i is the sum of all groups smaller than group i , and s_{i+1} is that same sum plus group i itself. So $s_{i+1} - s_i = c_i \cdot v_i$, where c_i is the count of elements in group i and v_i is their value.

Hint 3: What must s_1 be? And once every v_i is pinned down by the equation above, is there a group left with no equation constraining its value? What's the smallest legal choice for it?

E. Chronostasis (official hints)

Hint 1: The problem states that $b_i = a_i - a_{i-1}$. This directly implies that $a_i = a_{i-1} + b_i$. If you assume $a_0 = 0$, you can view a simply as the prefix sums of some permutation of b .

Hint 2: The standard way to achieve a lexicographically smallest sequence is to build it greedily from left to right, making a_1 as small as possible, then a_2 as small as possible, and so forth.

Hint 3: Since a_{i-1} is fixed from the previous steps, minimizing a_i means you must pick the smallest available b_i . However, the problem imposes a strict rule: every element in a must be strictly positive ($a_i \geq 1$). How does this restrict your choice of b_i ?

F. Whiplash (official hints)

Hint 1: Try tracing the array elements after performing two consecutive operations, say on index i and then on index j . If you look at the set of resulting values and ignore their order, how does it compare to the set of values you would get if you only performed a single operation on index j ?

Hint 2: Any sequence of operations results in an array that is just a permutation of either the original array a , or the array a after exactly one operation.

G. Nightcrawler (official hints)

Hint 1: Because no subset can contain branching paths, two distinct leaves can never belong to the same subset. If the tree has ℓ leaves, you need at least ℓ subsets. Any $k < \ell$ is impossible. For $k \geq \ell$, the condition is always satisfiable since you can always break paths into smaller singletons.

Hint 2: Try to build the solution bottom-up. Each leaf starts a path. When these paths meet at a parent node, the parent can only join one of them. To maximize the total score, which path's maximum should the parent node interact with?

Hint 3: It is always optimal for the parent to join the path that currently has the smallest maximum. If the parent's value is larger, it successfully increases that path's maximum, leaving the old maximum behind as a spare value. If the parent is smaller, the path is unaffected, and the parent itself becomes the spare value that we can use if we are allowed extra subsets.

Source: <https://codeforces.com/blog/entry/155666>

A - Appender

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

You are given a string S consisting of lowercase English letters.

Output the string T determined as follows.

- If the last character of S is e, T is the string obtained by appending r to the end of S .
- If the last character of S is not e, T is the string obtained by appending er to the end of S .

Constraints

- S is a string consisting of lowercase English letters.
- The length of S is between 1 and 10, inclusive.

Input

The input is given from Standard Input in the following format:

S

Output

Output the answer.

Sample Input 1

live

Sample Output 1

liver

The last character of live is e, so the string liver, obtained by appending r to the end of live, is the string T to be outputted.

Sample Input 2

```
femur
```

Sample Output 2

```
femurer
```

The last character of femur is not e, so the string femurer, obtained by appending er to the end of femur, is the string T to be outputted.

Sample Input 3

```
chimpanzee
```

Sample Output 3

```
chimpanzeer
```

The last character of chimpanzee is e, so the string chimpanzeer, obtained by appending r to the end of chimpanzee, is the string T to be outputted.

B - Wild Card

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

S is a string of length N consisting of lowercase English letters.

T is a string of length N consisting of lowercase English letters and $*$.

We say that S matches T if and only if T can be made equal to S by replacing each $*$ in T with some lowercase English letter.

Determine whether S matches T .

Constraints

- $1 \leq N \leq 100$
- S is a string of length N consisting of lowercase English letters.
- T is a string of length N consisting of lowercase English letters and $*$.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $S$   
 $T$ 
```

Output

Output Yes if S matches T , and No otherwise, in one line.

Sample Input 1

```
8  
chokudai  
**o*u*ai
```

Sample Output 1

```
Yes
```

By replacing the first, second, fourth, and sixth characters of T , which are *, with c, h, k, and d, respectively, T can be made equal to S .

Sample Input 2

```
5
snuke
snake
```

Sample Output 2

```
No
```

Sample Input 3

```
5
yiwiy
*****
```

Sample Output 3

```
Yes
```

C - Third Largest Number

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

You are given an integer N not less than 3 and a length- N sequence of positive integers $A = (A_1, A_2, \dots, A_N)$.

For each $k = 3, 4, \dots, N$, solve the following problem:

- When $A_1, A_2, \dots, A_k$ are sorted in descending order, find the third value from the front.

Constraints

- $3 \leq N \leq 5 \times 10^5$
- $1 \leq A_i \leq 10^9$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N
A_1 A_2 ... A_N
```

Output

Output $N - 2$ lines.

The i -th line ($1 \leq i \leq N - 2$) should contain the answer for $k = i + 2$.

Sample Input 1

```
5
1 2 1 2 3
```

Sample Output 1

```
1
1
2
```

- For $k = 3$: sorting A_1, A_2, A_3 in descending order gives 2, 1, 1. The third value from the front is 1, so output 1 on the first line.
- For $k = 4$: sorting A_1, A_2, A_3, A_4 in descending order gives 2, 2, 1, 1. The third value from the front is 1, so output 1 on the second line.
- For $k = 5$: sorting A_1, A_2, A_3, A_4, A_5 in descending order gives 3, 2, 2, 1, 1. The third value from the front is 2, so output 2 on the third line.

Sample Input 2

```
10
1 1 1 3 2 5 4 3 6 5
```

Sample Output 2

```
1
1
1
2
3
3
4
5
```

Sample Input 3

```
10
11 9 1 3 17 19 10 19 17 3
```

Sample Output 3

```
1
3
9
11
11
17
17
17
```


D - Automat

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 425 points

Problem Statement

In the Kingdom of AtCoder, two kinds of bills are in circulation: 1-dollar bills and K -dollar bills.

At Company J's fully automated cafeteria in the kingdom, desserts and drinks are sold as products.

The company's fully automated cafeteria has a dessert vending machine that sells N desserts, and a drink vending machine that sells M drinks. The desserts are numbered 1 through N , and the drinks are numbered 1 through M .

Dessert i costs A_i dollars, and drink j costs B_j dollars.

As payment, the dessert vending machine accepts both 1-dollar bills and K -dollar bills, but the drink vending machine accepts only K -dollar bills. Both vending machines give change using only 1-dollar bills. You cannot buy two or more of the same product.

Takahashi came to Company J's fully automated cafeteria with X 1-dollar bills and Y K -dollar bills.

Among the combinations of products that he can buy with the bills he has, find the maximum number of products he can purchase.

Constraints

- $1 \leq N \leq 2 \times 10^5$
 - $1 \leq M \leq 2 \times 10^5$
 - $2 \leq K \leq 10^9$
 - $0 \leq X \leq 10^{15}$
 - $0 \leq Y \leq 10^9$
 - $1 \leq A_i \leq 10^9$ ($1 \leq i \leq N$)
 - $1 \leq B_j \leq 10^9$ ($1 \leq j \leq M$)
 - All input values are integers.
-

Input

The input is given from Standard Input in the following format:

```
 $N$   $M$   $K$   
 $X$   $Y$   
 $A_1$   $\dots$   $A_N$   
 $B_1$   $\dots$   $B_M$ 
```

Output

Output the answer.

Sample Input 1

```
2 3 10  
50 6  
22 30  
20 12 24
```

Sample Output 1

```
4
```

By shopping as follows, Takahashi can buy four products.

- Pay two 10-dollar bills to buy drink 1. No change.
- Pay two 10-dollar bills to buy drink 2. The change is eight 1-dollar bills.
- Pay two 10-dollar bills and ten 1-dollar bills to buy dessert 2. No change.
- Pay twenty-two 1-dollar bills to buy dessert 1. No change.

He cannot buy more than four products, so the answer is 4.

Sample Input 2

```
1 7 67  
677677677766666 0  
777666777  
20 12 24 67 67 67 67
```

Sample Output 2

```
1
```

By shopping as follows, Takahashi can buy one product.

- Pay 600000777666777 1-dollar bills to buy dessert 1. The change is 6000000000000000 1-dollar bills.

He cannot buy more than one product, so the answer is 1.

Sample Input 3

```
20 20 30
605776135 133105105
97363214 218434035 697895427 109255624 299037330 227873982 195540071 411713803 828357845 24
4535208 138059186 639510883 39844882 707397687 371274487 696536603 351588202 319490007 4712
1612 87169661
32256972 567982330 554885983 299718223 443859449 687952877 264684780 666659381 576335424 94
1894234 406248934 321334900 423472560 863738035 213143887 384834384 468161291 673106162 164
648316 15903323
```

Sample Output 3

```
22
```

E - Min-Max Swap

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 450 points

Problem Statement

You are given a permutation $P = (P_1, P_2, \dots, P_N)$ of $(1, 2, \dots, N)$ and length- M integer sequences $L = (L_1, L_2, \dots, L_M)$ and $R = (R_1, R_2, \dots, R_M)$.

For this permutation P , perform the following operation for $i = 1, 2, \dots, M$ in this order:

- Among $P_{L_i}, P_{L_i+1}, \dots, P_{R_i}$, swap the positions of the element with the minimum value and the element with the maximum value.

Find each element of P after the M operations.

Constraints

- $2 \leq N \leq 2 \times 10^5$
- $1 \leq M \leq 2 \times 10^5$
- P is a permutation of $(1, 2, \dots, N)$.
- $1 \leq L_i < R_i \leq N$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N M
P_1 P_2 ... P_N
L_1 R_1
L_2 R_2
⋮
L_M R_M
```

Output

Output each element of P after the M operations, in order from front to back, separated by spaces.

Sample Input 1

```
5 3
3 1 4 2 5
1 3
1 5
1 4
```

Sample Output 1

```
3 4 2 5 1
```

Initially, $P = (3, 1, 4, 2, 5)$.

- For $i = 1$: among P_1, P_2, P_3 , the minimum value is P_2 and the maximum value is P_3 . Swapping P_2 and P_3 gives $P = (3, 4, 1, 2, 5)$.
- For $i = 2$: among P_1, P_2, P_3, P_4, P_5 , the minimum value is P_3 and the maximum value is P_5 . Swapping P_3 and P_5 gives $P = (3, 4, 5, 2, 1)$.
- For $i = 3$: among P_1, P_2, P_3, P_4 , the minimum value is P_4 and the maximum value is P_3 . Swapping P_3 and P_4 gives $P = (3, 4, 2, 5, 1)$.

After the three operations, $P = (3, 4, 2, 5, 1)$.

Sample Input 2

```
6 7
3 6 5 2 4 1
4 5
2 3
3 5
4 6
3 4
1 6
3 5
```

Sample Output 2

```
3 5 4 6 2 1
```

F - Chebyshev Cafe

Time Limit: 3 sec / Memory Limit: 2048 MiB

Score : 525 points

Problem Statement

There is a city represented by a grid with N rows and N columns. The cell at the i -th row from the top and j -th column from the left is denoted as cell (i, j) . Each cell has one cafe.

You are planning a meeting. Among the participants, the number of people living in cell (i, j) is $(A_i \times B_j) \bmod M$. The travel cost for one participant living in cell (s_r, s_c) to go to the cafe at cell (t_r, t_c) is $\max(|s_r - t_r|, |s_c - t_c|)$.

Let $f(i, j)$ be the total travel cost incurred when gathering all the participants at the cafe at cell (i, j) .

For each cell (i, j) , find the following value, and output the bitwise XOR of these values over all cells.

- $f(i, j) + (i - 1)N + (j - 1)$

► What is the bitwise XOR operation?

Constraints

- $1 \leq N \leq 1500$
- $2 \leq M \leq 2 \times 10^6$
- $1 \leq A_i \leq M - 1$ ($1 \leq i \leq N$)
- $1 \leq B_j \leq M - 1$ ($1 \leq j \leq N$)
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N M
A_1 ... A_N
B_1 ... B_N
```

Output

Output the answer.

Sample Input 1

```
4 19
2 3 1 1
3 4 4 10
```

Sample Output 1

```
467
```

The number of participants living in each cell is as follows:
$$\begin{bmatrix} 6 & 8 & 8 & 1 \\ 9 & 12 & 12 & 11 \\ 3 & 4 & 4 & 10 \\ 3 & 4 & 4 & 10 \end{bmatrix}.$$

For example, the value of $f(1, 2)$ is obtained as the sum 176 of the following 16 values:

- The total amount $6 \times 1 = 6$ paid by the 6 residents of cell $(1, 1)$, each of whom pays a travel cost of $\max(|1 - 1|, |1 - 2|) = 1$
- The total amount $8 \times 0 = 0$ paid by the 8 residents of cell $(1, 2)$, each of whom pays a travel cost of $\max(|1 - 1|, |2 - 2|) = 0$
- The total amount $8 \times 1 = 8$ paid by the 8 residents of cell $(1, 3)$, each of whom pays a travel cost of $\max(|1 - 1|, |3 - 2|) = 1$
- The total amount $1 \times 2 = 2$ paid by the 1 resident of cell $(1, 4)$, each of whom pays a travel cost of $\max(|1 - 1|, |4 - 2|) = 2$
- $\vdots$
- The total amount $3 \times 3 = 9$ paid by the 3 residents of cell $(4, 1)$, each of whom pays a travel cost of $\max(|4 - 1|, |1 - 2|) = 3$
- The total amount $4 \times 3 = 12$ paid by the 4 residents of cell $(4, 2)$, each of whom pays a travel cost of $\max(|4 - 1|, |2 - 2|) = 3$
- The total amount $4 \times 3 = 12$ paid by the 4 residents of cell $(4, 3)$, each of whom pays a travel cost of $\max(|4 - 1|, |3 - 2|) = 3$
- The total amount $10 \times 3 = 30$ paid by the 10 residents of cell $(4, 4)$, each of whom pays a travel cost of $\max(|4 - 1|, |4 - 2|) = 3$

The value of $f(i, j)$ for each cell is as follows:
$$\begin{bmatrix} 220 & 176 & 179 & 224 \\ 199 & 140 & 136 & 182 \\ 212 & 159 & 143 & 178 \\ 255 & 215 & 201 & 218 \end{bmatrix}.$$

Sample Input 2

```
6 67
1 6 9 2 3 1
1 10 3 7 2 2
```

Sample Output 2

```
1021
```

Sample Input 3

```
20 676767
60932 508200 22883 481150 426836 554098 276223 307959 133025 573906 17711 413230 274246 500
277 524959 477728 546547 43099 324227 660187
552697 293978 246000 159233 213736 628198 394200 194076 618720 453035 615100 331522 669419
180856 530693 399531 223149 159334 102107 598633
```

Sample Output 3

```
764668547
```

G - Increasing Popcount

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 625 points

Problem Statement

You are given positive integers L and R satisfying $L \leq R$.

A sequence of positive integers $A = (A_L, A_{L+1}, \dots, A_R)$ of length $R - L + 1$ satisfying the following condition is called a **good integer sequence**:

- For every pair of integers (i, j) satisfying $L \leq i < j \leq R$, if $A_i = A_j$, then $\text{popcount}(i) < \text{popcount}(j)$.

Find the minimum value of $\max(A_L, A_{L+1}, \dots, A_R)$ over all good integer sequences $A = (A_L, A_{L+1}, \dots, A_R)$.

You are given T test cases; solve each of them.

► What is popcount?

Constraints

- $1 \leq T \leq 10^4$
- $1 \leq L \leq R \leq 10^{18}$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
T
case1
case2
⋮
caseT
```

Each test case is given in the following format.

```
L R
```

Output

Output T lines.

The i -th line ($1 \leq i \leq T$) should contain the answer for the i -th test case case_i .

Sample Input 1

```
5
3 6
1 1
2 15
100 100000000
514284786278117031 620546740167642909
```

Sample Output 1

```
3
1
6
1636985
11375123021856567
```

Consider the first test case.

$(A_3, A_4, A_5, A_6) = (1, 2, 2, 3)$ is a good integer sequence, and $\max(A_3, A_4, A_5, A_6) = 3$.

There is no good integer sequence with $\max(A_3, A_4, A_5, A_6) < 3$, so output 3 on the first line.

Hints: AtCoder Beginner Contest 476

Taken from the official editorial, not written by us. Read one hint at a time.

A. Appender *(editorial, first line)*

Check if the last character of the given string coincides with e, append an appropriate string to S , and print it.

B. Wild Card *(editorial, first line)*

If S matches T , then for $i = 1, 2, \dots, N$, T_i equals either S_i or $*$.

C. Third Largest Number *(editorial, first line)*

Consider finding the answer in ascending order of k .

D. Automat *(editorial, first line)*

A combination of products $A_{i_1}, \dots, A_{i_n}, B_{j_1}, \dots, B_{j_m}$ can be purchased if and only if: the total price, $\sum_{1 \leq k \leq n} A_{i_k} + \sum_{1 \leq k \leq m} B_{j_k} \dots$

E. Min-Max Swap *(editorial, first line)*

Consider simulating the operations step-by-step.

F. Chebyshev Cafe *(editorial, first line)*

We will find the values of $f(i, j)$ for all N^2 cells.

G. Increasing Popcount *(editorial, first line)*

First, consider a graph with $(R - L + 1)$ vertices, vertex L through R .

Source: <https://atcoder.jp/contests/abc476/editorial>

A - Second Half Sum

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

You are given an integer sequence of length N : $A = (A_1, A_2, \dots, A_N)$. Here, N is even.
Find the sum of the latter half of A , that is, the sum of $A_{(N/2)+1}, A_{(N/2)+2}, \dots, A_N$.

Constraints

- All input values are integers.
- N is an even number satisfying $2 \leq N \leq 100$.
- $1 \leq A_i \leq 100$

Input

The input is given from Standard Input in the following format:

```
N
A_1 A_2 ... A_N
```

Output

Output the answer.

Sample Input 1

```
8
1 3 7 8 4 2 5 6
```

Sample Output 1

```
17
```

The length of the sequence A is $N = 8$, and you should find the sum of A_5, A_6, A_7, A_8 .
Since $A_5 + A_6 + A_7 + A_8 = 4 + 2 + 5 + 6 = 17$, output 17.

Sample Input 2

```
2
1 100
```

Sample Output 2

```
100
```

Sample Input 3

```
10
31 41 59 26 53 58 97 93 23 84
```

Sample Output 3

```
355
```

B - Old Maid

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

Takahashi currently has N cards. The i -th ($1 \leq i \leq N$) card has the integer A_i written on it.

He repeats the following operation as many times as possible.

- Choose two different cards with the same integer written on them, and eat those two cards. The eaten cards are permanently removed and cannot be chosen in subsequent operations.

Find the sum of the integers written on the cards that remain when the operation can no longer be performed.

Constraints

- $1 \leq N \leq 100$
- $1 \leq A_i \leq 100$ ($1 \leq i \leq N$)
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $A_1$   $A_2$   $\cdots$   $A_N$ 
```

Output

Output the sum of the integers written on the remaining cards.

Sample Input 1

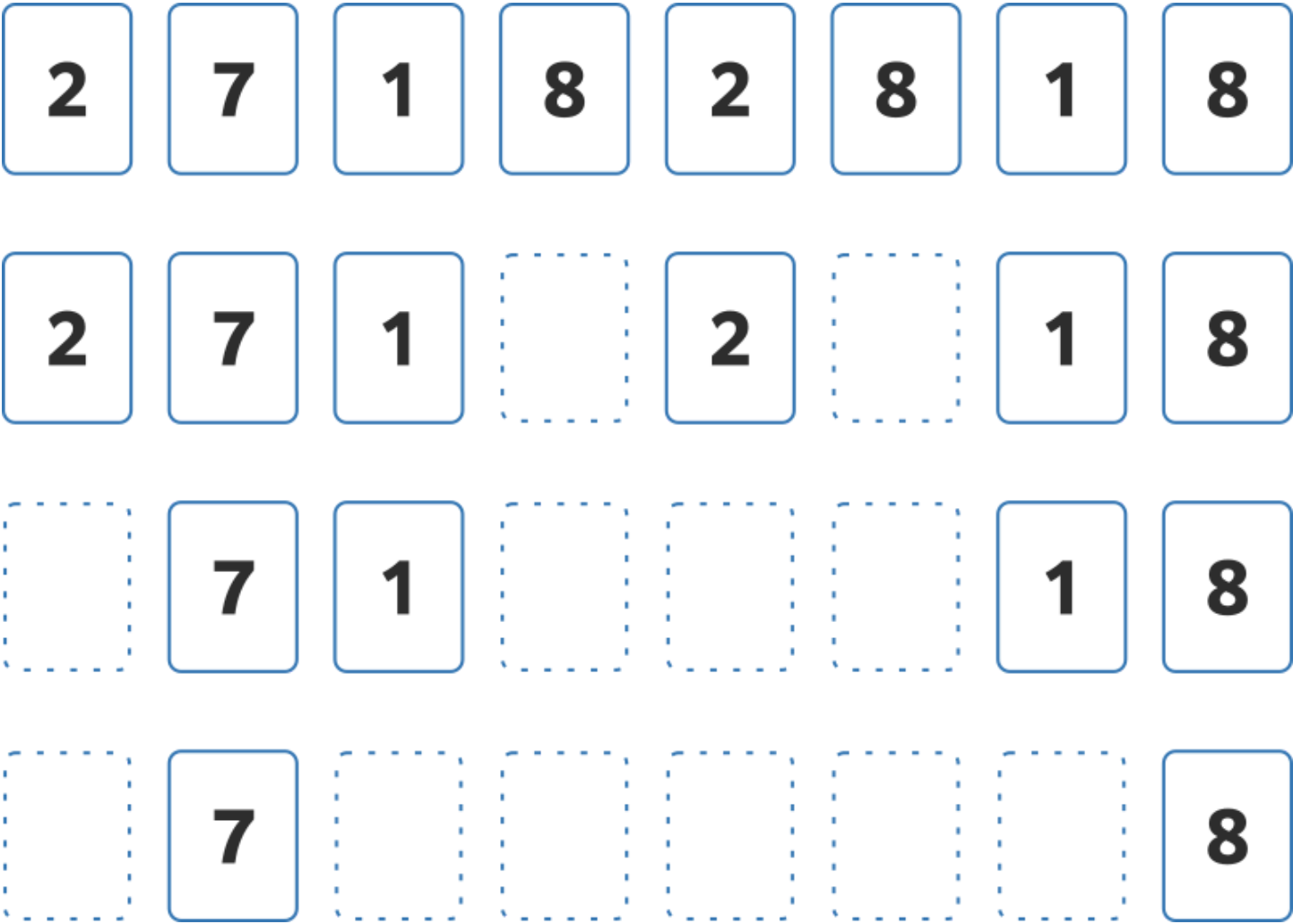
```
8  
2 7 1 8 2 8 1 8
```

Sample Output 1

15

For example, Takahashi operates as follows.

- 1. The fourth card and sixth card both have 8 written on them, so he chooses them and eats them.
- 2. The first card and fifth card both have 2 written on them, so he chooses them and eats them.
- 3. The third card and seventh card both have 1 written on them, so he chooses them and eats them.



Thus, the sum of the integers written on the remaining cards is $7 + 8 = 15$.

Note that there are three cards with 8 written on them, so once two of them are eaten, the remaining one cannot be eaten.

Sample Input 2

5
1 2 3 4 5

Sample Output 2

```
15
```

It is possible that Takahashi cannot perform the operation even once.

Sample Input 3

```
15
58 97 74 16 97 74 97 16 51 52 58 52 74 32 43
```

Sample Output 3

```
297
```

C - Change Schools

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

Currently, AtCoder High School has K classes and N students, and the i -th ($1 \leq i \leq N$) student belongs to the A_i -th class.

Takahashi is transferring to AtCoder High School in September. At that time, he can choose any one class among the K classes and belong to that class.

He will be sad if there is a class with more students belonging to it than the class he belongs to. Otherwise, he will be happy.

Find how many classes make him happy if he belongs to them.

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq K \leq N$
- $1 \leq A_i \leq K$ ($1 \leq i \leq N$)
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N K
A_1 A_2 ... A_N
```

Output

Output the number of classes satisfying the condition.

Sample Input 1

```
8 5
3 3 5 5 4 4 3 2
```

Sample Output 1

3

For example, if Takahashi chooses the fifth class and belongs to it, the number of students belonging to the fifth class becomes 3. Then, he will be happy since there is no class with more than three students belonging to it.

On the other hand, for example, if he chooses the first class and belongs to it, the number of students belonging to the first class becomes 1. Then, he will be sad since three students belong to the third class.

He will be happy if and only if he chooses the third, fourth, or fifth class, so output 3.

Sample Input 2

```
6 1
1 1 1 1 1 1
```

Sample Output 2

1

It is possible that AtCoder High School has only one class.

Sample Input 3

```
14 8
6 1 5 3 8 4 3 4 3 5 1 2 5 1
```

Sample Output 3

4

D - Coefficient Stair

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 400 points

Problem Statement

Output all length- N sequences $A = (A_1, A_2, \dots, A_N)$ consisting of non-negative integers that satisfy $\sum_{i=1}^N i \times A_i = K$, in lexicographic order from smallest to largest.

Here, you will only receive inputs such that the number of sequences satisfying the condition is at most 3×10^5 .

► What is lexicographic order for sequences?

Constraints

- $1 \leq N \leq 10$
- $1 \leq K \leq 2 \times 10^5$
- The number of sequences satisfying the condition is at most 3×10^5 .
- All input values are integers.

Input

The input is given from Standard Input in the following format:

N K

Output

Let q be the number of sequences of non-negative integers satisfying the condition; output them over q lines. Each line should contain the elements of a sequence of non-negative integers satisfying the condition, in order, separated by spaces. For every sequence, all sequences outputted before it must be lexicographically smaller than it.

Sample Input 1

3 8

Sample Output 1

```
0 1 2
0 4 0
1 2 1
2 0 2
2 3 0
3 1 1
4 2 0
5 0 1
6 1 0
8 0 0
```

For example, for the sequence $(0, 1, 2)$ we have $0 \times 1 + 1 \times 2 + 2 \times 3 = 0 + 2 + 6 = 8$, so it satisfies the condition. There is no sequence satisfying the condition that is lexicographically smaller than this one, so output `0 1 2` on the first line.

Including $(0, 1, 2)$, ten sequences satisfy the condition. Output these in lexicographic order from smallest to largest.

Sample Input 2

```
1 200000
```

Sample Output 2

```
200000
```

Sample Input 3

```
8 9
```

Sample Output 3

```
0 0 0 1 1 0 0 0
0 0 1 0 0 1 0 0
0 0 3 0 0 0 0 0
0 1 0 0 0 0 1 0
0 1 1 1 0 0 0 0
0 2 0 0 1 0 0 0
0 3 1 0 0 0 0 0
1 0 0 0 0 0 0 1
1 0 0 2 0 0 0 0
1 0 1 0 1 0 0 0
1 1 0 0 0 1 0 0
1 1 2 0 0 0 0 0
1 2 0 1 0 0 0 0
1 4 0 0 0 0 0 0
2 0 0 0 0 0 1 0
2 0 1 1 0 0 0 0
2 1 0 0 1 0 0 0
2 2 1 0 0 0 0 0
3 0 0 0 0 1 0 0
3 0 2 0 0 0 0 0
3 1 0 1 0 0 0 0
3 3 0 0 0 0 0 0
4 0 0 0 1 0 0 0
4 1 1 0 0 0 0 0
5 0 0 1 0 0 0 0
5 2 0 0 0 0 0 0
6 0 1 0 0 0 0 0
7 1 0 0 0 0 0 0
9 0 0 0 0 0 0 0
```

E - K-Divisible Subarrays

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 450 points

Problem Statement

You are given a sequence of length N consisting of non-negative integers: $A = (A_1, A_2, \dots, A_N)$.

Define the **score** of a sequence of length 1 or more consisting of sequences of non-negative integers, $S = (S_1, S_2, \dots, S_k)$, as the number of sequences S_i ($1 \leq i \leq k$) whose sum of elements is divisible by K .

Find the maximum possible score of a sequence of non-negative integer sequences obtained by dividing A into one or more contiguous subsequences. Here, dividing a sequence A of length N into one or more contiguous subsequences means choosing a sequence of integers $l = (l_1, l_2, \dots, l_k)$ ($1 = l_1 < l_2 < \dots < l_k \leq N$) of length 1 or more, and forming the following k sequences.

- $(A_{l_i}, A_{l_i+1}, \dots, A_{l_{i+1}-1})$ ($1 \leq i \leq k$) (Here, let $l_{k+1} = N + 1$.)

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq K \leq 10^9$
- $0 \leq A_i < K$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N K
A_1 A_2 ... A_N
```

Output

Output the answer.

Sample Input 1

```
6 10
6 8 2 2 6 4
```

Sample Output 1

2

For example, A can be divided into four contiguous subsequences as $(6), (8, 2), (2), (6, 4)$. The sums of the elements of the second and fourth sequences are multiples of 10, so the score of $((6), (8, 2), (2), (6, 4))$ is 2.

A cannot be divided so that the score is 3 or more, so output 2.

Sample Input 2

8 1
0 0 0 0 0 0 0 0

Sample Output 2

8

Sample Input 3

30 8
5 0 4 2 7 3 2 3 2 4 0 1 4 0 4 1 7 5 2 5 0 3 6 6 2 3 2 2 4 2

Sample Output 3

8

F - A/AB Insertion

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 525 points

Problem Statement

You are given a string S of length N consisting of A and B.

Process a total of Q queries of the following types.

1 i c

Type 1: Change the i -th character of S to c .

2 l r

Type 2:

Let string T be the string obtained by extracting the l -th through r -th characters of the current string S .

If it is possible to obtain string T by the following operation, output Yes; otherwise, output No.

- Starting from the empty string, perform the following two operations any number of times, possibly zero, in any order.
 - Choose any position in the string (possibly the beginning or the end) and insert A there.
 - Choose any position in the string (possibly the beginning or the end) and insert AB there.

Constraints

- N is an integer satisfying $1 \leq N \leq 5 \times 10^5$.
- S is a string of length N consisting of A and B.
- Q is an integer satisfying $1 \leq Q \leq 2 \times 10^5$.
- Each given query is of type 1 or 2.
- Queries of type 1 satisfy the following constraints:
 - i is an integer satisfying $1 \leq i \leq N$, and
 - c is A or B.
- Queries of type 2 satisfy the following constraints:
 - l and r are integers satisfying $1 \leq l \leq r \leq N$.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $S$   
 $Q$   
Query1  
Query2  
⋮  
Query $Q$ 
```

Here, Query _{i} represents the i -th query, and the input format for a query follows the format described in the problem statement.

Output

Each time a query of type 2 is given, output the answer on one line.

Sample Input 1

```
10  
AABBAABABB  
6  
2 1 10  
1 5 B  
2 1 10  
2 6 8  
1 3 A  
2 1 10
```

Sample Output 1

```
Yes
No
Yes
Yes
```

This input contains six queries.

- Initially, $S = \text{AABBAABABB}$.
- For the 1-st query, $T = \text{AABBAABABB}$, obtained by extracting the 1-st through 10-th characters of S , can be obtained by the following steps, so output Yes.
 - Start from the empty string.
 - Insert AB at the beginning of the empty string, making the string AB.
 - Insert AB right after the 1-st character of AB, making the string AABB.
 - Insert AB at the end of AABB, making the string AABBAB.
 - Insert AB right after the 5-th character of AABBAB, making the string AABBAABB.
 - Insert AB right after the 7-th character of AABBAABB, making the string AABBAABABB.
- For the 2-nd query, change the 5-th character of S to B. As a result, $S = \text{AABBBABABB}$.
- For the 3-rd query, $T = \text{AABBBABABB}$, obtained by extracting the 1-st through 10-th characters of S , cannot be obtained by the operation described in the problem statement, so output No.
- For the 4-th query, $T = \text{ABA}$, obtained by extracting the 6-th through 8-th characters of S , can be obtained by the operation described in the problem statement, so output Yes.
- For the 5-th query, change the 3-rd character of S to A. As a result, $S = \text{AAABBABABB}$.
- For the 6-th query, $T = \text{AAABBABABB}$, obtained by extracting the 1-st through 10-th characters of S , can be obtained by the operation described in the problem statement, so output Yes.

G - Wipeout

Time Limit: 3 sec / Memory Limit: 1024 MiB

Score : 600 points

Problem Statement

There are N cards arranged face down in a row, each with one of the integers $1, 2, \dots, N$ written on its face.

The order of these cards is determined uniformly at random from the $N!$ possible orders.

You know that there is exactly one card with each of the integers $1, 2, \dots, N$ written on its face and that the cards were arranged uniformly at random, but you have no other information about the integers written on the faces of the cards.

You play the following game.

- Initially, let the variable $x = 1$.
- As long as $x \leq N$, repeat the following operation. One operation consists of the following three steps.
 - Specify one card and turn it face up.
 - If the integer written on the card is x , eat that card and add 1 to x .
 - Otherwise, turn that card face down. You can permanently remember the integer written on that card.

You always act so that the expected value of the total number of operations until all cards have been eaten is minimized.

In this case, what is the probability that the total number of operations is K ? Find it modulo 998244353.

► Definition of probability modulo 998244353

Constraints

- All input values are integers.
- $1 \leq N \leq 5 \times 10^5$
- $N \leq K \leq 10^9$

Input

The input is given from Standard Input in the following format:

N K

Output

Output the answer.

Sample Input 1

3 4

Sample Output 1

```
499122177
```

For this input, $N = 3$. Let us call the cards a, b, c in the order they are arranged.

Below is an example of your actions when acting so that the expected value of the total number of operations is minimized.

- First, turn a face up.
- If 1 is written on a , eat that card.
 - Next, turn b face up.
 - If 2 is written on b , eat that card.
 - Next, turn c face up; since 3 is necessarily written on it, eat it. In this case, you eat all cards in three operations, and the probability of this happening is $1/6$.
 - If 3 is written on b , turn that card face down.
 - Next, turn c face up; since 2 is necessarily written on it, eat it. After that, turn b face up and eat it. In this case, you eat all cards in four operations, and the probability of this happening is $1/6$.
- If 2 is written on a , turn that card face down.
 - Next, turn b face up.
 - If 1 is written on b , eat that card.
 - Next, turn a face up and eat it. Next, turn c face up; since 3 is necessarily written on it, eat it. In this case, you eat all cards in four operations, and the probability of this happening is $1/6$.
 - If 3 is written on b , turn that card face down.
 - At this point, it has become clear which integer is written on every one of the cards. So, perform three operations and eat all the cards in order of their numbers. In this case, you eat all cards in five operations, and the probability of this happening is $1/6$.
- If 3 is written on a , turn that card face down.
 - Next, turn b face up.
 - If 1 is written on b , eat that card.
 - Next, turn c face up; since 2 is necessarily written on it, eat it. After that, turn a face up and eat it. In this case, you eat all cards in four operations, and the probability of this happening is $1/6$.
 - If 2 is written on b , turn that card face down.
 - At this point, it has become clear which integer is written on every one of the cards. So, perform three operations and eat all the cards in order of their numbers. In this case, you eat all cards in five operations, and the probability of this happening is $1/6$.

Combining everything, the probability that the total number of operations is 3 is $1/6$, the probability that it is 4 is $1/2$, and the probability that it is 5 is $1/3$.

For this sample, output 499122177, which represents $1/2$ modulo 998244353.

Sample Input 2

```
3 6
```

Sample Output 2

```
0
```

Sample Input 3

```
500000 777777
```

Sample Output 3

```
251612105
```

Hints: AtCoder Beginner Contest 473

Taken from the official editorial, not written by us. Read one hint at a time.

A. Second Half Sum *(editorial, first line)*

The problem can be solved by performing the following two operations: Receive the array A from Standard Input.; Take the sum of the latter half of the array A , and print it.

B. Old Maid *(editorial, first line)*

This problem can be solved by the following procedure: inspect the card one by one, if there is no card with the same integer written on it, confirm that the card is never eaten;; if there is a card with ...

C. Change Schools *(editorial, first line)*

Let M be the maximum number of students in a class before Takahashi transfers.

D. Coefficient Stair *(editorial, first line)*

To exhaustively enumerate objects lexicographically, DFS (Depth-First Search) is effective.

E. K-Divisible Subarrays *(editorial, first line)*

The problem can be rephrased using the mod- K prefix sums S of A , defined as $S_i = \sum_{j=1}^i A_j \pmod K$: Define the score of an integer sequence by the number ...

F. A/AB Insertion *(editorial, first line)*

Interpret A as $+1$, and B as -1 .

G. Wipeout *(editorial, first line)*

It is meaningless to flip the same card three or more times.

Source: <https://atcoder.jp/contests/abc473/editorial>

A - A

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

You are given a string S consisting of uppercase English letters.

Output the string obtained by replacing every character in S other than A with ..

Constraints

- S is a string of length between 1 and 100 (inclusive) consisting of uppercase English letters.
-

Input

The input is given from Standard Input in the following format:

S

Output

Output the answer.

Sample Input 1

ATCODER

Sample Output 1

A.....

Since the first character is the only occurrence of A in S , output the string obtained by changing the second through seventh characters of S to ..

Sample Input 2

BANANA

Sample Output 2

.A.A.A

S may contain two or more occurrences of A.

Sample Input 3

CORRECT

Sample Output 3

.....

S may contain no occurrence of A.

B - Break a Stick

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

There is one stick. This stick has $N - 1$ notches, which divide it into N parts.

The lengths of the parts are $L_1, L_2, \dots, L_N$ in order from one end.

When choosing one notch and breaking the stick there to obtain two sticks, find the minimum possible absolute value of the difference between the lengths of the two resulting sticks.

Here, the width of a notch can be ignored, and the length of each resulting stick is the sum of the lengths of the parts it contains.

Constraints

- $2 \leq N \leq 100$
- $1 \leq L_i \leq 10^5$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N
L_1 L_2 ... L_N
```

Output

Output the answer.

Sample Input 1

```
4
5 2 3 8
```

Sample Output 1

```
2
```

Breaking the stick at each notch results in the following:

- Breaking at the first notch from the end results in two sticks of lengths **5** and **13**, with an absolute difference of **8**.
- Breaking at the second notch from the end results in two sticks of lengths **7** and **11**, with an absolute difference of **4**.
- Breaking at the third notch from the end results in two sticks of lengths **10** and **8**, with an absolute difference of **2**.

Sample Input 2

```
7
31 41 59 26 53 58 97
```

Sample Output 2

```
51
```

Sample Input 3

```
10
67011 35764 33042 24098 63738 98760 17199 68579 21812 45408
```

Sample Output 3

```
28105
```

C - On a Diet

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

Takahashi is staying at his parents' house for an N -day homecoming trip.

At his parents' house, a snack is prepared every day, and the calorie count of the snack on day i is A_i .

To manage his health, he repeats the following: he eats the snack on a given day if and only if the total calorie count of the snacks he has eaten in the most recent M days does not exceed K .

Specifically, in the order $i = 1, 2, \dots, N$, he decides whether to eat the snack on day i according to the following rule:

- Assuming that he eats the snack on day i , if the total calorie count of the snacks eaten from day $\max(i - M + 1, 1)$ through day i is at most K , then he actually eats the snack on day i . Otherwise, he does not eat the snack on day i .

For each $i = 1, 2, \dots, N$, determine whether Takahashi eats the snack on day i .

Constraints

- $1 \leq M \leq N \leq 2 \times 10^5$
- $1 \leq K \leq 10^{15}$
- $1 \leq A_i \leq 10^9$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   $M$   $K$   
 $A_1$   $A_2$   $\dots$   $A_N$ 
```

Output

Output N lines. The i -th line should contain Yes if Takahashi eats the snack on day i , and No otherwise.

Sample Input 1

```
5 3 83
48 73 59 90 21
```

Sample Output 1

```
Yes
No
No
No
Yes
```

For each day, assuming that he eats the snack, the total calorie count of the snacks eaten in the most recent three days is as follows:

- Day 1: 48
- Day 2: $48 + 73 = 121$
- Day 3: $48 + 59 = 107$
- Day 4: 90
- Day 5: 21

Sample Input 2

```
7 4 728
187 816 349 609 255 308 175
```

Sample Output 2

```
Yes
No
Yes
No
Yes
No
Yes
```

Sample Input 3

```
10 3 1368290936
216519459 804733999 297250023 775422599 287963235 999315644 354987425 974810607 653940822 1
17157941
```

Sample Output 3

```
Yes
Yes
Yes
No
Yes
Yes
No
No
Yes
Yes
```

D - Bomber Mad

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 400 points

Problem Statement

There is a grid with H rows and W columns. Each cell is an empty cell or a bomb cell. Let (i, j) denote the cell at the i -th row from the top and the j -th column from the left. The grid is given by H length- W strings $S_1, S_2, \dots, S_H$: if the j -th character of S_i is `.`, (i, j) is an empty cell, and if the j -th character of S_i is `#`, (i, j) is a bomb cell.

For an empty cell (i, j) , if there is no bomb cell in the i -th row nor in the j -th column, that cell is called a **safe empty cell**.

In one move, you can move from the current cell to an adjacent empty cell in the up, down, left, or right direction (you cannot move to a bomb cell). Find the number of empty cells (i, j) satisfying the following condition:

- A **safe empty cell** can be reached from (i, j) in at most K moves.

Constraints

- $1 \leq H, W \leq 5 \times 10^5$
- $H \times W \leq 5 \times 10^5$
- $0 \leq K \leq H \times W - 1$
- S_i is a string of length W consisting of `.` and `#`.
- H, W , and K are integers.

Input

The input is given from Standard Input in the following format:

```
H W K
S1
S2
⋮
SH
```

Output

Output the number of empty cells satisfying the condition.

Sample Input 1

```
3 3 1
#..
...
..#
```

Sample Output 1

5

The only **safe empty cell** is $(2, 2)$. There are five empty cells from which you can reach $(2, 2)$ in at most one move: $(1, 2)$, $(2, 1)$, $(2, 2)$, $(2, 3)$, $(3, 2)$, so the answer is 5.

Sample Input 2

```
2 3 0
...
...
```

Sample Output 2

6

Since there is no bomb cell, all six cells are **safe empty cells**. Therefore, every empty cell satisfies the condition with zero moves.

Sample Input 3

```
5 7 2
..#....
..#....
.....
...#...
...#...
```

Sample Output 3

29

E - Odd Cycle

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 450 points

Problem Statement

You are given a simple connected undirected graph with N vertices numbered 1 through N and M edges. The i -th edge connects vertices a_i and b_i .

Determine whether there exists a cycle consisting of an odd number of vertices, and if one exists, find one such cycle.

Formally, determine whether there exists an integer sequence $(v_1, v_2, \dots, v_K)$ satisfying all of the following conditions, and if one exists, find one such sequence.

- K is an odd number at least 3.
- $v_1, v_2, \dots, v_K$ are all distinct.
- For every integer i with $1 \leq i \leq K$, there is an edge between vertices v_i and v_{i+1} , where $v_{K+1} = v_1$.

You are given T test cases; solve each of them.

Constraints

- $1 \leq T \leq 2 \times 10^5$
 - $1 \leq N, M \leq 2 \times 10^5$
 - The sum of N over all test cases is at most 2×10^5 .
 - The sum of M over all test cases is at most 2×10^5 .
 - $1 \leq a_i, b_i \leq N$
 - $a_i \neq b_i$
 - The given graph is a simple connected undirected graph.
 - All input values are integers.
-

Input

The input is given from Standard Input in the following format:

```
 $T$   
 $\text{case}_1$   
 $\text{case}_2$   
 $\vdots$   
 $\text{case}_T$ 
```

Each test case is given in the following format:

```
 $N$   $M$   
 $a_1$   $b_1$   
 $a_2$   $b_2$   
 $\vdots$   
 $a_M$   $b_M$ 
```

Output

For each test case, if there is no sequence satisfying the conditions, output -1. If one exists, output one such sequence in the following format:

```
 $K$   
 $v_1$   $v_2$   $\dots$   $v_K$ 
```

If multiple sequences satisfy the conditions, any of them will be accepted.

Sample Input 1

```
4
3 3
1 2
2 3
1 3
7 7
1 2
2 3
3 4
1 4
4 5
5 6
6 7
5 5
1 2
2 3
3 4
4 5
1 5
9 10
1 2
2 3
3 4
4 5
1 5
6 7
7 8
8 9
6 9
1 6
```

Sample Output 1

```
3
2 1 3
-1
5
3 2 1 5 4
5
3 2 1 5 4
```

In the first test case, the sequence $(2, 1, 3)$ satisfies the conditions: the edges $(2, 1)$, $(1, 3)$, $(3, 2)$ all exist. Outputs such as $v = (2, 3, 1)$ are also accepted.

In the second test case, there is no cycle with an odd number of vertices, so no sequence satisfies the conditions.

F - Centroid of a Slice

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 525 points

Problem Statement

There is a convex N -gon P on the xy -plane. The vertices of P are numbered $1, 2, \dots, N$ counterclockwise, and the coordinates of vertex i are (x_i, y_i) .

Answer Q questions.

The j -th question gives two distinct vertices u_j and v_j . Here, the vertices u_j and v_j are not adjacent on the boundary of P .

The segment connecting vertices u_j and v_j divides P into two polygons; let P' be the one lying to the right of the directed line from vertex u_j to vertex v_j . Find the coordinates of the geometric center of P' (the centroid obtained by regarding the interior of the polygon as a thin plate of uniform density).

Constraints

- $4 \leq N \leq 3 \times 10^4$
- $1 \leq Q \leq 2 \times 10^5$
- $|x_i|, |y_i| \leq 5 \times 10^5$
- $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$ form a convex polygon in counterclockwise order.
- All interior angles of P are less than 180 degrees.
- $1 \leq u_j, v_j \leq N$
- $u_j \neq v_j$
- Vertices u_j and v_j are not adjacent on the boundary of P .
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   $Q$   
 $x_1$   $y_1$   
 $x_2$   $y_2$   
 $\vdots$   
 $x_N$   $y_N$   
 $u_1$   $v_1$   
 $u_2$   $v_2$   
 $\vdots$   
 $u_Q$   $v_Q$ 
```

Output

Output Q lines. On the j -th line, letting (p_j, q_j) be the coordinates of the answer to the j -th question, output p_j and q_j in this order, separated by a space.

The answer is considered correct if, for each of p_j and q_j , its absolute or relative error from the true value is at most 10^{-6} .

Sample Input 1

```
5 4  
4 1  
1 3  
-3 2  
-1 -2  
2 -1  
2 4  
5 3  
1 4  
4 2
```

Sample Output 1

```
-1.0000000000000000 1.0000000000000000
0.864197530864198 1.209876543209877
0.198198198198198 0.828828828828829
1.391304347826087 0.434782608695652
```

For the first question, P' is the triangle with vertices $(1, 3)$, $(-3, 2)$, $(-1, -2)$, and its geometric center is $(-1, 1)$.

For the second question, P' is the quadrilateral with vertices $(2, -1)$, $(4, 1)$, $(1, 3)$, $(-3, 2)$, and its geometric center is $(\frac{70}{81}, \frac{98}{81})$.

Sample Input 2

```
10 10
480451 142016
404696 274257
7097 390813
-134582 195390
-167108 143104
-359470 -180367
-406293 -273476
74335 -494832
486873 -391664
493159 -292349
5 8
8 6
5 2
5 1
10 3
1 7
3 5
2 5
4 9
1 8
```

Sample Output 2

```
-167917.431926206833203 -207268.968943785114349
126221.026579795702381 -68771.715180365014040
111801.954824466955586 -124063.178438661167072
99590.935535112756283 -153792.378227918737825
321662.353363423237819 113722.100435559268610
43574.061369374611386 102411.022160871806300
-98197.666666666666667 243102.333333333333333
79984.488622522356481 269160.678085376751920
-3267.792578363696043 -210454.379232334372417
46257.363593155145794 -38546.483790939251583
```


G - Cascading Grid

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 600 points

Problem Statement

There is a grid with H rows and W columns. Each cell has one of the characters $+$, $-$, $\#$ written on it. Let (i, j) denote the cell at the i -th row from the top and the j -th column from the left. The grid is given by H length- W strings $S_1, S_2, \dots, S_H$: the j -th character of S_i is written on (i, j) .

You can perform the following operation zero or more times:

- Choose one cell that is not $\#$. Change to $\#$ all cells that are "reachable from the chosen cell only by moving to an adjacent cell in the left, right, or down direction without passing through a cell that is $\#$." Here, the chosen cell itself is included among the reachable cells.

Find the maximum possible value of the following value in the grid after the operations: the number of $+$ cells minus the number of $-$ cells.

Constraints

- $1 \leq H, W \leq 30$
- S_i is a string of length W consisting of $+$, $-$, $\#$.
- H and W are integers.

Input

The input is given from Standard Input in the following format:

```
 $H$   $W$ 
 $S_1$ 
 $S_2$ 
 $\vdots$ 
 $S_H$ 
```

Output

Output the answer.

Sample Input 1

```
2 3
+-+
--+
```

Sample Output 1

```
1
```

If you choose $(2, 1)$, all cells in row 2 change to # (note that you cannot move upward). The remaining row 1 has two cells of + and one cell of -, so the value is $2 - 1 = 1$, and this is the maximum.

Sample Input 2

```
3 3
+ - -
-#-
#+#
```

Sample Output 2

```
1
```

If you choose $(1, 1)$, all cells except $(3, 2)$ become #. Note that the chosen cell $(1, 1)$ itself is included among the reachable cells, and that you cannot pass through a cell that is #.

Sample Input 3

```
5 7
++#--++
-+---+#
###+-++
--#-++-
+---#++
```

Sample Output 3

```
5
```

Hints: AtCoder Beginner Contest 472

Taken from the official editorial, not written by us. Read one hint at a time.

A. A *(editorial, first line)*

(C++) Replace characters one by one (Python) Print the answer one by one (C++) Construct another answer string

B. Break a Stick *(editorial, first line)*

When the stick is broken at the i -th notch, the two sticks have lengths $L_1 + L_2 + \dots + L_i$ and $L_{i+1} + L_{i+2} + \dots + L_N$.

C. On a Diet *(editorial, first line)*

Perform the simulation actually.

D. Bomber Mad *(editorial, first line)*

First, by managing whether each row and each column contains a bomb, we can enumerate all safe empty cells in $O(HW)$ time.

E. Odd Cycle *(editorial, first line)*

To get straight to the point, the following algorithm is valid: Take a spanning tree, and color each vertex with 0 or 1 so that $C_u \neq C_v$ for all edge (u, v) in the spanning tree.; ...

F. Centroid of a Slice *(editorial, first line)*

For a polygon P , denote its signed area by $\text{area}(P)$, its geometric center by $g(P)$.

G. Cascading Grid *(editorial, first line)*

For simplicity, we regard + and - collectively as empty cells ..

Source: <https://atcoder.jp/contests/abc472/editorial>

A - Nine or Nein

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

You are given positive integers A and B .

If at least one of the following values is equal to 9, output Nine; otherwise, output Nein.

- $A + B$
- $A - B$
- $A \times B$
- $A \div B$

Constraints

- $1 \leq A \leq 100$
- $1 \leq B \leq 100$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

A B

Output

Output the answer.

Sample Input 1

16 7

Sample Output 1

Nine

Since $16 - 7 = 9$, output Nine.

Sample Input 2

66 7

Sample Output 2

Nein

None of the following values is equal to 9, so output Nein.

- $66 + 7 = 73$
 - $66 - 7 = 59$
 - $66 \times 7 = 462$
 - $66 \div 7 = \frac{66}{7}$
-

Sample Input 3

9 1

Sample Output 3

Nine

Sample Input 4

9 9

Sample Output 4

Nein

B - Survey Tabulation

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

Takahashi is tallying the results of a survey.

N people answered the survey, and the i -th person's answer is a string S_i consisting of English letters.

Find the maximum number of people who gave the same answer in this survey.

Here, the case of the letters in the answers is not distinguished.

For example, AtCoder, ATCODER, and atcoder are all considered the same answer.

Constraints

- $1 \leq N \leq 100$
- S_i is a string of length between 1 and 10 (inclusive) consisting of uppercase and lowercase English letters.
- N is an integer.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $S_1$   
 $\vdots$   
 $S_N$ 
```

Output

Output the answer.

Sample Input 1

```
7
ARC
abc
ahc
ABC
beginner
AbC
ahc
```

Sample Output 1

```
3
```

There are three answers that are the same as ABC, and this number is the maximum.

Sample Input 2

```
10
x
x
x
x
x
x
x
x
x
x
x
x
x
```

Sample Output 2

```
10
```

C - Cookies and Greedy Takahashi

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

There are cookies at N positions on a number line. The coordinate of the i -th cookie is A_i .

Takahashi is initially at coordinate 0 on the number line, and repeats the following action until he has picked up all N cookies.

- Action: Move to the coordinate of the nearest cookie from his current position (if there are multiple such cookies, the one with the smallest coordinate), and pick up that cookie.

Find the total distance Takahashi travels until he picks up all the cookies.

Constraints

- $1 \leq N \leq 3 \times 10^5$
- $-10^9 \leq A_i \leq 10^9$
- $A_i \neq 0$
- The A_i are distinct.
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $A_1 \ \dots \ A_N$ 
```

Output

Output the answer.

Sample Input 1

```
4  
-1 -4 2 -11
```

Sample Output 1

```
23
```

Takahashi acts as follows.

- He moves from coordinate 0 to coordinate -1 and picks up the cookie. The distance traveled is 1 .
- He moves from coordinate -1 to coordinate -4 and picks up the cookie. The distance traveled is 3 .
- He moves from coordinate -4 to coordinate 2 and picks up the cookie. The distance traveled is 6 .
- He moves from coordinate 2 to coordinate -11 and picks up the cookie. The distance traveled is 13 .

Thus, the total distance traveled is $1 + 3 + 6 + 13 = 23$.

In the second action, the distances to the cookie at coordinate -4 and the cookie at coordinate 2 are both 3 , and Takahashi moves to -4 , the smaller coordinate.

Sample Input 2

```
10
1 2 3 4 5 -1 -2 -3 -4 -6
```

Sample Output 2

```
17
```

D - Chargers

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 400 points

Problem Statement

There is a charger with an unlimited number of charging slots. At time 0, all slots are empty.

The maximum capacity of a battery is V . While a battery is plugged into a slot, it is charged at a rate of 1 until its charge level reaches the maximum capacity (that is, the charge level increases by 1 for every 1 unit of time that passes).

Process Q queries in order. The q -th query is given in one of the following formats. Here, it is guaranteed that $t_1 < \dots < t_Q$.

- Type 1 (1 t_q w_q): At time t_q , plug one battery with charge level w_q into a slot.
- Type 2 (2 t_q): At time t_q , unplug one battery with the highest charge level from its slot, and output that battery's charge level. If there is no battery plugged into any slot, output -1 instead.

Constraints

- $1 \leq Q \leq 3 \times 10^5$
 - $1 \leq V \leq 10^9$
 - For type 1 queries, $1 \leq t_q \leq 10^9$.
 - For type 1 queries, $0 \leq w_q \leq V$.
 - For type 2 queries, $1 \leq t_q \leq 10^9$.
 - $t_1 < \dots < t_Q$
 - The input values are all integers.
-

Input

The input is given from Standard Input in the following format:

```
 $Q \ V$   
query1  
⋮  
query $Q$ 
```

Here, query _{q} represents the q -th query, and is given in one of the following two formats:

```
1   $t_q$    $w_q$ 
```

```
2   $t_q$ 
```

Output

Let x be the number of type 2 queries. Output x lines.

The k -th line ($1 \leq k \leq x$) should contain the value to be outputted for the k -th type 2 query.

Sample Input 1

```
7 100  
1 15 60  
1 25 80  
2 30  
1 45 0  
2 60  
2 70  
2 80
```

Sample Output 1

```
85
100
25
-1
```

The seven queries are processed in the following order.

- At time 15, a battery with charge level 60 is plugged in. At this point, the charger has a battery with charge level 60.
- At time 25, a battery with charge level 80 is plugged in. At this point, the charger has batteries with charge levels 70, 80.
- At time 30, the charger has batteries with charge levels 75, 85. Of these, the battery with charge level 85 is unplugged.
- At time 45, a battery with charge level 0 is plugged in. At this point, the charger has batteries with charge levels 0, 90.
- At time 60, the charger has batteries with charge levels 15, 100. Of these, the battery with charge level 100 is unplugged.
- At time 70, the charger has a battery with charge level 25. This battery, with charge level 25, is unplugged.
- At time 80, no battery is plugged into the charger. Thus, no battery is unplugged.

Sample Input 2

```
20 380736236
1 21873985 256702097
2 86369729
1 114301317 288304981
1 147244640 305840435
2 150951976
1 331581391 50335458
1 352989552 47577202
1 400130024 345362760
2 458793150
2 509082216
1 591375600 197371572
1 617022014 101276068
1 679649471 310249627
1 796351653 268586022
1 825648347 129608152
2 908069704
2 921770319
1 949684819 372272469
1 971850999 335461408
2 986253026
```

Sample Output 2

```
321197841
324955640
380736236
380736236
380736236
380736236
380736236
```

E - Sum of Square of Sum

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 450 points

Problem Statement

There are N balls numbered 1 to N . Ball i has an integer A_i written on it.

For a way of choosing some balls from the N balls, define the **score** of that choice as the square of the sum of the numbers written on the chosen balls.

Find the sum, modulo 998244353, of the scores of all $\binom{N}{K}$ ways of choosing K balls from the N balls.

Constraints

- $1 \leq K \leq N \leq 2 \times 10^5$
- $1 \leq A_i \leq 10^9$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   $K$   
 $A_1$  ...  $A_N$ 
```

Output

Output the answer.

Sample Input 1

```
3 2  
1 10 100
```

Sample Output 1

```
22422
```

There are three ways of choosing two balls from three balls: choosing balls 1, 2 gives a score of $(1 + 10)^2 = 121$, choosing balls 1, 3 gives a score of $(1 + 100)^2 = 10201$, and choosing balls 2, 3 gives a score of $(10 + 100)^2 = 12100$.

The answer is the sum of these, $121 + 10201 + 12100 = 22422$.

Sample Input 2

```
5 2
10 10 20 20 20
```

Sample Output 2

```
10600
```

Multiple balls may have the same number written on them.

Sample Input 3

```
2 1
998244353 998244353
```

Sample Output 3

```
0
```

Find the sum modulo 998244353.

F - Concat (maximize)

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 550 points

Problem Statement

You are given N strings $S_1, \dots, S_N$ consisting of digits.

Find the maximum possible value, when interpreted as a decimal integer (after removing extra leading zeros), of a string obtained by choosing exactly K of these strings and concatenating them in any order of your choice.

Here, the value obtained by interpreting a string consisting entirely of the character 0 as a decimal integer after removing extra leading zeros is defined to be 0.

Constraints

- $1 \leq K \leq N \leq 10^5$
- S_i is a string of length between 1 and 10 (inclusive) consisting of digits.
- N and K are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   $K$   
 $S_1$   
 $\vdots$   
 $S_N$ 
```

Output

Output the answer.

Sample Input 1

```
3 2
123
45
6
```

Sample Output 1

```
45123
```

There are six strings obtained by choosing two of 123, 45, 6 and concatenating them: 12345, 1236, 45123, 456, 6123, 645.

Among these, the maximum value when interpreted as a decimal integer is 45123.

Sample Input 2

```
5 3
00000
00
1
0000
000
```

Sample Output 2

```
1000000000
```

Sample Input 3

```
9 9
0000000001
000000002
00000003
0000004
000005
00006
0007
008
09
```

Sample Output 3

```
90080007000060000050000004000000030000000020000000001
```

The answer must be output without extra leading zeros.

G - Caesar Syllables

Time Limit: 5 sec / Memory Limit: 1024 MiB

Score : 600 points

Problem Statement

There are K kinds of symbols numbered 0 to $K - 1$.

Each symbol is either a vowel or not a vowel. If $V_j = 1$, symbol j is a **vowel**; if $V_j = 0$, symbol j is not a vowel.

Define the **number of syllables** of a symbol string as the number of maximal contiguous substrings of that string consisting of vowels. Formally, the number of syllables of a length- N symbol string $(a_1, \dots, a_N)$ is defined as the number of pairs of integers (l, r) satisfying $1 \leq l \leq r \leq N$ and all of the following:

- The symbols $a_l, \dots, a_r$ are all vowels.
- If $l > 1$, symbol a_{l-1} is not a vowel.
- If $r < N$, symbol a_{r+1} is not a vowel.

You are given a length- N sequence $A = (A_1, \dots, A_N)$. For each $k = 0, \dots, K - 1$, answer the following question:

- Define a length- N symbol string $A' = (A'_1, \dots, A'_N)$ by $A'_i := (A_i + k) \bmod K$. What is the number of syllables of A' ?

Input Format

The input for this problem is given in a special format.

Instead of $A_1, \dots, A_N$, the integers seed , M , $b_1, \dots, b_M$ are given from Standard Input. Restore $A_1, \dots, A_N$ using the computation represented by the following pseudocode.

Here, all variables in the pseudocode are unsigned 64-bit integers. $s \oplus t$ denotes the bitwise XOR of s and t , $s \gg t$ denotes $\lfloor s/2^t \rfloor$ (the right shift operation), and $s \ll t$ denotes $s \times 2^t$ (the left shift operation).

```
state ← seed

for i = 1, ..., N:
    if i ≤ M:
        A_i ← b_i
    else:
        x ← (((state ≫ 18) ⊕ state) ≫ 27) mod 232
        r ← state ≫ 59
        y ← ((x ≫ r) + (x ≪ (32 − r))) mod 232
        A_i ← y mod K
        state ← (state × 6364136223846793005 + 2026081520260815) mod 264
```

Constraints

- $1 \leq N \leq 7 \times 10^6$
- $1 \leq K \leq 2300$
- $0 \leq A_i \leq K - 1$ ($1 \leq i \leq N$)
- $V_j \in \{0, 1\}$ ($0 \leq j \leq K - 1$)
- $0 \leq \text{seed} \leq 2^{60} - 1$
- $1 \leq M \leq \min(N, 10^5)$
- $0 \leq b_i \leq K - 1$ ($1 \leq i \leq M$)
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N  K  seed  M
b_1 b_2 ... b_M
V_0 V_1 ... V_{K-1}
```

Output

Output K lines. The m -th line ($1 \leq m \leq K$) should contain the answer for $k = m - 1$.

Sample Input 1

```
4 6 12233445577788999 4
4 2 4 1
1 1 0 0 1 0
```

Sample Output 1

```
2
0
1
2
1
2
```

In this input, $A = (4, 2, 4, 1)$.

- For $k = 0$, $A' = (4, 2, 4, 1)$, and the number of syllables is 2.
- For $k = 1$, $A' = (5, 3, 5, 2)$, and the number of syllables is 0.
- For $k = 2$, $A' = (0, 4, 0, 3)$, and the number of syllables is 1.
- For $k = 3$, $A' = (1, 5, 1, 4)$, and the number of syllables is 2.
- For $k = 4$, $A' = (2, 0, 2, 5)$, and the number of syllables is 1.
- For $k = 5$, $A' = (3, 1, 3, 0)$, and the number of syllables is 2.

Sample Input 2

```
15 12 998154573227378904 2
5 6
0 0 1 1 0 1 0 0 1 1 0 1
```

Sample Output 2

```
4
4
3
3
3
4
4
4
3
3
3
4
```

In this input, $A = (5, 6, 0, 6, 8, 8, 8, 3, 0, 2, 3, 7, 3, 2, 5)$.

Sample Input 3

```
7000000 15 409873722375451899 3
7 0 4
1 0 1 1 1 0 1 1 1 0 1 0 1 0 0
```

Sample Output 3

```
1680078
1681239
1679837
1680730
1680470
1679454
1679615
1679371
1681263
1679670
1680218
1680010
1680211
1680521
1681744
```

Hints: AtCoder Beginner Contest 471

Taken from the official editorial, not written by us. Read one hint at a time.

A. Nine or Nein *(editorial, first line)*

Maintain a Boolean variable `hasnine` that manages whether any condition is satisfied, initialized with “false.” Check each of the four condition; if it is satisfied, set `hasnine` ...

B. Survey Tabulation *(editorial, first line)*

We can regard two answers as the same answer if and only if the two answers are equal as strings when converted to lowercase.

C. Cookies and Greedy Takahashi *(editorial, first line)*

On making an action, the cookie closest to Takahashi's current coordinate X is always either “the largest coordinate less than X ” or “the smallest coordinate greater than X .” If these values ...

D. Chargers *(editorial, first line)*

For the battery plugged at time t_0 with charge level w_0 , the charge level at time t is $w = \min(w_0 + (t - t_0), V)$.

E. Sum of Square of Sum *(editorial, first line)*

For $N = 1$, the answer is obvious.

F. Concat (maximize) *(editorial, first line)*

Let $L = \max_i |S_i|$.; Denote the concatenation of two strings A and B by $A + B$.; For a digit string S , let $\text{INT}(S)$ be the integer represented by S in decimal.; ...

G. Caesar Syllables *(editorial, first line)*

In this article, we denote by $[x^i]P$ the coefficient of x^i in a polynomial P , and use the same notation for multi-variable polynomials too.

Source: <https://atcoder.jp/contests/abc471/editorial>

A - Fizz

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

You are given a positive integer N .

Output N lines. The i -th line ($1 \leq i \leq N$) should contain `Fizz` if i is a multiple of `3`, and i if i is not a multiple of `3`.

Constraints

- N is an integer between 1 and 100, inclusive.

Input

The input is given from Standard Input in the following format:

N

Output

Output the answer.

Sample Input 1

4

Sample Output 1

```
1
2
Fizz
4
```

The first line should contain **1** since **1** is not a multiple of **3**.

The second line should contain **2** since **2** is not a multiple of **3**.

The third line should contain **Fizz** since **3** is a multiple of **3**.

The fourth line should contain **4** since **4** is not a multiple of **3**.

Sample Input 2

```
10
```

Sample Output 2

```
1
2
Fizz
4
5
Fizz
7
8
Fizz
10
```

B - Monocolor

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

There are N balls. Each ball is painted in one of N colors: color 1 through color N . The color of the i -th ball ($1 \leq i \leq N$) is C_i .

In one operation, you can change the color of any one ball to any of the N colors.

Find the minimum number of operations required to make all the balls the same color.

Constraints

- $1 \leq N \leq 100$
 - $1 \leq C_i \leq N$
 - All input values are integers.
-

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $C_1$   $C_2$   $\dots$   $C_N$ 
```

Output

Output the answer.

Sample Input 1

```
4  
3 1 2 1
```

Sample Output 1

```
2
```

By changing the color of the first ball to 1 and the color of the third ball to 1, you can make all the balls the same color.

It is impossible to make all the balls the same color by changing the colors of fewer than two balls, so output 2.

Sample Input 2

```
5
3 3 3 3 3
```

Sample Output 2

```
0
```

All balls may have the same color from the beginning.

Sample Input 3

```
9
4 2 3 3 4 1 2 7 1
```

Sample Output 3

```
7
```

C - Inc, Dec, Xor

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

There is a length- N integer sequence $A = (A_1, A_2, \dots, A_N)$. Initially, all elements of A are 0.

You will be given Q queries, which should be processed in order. There are two types of queries, each given in one of the following formats:

- 1 x : Increase the value of A_x by 1.
- 2: For each $i = 1, 2, \dots, N$, if $A_i \geq 1$, decrease the value of A_i by 1.

Find the bitwise XOR of $A_1, A_2, \dots, A_N$ immediately after processing each query.

► What is bitwise XOR?

Constraints

- $1 \leq N \leq 5 \times 10^5$
- $1 \leq Q \leq 5 \times 10^5$
- $1 \leq x \leq N$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N Q
query1
query2
⋮
queryQ
```

Each query is given in one of the following 2 formats:

```
1 x
```

```
2
```

Output

Output Q lines.

The i -th line ($1 \leq i \leq Q$) should contain the bitwise XOR of $A_1, A_2, \dots, A_N$ for A immediately after processing the i -th query.

Sample Input 1

```
2 5
1 2
1 2
1 1
2
2
```

Sample Output 1

```
1
2
3
1
0
```

After processing the first query, $A = (0, 1)$. The bitwise XOR of 0, 1 is 1, so output 1 on the first line.

After processing the second query, $A = (0, 2)$. The bitwise XOR of 0, 2 is 2, so output 2 on the second line.

After processing the third query, $A = (1, 2)$. The bitwise XOR of 1, 2 is 3, so output 3 on the third line.

After processing the fourth query, $A = (0, 1)$. The bitwise XOR of 0, 1 is 1, so output 1 on the fourth line.

After processing the fifth query, $A = (0, 0)$. The bitwise XOR of 0, 0 is 0, so output 0 on the fifth line.

Sample Input 2

```
3 8
1 2
1 3
1 1
1 2
1 1
2
1 3
1 1
```

Sample Output 2

```
1
0
1
2
1
0
1
2
```

D - Inverse and Swap

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 400 points

Problem Statement

You are given a permutation $P = (P_1, \dots, P_N)$ of $(1, \dots, N)$.

Process Q queries in order. There are two types of queries as follows:

- 1 $\times$ y : Swap the values of P_x and P_y .
- 2: Construct the permutation $P' = (P'_1, \dots, P'_N)$ of $(1, \dots, N)$ satisfying the following condition, and replace the values of $P_1, \dots, P_N$ with $P'_1, \dots, P'_N$, respectively. (One can prove that such P' uniquely exists.)
 - $P_{P'_i} = i$ for every integer i satisfying $1 \leq i \leq N$.

Output the values of $P_1, \dots, P_N$ after processing all queries.

Constraints

- $2 \leq N \leq 5 \times 10^5$
 - $1 \leq Q \leq 5 \times 10^5$
 - $(P_1, \dots, P_N)$ is a permutation of $(1, \dots, N)$.
 - $1 \leq x < y \leq N$ for queries of type 1.
 - All input values are integers.
-

Input

The input is given from Standard Input in the following format:

```

 $N$   $Q$ 
 $P_1$   $P_2$   $\dots$   $P_N$ 
query1
⋮
query $Q$ 

```

Here, query _{q} represents the q -th query, and is given in one of the following two formats:

```
1  $x$   $y$ 
```

```
2
```

Output

Output the values of $P_1, \dots, P_N$ after processing all queries, separated by spaces, on one line.

Sample Input 1

```

5 5
2 1 3 5 4
1 2 4
2
1 2 3
1 3 4
2

```

Sample Output 1

```
4 5 2 1 3
```

At the point when each query has been processed, the values of $P_1, \dots, P_N$ are as follows:

- After processing the first query, $P = (2, 5, 3, 1, 4)$.
- After processing the second query, $P = (4, 1, 3, 5, 2)$.
- After processing the third query, $P = (4, 3, 1, 5, 2)$.
- After processing the fourth query, $P = (4, 3, 5, 1, 2)$.
- After processing the fifth query, $P = (4, 5, 2, 1, 3)$.

Sample Input 2

```
7 4
3 7 5 6 4 2 1
2
2
2
2
```

Sample Output 2

```
3 7 5 6 4 2 1
```

Sample Input 3

```
10 8
7 3 2 4 8 5 10 9 1 6
2
1 4 10
1 6 9
2
1 9 10
1 3 10
2
1 4 6
```

Sample Output 3

```
3 10 2 8 6 7 1 5 9 4
```

E - Concentration

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 450 points

Problem Statement

Takahashi is playing a solitaire game similar to the memory-matching game.

There are $2N$ cards. A number is written on the front of each card, and nothing is written on the back. For each i satisfying $1 \leq i \leq N$, there are exactly two cards on which A_i is written. (The A_i are pairwise distinct.)

Takahashi plays the game with these cards using the following procedure.

- Shuffle the $2N$ cards and lay them out face down.
- Set **life** to L and **score** to 0.
- Repeat the following until life becomes 0 or there are no cards left on the table:
 - Choose one face-down card on the table, turn it face up, and check the number X written on it.
 - Choose one face-down card on the table, turn it face up, and check the number Y written on it.
 - If $X = Y$, remove those two cards from the table, and increase the score by X .
 - If $X \neq Y$, turn the two cards face down again, and decrease life by 1.

Find the expected value of the score at the end of the game when Takahashi acts optimally to maximize the score at the end of the game.

Below is a more formal description of the game.

- Takahashi knows the rules of this game described below.
- Takahashi knows the values of $A_1, \dots, A_N$.
- Let B be a sequence obtained by choosing a permutation of the length- $2N$ sequence $(A_1, A_1, A_2, A_2, \dots, A_N, A_N)$ uniformly at random.
- Initially, Takahashi knows none of the values of B . Once he learns the value of B_i , he remembers it completely thereafter.
- Let life be L , score be 0, and S be $\{1, 2, 3, \dots, 2N\}$. Takahashi always knows these values.
- Repeat the following until life becomes 0 or S becomes empty:
 - Based on the information obtained up to this point, Takahashi chooses an element from S , and calls it i .
 - The value of B_i is revealed, and Takahashi learns it.
 - Based on the information obtained up to this point (including the value of B_i), Takahashi chooses an element from $S \setminus \{i\}$, and calls it j .
 - The value of B_j is revealed, and Takahashi learns it.
 - If $B_i = B_j$, remove i and j from S , and add B_i to the score.
 - If $B_i \neq B_j$, decrease life by 1.
- Takahashi acts optimally to maximize the expected value of the score at the end of the game.

Constraints

- $1 \leq N \leq 200$
- $1 \leq L \leq 200$
- $1 \leq A_1 < A_2 < \dots < A_N \leq 10^5$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   $L$   
 $A_1$   $A_2$   $\dots$   $A_N$ 
```

Output

Output the answer.

Your output will be considered correct if its absolute or relative error from the true answer is at most 10^{-5} .

Sample Input 1

```
3 2  
1 2 3
```

Sample Output 1

```
3.8666666667
```

The game may proceed as follows, for example. To distinguish the six cards, let us call them A, B, C, D, E, F.

- Start the game with life 2 and score 0.
- Turn card A face up. 3 is written on it.
- Turn card B face up. 2 is written on it.
- Since different numbers are written, turn both cards face down again, and decrease life by 1 to 1.
- Turn card C face up. 3 is written on it.
- Turn card A face up. 3 is written on it.
- Since the same number is written, remove both cards from the table, and increase the score by 3 to 3.
- Turn card D face up. 1 is written on it.
- Turn card E face up. 2 is written on it.
- Since different numbers are written, turn both cards face down again, and decrease life by 1 to 0.
- Since life has become 0, the game ends. The score is 3.

Note that, immediately after turning card C face up in this sequence, Takahashi can make the choice of "turning face up card A, the other already-known card with 3 written on it, based on the fact that 3 was written on the front of card C."

Sample Input 2

```
5 2
2 3 5 7 101
```

Sample Output 2

```
17.8560846561
```

Sample Input 3

```
20 10
10 20 30 40 50 60 70 80 90 100 110 120 130 140 150 160 170 180 190 200
```

Sample Output 3

```
770.7122293087
```


F - Googol Swaps

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 500 points

Problem Statement

You are given a string S of length N consisting of lowercase English letters.

Find the number, modulo 998244353, of strings that S can become after performing the following operation **exactly** 10^{100} times.

- Choose an integer i between 1 and M , inclusive, and swap the A_i -th and B_i -th characters of S .

Constraints

- N and M are integers.
- $2 \leq N \leq 2 \times 10^5$
- $1 \leq M \leq 2 \times 10^5$
- S is a string of length N consisting of lowercase English letters.
- A_i and B_i are integers.
- $1 \leq A_i < B_i \leq N$
- $(A_1, B_1), \dots, (A_M, B_M)$ are pairwise distinct.

Input

The input is given from Standard Input in the following format:

```
 $N$   $M$   
 $S$   
 $A_1$   $B_1$   
 $\vdots$   
 $A_M$   $B_M$ 
```

Output

Output the answer.

Sample Input 1

```
5 3
miria
1 3
2 5
4 5
```

Sample Output 1

```
6
```

The following six strings are possible as the final S :

- marii
- mirai
- miria
- ramii
- rimai
- rimia

Sample Input 2

```
6 6
yiwayi
1 2
1 3
2 3
4 5
4 6
5 6
```

Sample Output 2

```
18
```

Sample Input 3

```
29 25
hexakosioihexekontahexaphobia
1 2
1 4
1 6
1 8
1 15
1 16
2 3
3 4
4 20
5 6
5 8
8 22
8 23
9 15
9 17
11 21
12 20
13 19
14 29
15 28
16 17
18 19
18 21
19 20
20 21
```

Sample Output 3

```
346192062
```

G - ΣΣX

Time Limit: 3 sec / Memory Limit: 1024 MiB

Score : 600 points

Problem Statement

You are given a length- N sequence of non-negative integers $A = (A_1, \dots, A_N)$.

Find the sum of $\text{mex}(A_l, \dots, A_r)$ over all pairs of integers (l, r) satisfying $1 \leq l \leq r \leq N$.

Here, $\text{mex}(A_l, \dots, A_r)$ denotes the smallest non-negative integer not contained in $A_l, \dots, A_r$.

Constraints

- $1 \leq N \leq 3 \times 10^5$
- $0 \leq A_i \leq N$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N
A_1 A_2 \cdots A_N
```

Output

Output the answer.

Sample Input 1

```
3
1 2 0
```

Sample Output 1

5

The answer is 5, the sum of the following 6 values.

- $\text{mex}(A_1) = \text{mex}(1) = 0$
- $\text{mex}(A_1, A_2) = \text{mex}(1, 2) = 0$
- $\text{mex}(A_1, A_2, A_3) = \text{mex}(1, 2, 0) = 3$
- $\text{mex}(A_2) = \text{mex}(2) = 0$
- $\text{mex}(A_2, A_3) = \text{mex}(2, 0) = 1$
- $\text{mex}(A_3) = \text{mex}(0) = 1$

Sample Input 2

6
2 1 0 2 1 4

Sample Output 2

31

The answer is 31, the sum of the following 21 values.

- $\text{mex}(A_1) = \text{mex}(2) = 0$
- $\text{mex}(A_1, A_2) = \text{mex}(2, 1) = 0$
- $\text{mex}(A_1, A_2, A_3) = \text{mex}(2, 1, 0) = 3$
- $\text{mex}(A_1, A_2, A_3, A_4) = \text{mex}(2, 1, 0, 2) = 3$
- $\text{mex}(A_1, A_2, A_3, A_4, A_5) = \text{mex}(2, 1, 0, 2, 1) = 3$
- $\text{mex}(A_1, A_2, A_3, A_4, A_5, A_6) = \text{mex}(2, 1, 0, 2, 1, 4) = 3$
- $\text{mex}(A_2) = \text{mex}(1) = 0$
- $\text{mex}(A_2, A_3) = \text{mex}(1, 0) = 2$
- $\text{mex}(A_2, A_3, A_4) = \text{mex}(1, 0, 2) = 3$
- $\text{mex}(A_2, A_3, A_4, A_5) = \text{mex}(1, 0, 2, 1) = 3$
- $\text{mex}(A_2, A_3, A_4, A_5, A_6) = \text{mex}(1, 0, 2, 1, 4) = 3$
- $\text{mex}(A_3) = \text{mex}(0) = 1$
- $\text{mex}(A_3, A_4) = \text{mex}(0, 2) = 1$
- $\text{mex}(A_3, A_4, A_5) = \text{mex}(0, 2, 1) = 3$
- $\text{mex}(A_3, A_4, A_5, A_6) = \text{mex}(0, 2, 1, 4) = 3$
- $\text{mex}(A_4) = \text{mex}(2) = 0$
- $\text{mex}(A_4, A_5) = \text{mex}(2, 1) = 0$
- $\text{mex}(A_4, A_5, A_6) = \text{mex}(2, 1, 4) = 0$
- $\text{mex}(A_5) = \text{mex}(1) = 0$
- $\text{mex}(A_5, A_6) = \text{mex}(1, 4) = 0$
- $\text{mex}(A_6) = \text{mex}(4) = 0$

Sample Input 3

25
2 2 1 2 2 0 2 2 2 1 2 2 2 0 2 2 2 2 1 2 2 2 2 0 2

Sample Output 3

624

Hints: AtCoder Beginner Contest 470

Taken from the official editorial, not written by us. Read one hint at a time.

A. Fizz *(editorial, first line)*

Use a for statement to print N lines.

B. Monocolor *(editorial, first line)*

It is optimal to paint them with the most frequent color among the current N balls.

C. Inc, Dec, Xor *(editorial, first line)*

Let $f(A)$ be the total XOR of A .

D. Inverse and Swap *(editorial, first line)*

By identifying a permutation P with a (one-to-one)

mapping $\begin{pmatrix} 1 & 2 & \dots & N \\ P_1 & P_2 & \dots & P_N \end{pmatrix}$, a query of type 2 can be seen as a replacement of ...

E. Concentration *(editorial, first line)*

Let us call an iteration (flip two cards, and gain score or lose life) a turn.

F. Googol Swaps *(editorial, first line)*

Define an undirected graph G with N vertices and M edges, whose vertices represent the positions of characters and each edge (A_i, B_i) represent the choice of i in the operation.

G. ΣΠΧ *(editorial, first line)*

Let S_k be the set of intervals where all integers between 0 and k occur at least once.

Source: <https://atcoder.jp/contests/abc470/editorial>

A - Train Car

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

There is a train consisting of N cars. What is the position, counted from the back, of the car that is K -th from the front of this train?

Constraints

- $1 \leq K \leq N \leq 100$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   $K$ 
```

Output

If the car is X -th from the back, output the integer X in one line.

Sample Input 1

```
5 2
```

Sample Output 1

```
4
```

For a train consisting of five cars, the car that is second from the front is fourth from the back.

Sample Input 2

```
1 1
```

Sample Output 2

1

Sample Input 3

99 50

Sample Output 3

50

B - Isolated Seats

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

You are given a string S of length N consisting of o and x.

N chairs are arranged in a row from left to right. If the i -th character of S is o, a person is sitting in the i -th chair from the left; if it is x, no person is sitting there.

Find the number of chairs that satisfy all of the following conditions.

- No person is sitting in it.
- There is no chair to its left, or no person is sitting in the chair to its left.
- There is no chair to its right, or no person is sitting in the chair to its right.

Constraints

- $1 \leq N \leq 100$
- N is an integer.
- S is a string of length N consisting of o and x.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $S$ 
```

Output

Output the answer in one line.

Sample Input 1

```
8  
xxOxxxOx
```

Sample Output 1

```
2
```

Only the first and fifth chairs from the left satisfy the conditions.

Sample Input 2

```
5  
00000
```

Sample Output 2

```
0
```

Sample Input 3

```
1  
x
```

Sample Output 3

```
1
```

C - Cantrip

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

You are given a string S of length N consisting of o and x.

N bags are arranged in a row, and each bag contains one sweet.

On the i -th bag, "hit" is written if the i -th character of S is o, and "miss" is written if it is x.

For each $k = 1, 2, \dots, N$, solve the following problem.

Takahashi receives the first k bags from the front of the row, eats the sweets inside them, and keeps the bags.

Then, he repeats the following action as many times as possible.

- He discards one bag marked "hit" that he is holding, receives the bag at the front of the row, eats the sweet inside it, and keeps that bag. This action can only be performed when there is still a bag remaining in the row and he is holding a bag marked "hit".

Find the number of sweets he can eat.

Note that when he receives a bag, that bag is removed from the row.

Constraints

- $1 \leq N \leq 8 \times 10^5$
- N is an integer.
- S is a string of length N consisting of o and x.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $S$ 
```

Output

Output a total of N lines.

The l -th line should contain the answer for $k = l$.

Sample Input 1

```
5
OXOXO
```

Sample Output 1

```
2
4
5
5
5
```

For $k = 1$, the bag he receives is marked "hit". He discards it, and the bag he then receives is marked "miss", so he cannot perform any further actions.

Sample Input 2

```
3
000
```

Sample Output 2

```
3
3
3
```

Sample Input 3

```
1
x
```

Sample Output 3

```
1
```

D - The Big Two

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 400 points

Problem Statement

There is a game with N players $1, 2, \dots, N$.

This game is played in a format where two players face each other one-on-one.

A tournament among the N players was held M times, and the two players A_m and B_m advanced to the final of the m -th tournament.

Find the number of pairs of integers x and y satisfying the following conditions.

- $1 \leq x < y \leq N$
- In every tournament, at least one of players x and y advanced to the final.

Constraints

- $2 \leq N \leq 2 \times 10^5$
- $1 \leq M \leq 2 \times 10^5$
- $1 \leq A_i < B_i \leq N$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$    $M$   
 $A_1$   $B_1$   
 $A_2$   $B_2$   
 $\vdots$   
 $A_M$   $B_M$ 
```

Output

Output the answer in one line.

Sample Input 1

```
5 5
1 2
3 4
1 3
2 3
2 5
```

Sample Output 1

```
1
```

The conditions are satisfied when $(x, y) = (2, 3)$, and not satisfied otherwise.

For example, in the fourth tournament, neither player 1 nor player 4 advanced to the final, so $(x, y) = (1, 4)$ does not satisfy the conditions.

Sample Input 2

```
7 8
2 4
1 3
1 7
1 3
1 2
1 6
1 5
1 3
```

Sample Output 2

```
2
```

There are two pairs (x, y) satisfying the conditions: $(1, 2)$ and $(1, 4)$.

Sample Input 3

```
5 8
1 2
2 4
1 3
1 3
1 2
1 2
1 5
1 2
```

Sample Output 3

```
2
```

E - Pro Exam Eligibility

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 475 points

Problem Statement

You are given a string S of length N consisting of o and x.

It is guaranteed that S contains at least K occurrences of o.

Takahashi played a certain game N times.

In the i -th game, he won if the i -th character of S is o, and lost if it is x.

Takahashi chooses a pair of integers l and r satisfying the following conditions.

- $1 \leq l \leq r \leq N$
- He won at least K times in the games from the l -th through the r -th.

Find the maximum possible value of the win rate in the games from the l -th through the r -th.

Constraints

- $1 \leq K \leq N \leq 10^6$
- N and K are integers.
- S is a string of length N consisting of o and x.
- S contains at least K occurrences of o.

Input

The input is given from Standard Input in the following format:

```
 $N$   $K$   
 $S$ 
```

Output

Output the answer in one line. Answers with an absolute or relative error of at most 10^{-6} from the true answer will be accepted.

Sample Input 1

```
10 4
0X00X0XX0X
```

Sample Output 1

```
0.6666666666
```

Choosing $(1, 6)$ as (l, r) gives a win rate of $\frac{2}{3}$.

It is impossible to make the win rate larger than this while satisfying the conditions.

Sample Input 2

```
5 1
XX0XX
```

Sample Output 2

```
1
```

Sample Input 3

```
16 10
XXX0X0000X0X0000
```

Sample Output 3

```
0.769230769230769
```

F - GCD Maximum Spanning Tree

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 500 points

Problem Statement

You are given a sequence of positive integers A of length N .

There is a weighted complete undirected graph with N vertices.

The vertices are numbered $1, 2, \dots, N$.

For i and j satisfying $1 \leq i < j \leq N$, the weight of the edge connecting vertices i and j is the greatest common divisor of A_i and A_j .

Find the maximum possible value of the sum of the weights of the edges included in a spanning tree of this graph.

Constraints

- $2 \leq N \leq 2 \times 10^5$
- $1 \leq A_1 < A_2 < \dots < A_N \leq 10^6$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N
A_1 A_2 ... A_N
```

Output

Output the answer in one line.

Sample Input 1

```
3
4 6 12
```

Sample Output 1

```
10
```

The complete graph has the following three edges.

- The edge connecting vertices 1 and 2 with weight 2
- The edge connecting vertices 1 and 3 with weight 4
- The edge connecting vertices 2 and 3 with weight 6

Choosing the second and third edges gives a sum of weights of 10, which is the maximum possible value.

Sample Input 2

```
5
5 14 15 21 42
```

Sample Output 2

```
43
```

Sample Input 3

```
2
1 1000000
```

Sample Output 3

```
1
```

G - K-nacci Operations

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 575 points

Problem Statement

You are given K strings $S_1, S_2, \dots, S_K$ consisting of a and b. Also, for an integer $i > K$, define the string S_i as the concatenation of $S_{i-1}, S_{i-2}, \dots, S_{i-K}$ in this order.

You perform the following operation on a string T .

- Perform the following operations for $j = 1, 2, \dots, |S_N|$ in order, where $|S_N|$ is the length of S_N .
 - If the j -th character of S_N is a, move the first character of T to the end.
 - If the j -th character of S_N is b, reverse T entirely. That is, reverse the order of the characters of T .

Find the string T after the series of operations.

Constraints

- $2 \leq K \leq 100$
- K is an integer.
- S_i is a non-empty string consisting of a and b. ($1 \leq i \leq K$)
- The sum of the lengths of $S_1, S_2, \dots, S_K$ is at most 2×10^5 .
- $1 \leq N \leq 10^{18}$
- N is an integer.
- T is a string consisting of lowercase English letters with length between 1 and 2×10^5 , inclusive.

Input

The input is given from Standard Input in the following format:

```

K
S_1
S_2
⋮
S_K
N
T

```

Output

Output the answer.

Sample Input 1

```
3
a
aa
b
5
abc
```

Sample Output 1

```
cab
```

$S_1 = a$, $S_2 = aa$, $S_3 = b$, $S_4 = baaa$, and $S_5 = baaabaa$.

Through the operations, the string T changes as $abc \rightarrow cba \rightarrow bac \rightarrow acb \rightarrow cba \rightarrow abc \rightarrow bca \rightarrow cab$, so the string T after the series of operations is cab.

Sample Input 2

```
2
a
ba
6
fibonacci
```

Sample Output 2

```
canobific
```

Sample Input 3

```
5  
aba  
a  
bb  
ba  
aab  
10000000000000000000000000000000  
abcba
```

Sample Output 3

aabcb

Hints: AtCoder Beginner Contest 469

Taken from the official editorial, not written by us. Read one hint at a time.

A. Train Car *(editorial, first line)*

For each car, assign numbers $1, 2, \dots, N$ from the front to back.

B. Isolated Seats *(editorial, first line)*

For each chair, determine if the conditions are satisfied.

C. Cantrip *(editorial, first line)*

Takahashi always has exactly k bags.

D. The Big Two *(editorial, first line)*

At least one of A_1 and B_1 is equal to x or y .

E. Pro Exam Eligibility *(editorial, first line)*

We binary search by the answer.

F. GCD Maximum Spanning Tree *(editorial, first line)*

We cannot enumerate all edges to apply Kruskal's algorithm within the execution time limit, so we need to optimize it.

G. K-nacci Operations *(editorial, first line)*

Let M be the length of T .

Source: <https://atcoder.jp/contests/abc469/editorial>

A - Maximal Value

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

You are given an integer sequence of length N : $A = (A_1, A_2, \dots, A_N)$.

Find the number of integers i between 1 and $N - 2$ (inclusive) satisfying $A_i < A_{i+1} > A_{i+2}$.

Constraints

- $3 \leq N \leq 100$
- $1 \leq A_i \leq 100$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N
A_1 A_2 ... A_N
```

Output

Output the answer.

Sample Input 1

```
6
3 1 4 1 5 2
```

Sample Output 1

```
2
```

Two values $i = 2, 4$ satisfy the condition. Thus, output 2.

Sample Input 2

```
5
1 1 1 2 1
```

Sample Output 2

```
1
```

Sample Input 3

```
10
7 3 9 8 10 3 1 5 5 4
```

Sample Output 3

```
2
```

B - Corridor Watch

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

You are given integers M , D and a string S of length M consisting of G and ..

There are M cells arranged in a row from left to right, numbered 1 through M from the left.

Some of the cells have a guardman standing on them. Specifically, a guardman stands on cell i if $S_i = \text{G}$, and no one stands on cell i if $S_i = \text{.}$.

A cell whose distance from a cell with a guardman is at most D is watched by that guardman. That is, a cell x is watched by a guardman if there exists a cell i such that $S_i = \text{G}$ and $|x - i| \leq D$.

Among the M cells, find the number of cells that are **not** watched.

Constraints

- $0 \leq D < M \leq 100$
- D and M are integers.
- S_i is a string of length M consisting of G and ..

Input

The input is given from Standard Input in the following format:

```
 $M$   $D$   
 $S$ 
```

Output

Output the answer.

Sample Input 1

```
7 1  
.G...GG
```

Sample Output 1

1

Only cell 4 is not watched.

Sample Input 2

6 5
.....

Sample Output 2

6

All cells are not watched.

Sample Input 3

21 2
....G...GG.....G.....

Sample Output 3

6

C - Between P and Q

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

You are given an integer N and integer sequences $P = (P_1, P_2, \dots, P_N)$ and $Q = (Q_1, Q_2, \dots, Q_N)$, each of which is a permutation of $(1, 2, \dots, N)$.

Find the number of integer sequences that are a permutation of $(1, 2, \dots, N)$ and are lexicographically greater than P and lexicographically less than Q .

► What is lexicographic order for integer sequences?

Constraints

- $1 \leq N \leq 10$
- P and Q are integer sequences that are permutations of $(1, 2, \dots, N)$.
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N
P1 P2 ... PN
Q1 Q2 ... QN
```

Output

Output the answer.

Sample Input 1

```
3
1 3 2
3 1 2
```

Sample Output 1

```
2
```

Two sequences $(2, 1, 3)$, $(2, 3, 1)$ satisfy the condition. Thus, output 2.

Sample Input 2

```
5
5 4 2 1 3
5 1 2 3 4
```

Sample Output 2

```
0
```

There is no permutation of $(1, 2, 3, 4, 5)$ satisfying the condition.

Sample Input 3

```
7
3 6 5 2 7 1 4
4 1 5 7 2 3 6
```

Sample Output 3

```
223
```

D - Pre-Palindrome

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 400 points

Problem Statement

A string consisting of lowercase English letters is called a **good string** if it satisfies the following condition.

- It can be turned into a palindrome by rewriting at most one character.

For example, a, iwai, and abcdcza are good strings, but abcd and atcoder are not good strings. Note that, in particular, a palindrome is also a good string.

You are given a string S consisting of lowercase English letters.

Find the number of non-empty substrings (contiguous subsequences) of S that are good strings.

Two substrings taken from different positions of S are counted separately even if they are equal as strings.

► What is a substring?

Constraints

- S is a string of length between 1 and 10^4 , inclusive, consisting of lowercase English letters.

Input

The input is given from Standard Input in the following format:

S

Output

Output the answer.

Sample Input 1

ababa

Sample Output 1

13

The substring `ba` from the 2nd through 3rd characters of S and the substring `ababa` from the 1st through 5th characters are some good strings.

Among the 15 substrings, the good strings are the 13 substrings other than `abab` from the 1st through 4th characters and `baba` from the 2nd through 5th characters.

Sample Input 2

atcoder

Sample Output 2

18

Sample Input 3

abccbacbacb

Sample Output 3

40

E - Sum of Average

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 450 points

Problem Statement

You are given a positive integer N and a length- N integer sequence $A = (A_1, A_2, \dots, A_N)$.

Define $f(l, r)$ as the (arithmetic) mean of $A_l, A_{l+1}, \dots, A_r$.

Find $\sum_{1 \leq l \leq r \leq N} f(l, r)$, modulo 998244353.

► Definition of a rational number modulo 998244353

Constraints

- $1 \leq N \leq 5 \times 10^5$
- $0 \leq A_i < 998244353$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $A_1$   $A_2$   $\dots$   $A_N$ 
```

Output

Output the answer.

Sample Input 1

```
2  
2 3
```

Sample Output 1

499122184

We have $f(1, 1) = 2, f(1, 2) = \frac{2 + 3}{2} = \frac{5}{2}, f(2, 2) = 3$. Thus, $\sum_{1 \leq l \leq r \leq N} f(l, r) = 2 + \frac{5}{2} + 3 = \frac{15}{2}$.
 $\frac{15}{2}$ in modulo-998244353 expression is 499122184, so output 499122184.

Sample Input 2

6
1 2 3 4 5 6

Sample Output 2

499122250

Sample Input 3

9
3 1 4 1 5 9 2 6 5

Sample Output 3

855638200

F - Chmax

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 500 points

Problem Statement

You are given a positive integer N and a permutation $P = (P_1, P_2, \dots, P_N)$ of $(1, 2, \dots, N)$.

There are variables x, y, c . Initially, $x = y = c = 0$.

For $k = 1, 2, \dots, N$ in this order, you perform one of the following operations:

- Operation 1: Increase c by 1 if $x < P_k$. Then, replace x with $\max(x, P_k)$.
- Operation 2: Increase c by 1 if $y < P_k$. Then, replace y with $\max(y, P_k)$.

Find the maximum possible final value of c .

Constraints

- $1 \leq N \leq 5 \times 10^5$
- P is a permutation of $(1, 2, \dots, N)$.
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N
P_1 P_2 ... P_N
```

Output

Output the answer.

Sample Input 1

```
5
4 3 1 2 5
```

Sample Output 1

4

By performing the operations as follows, you can achieve $c = 4$:

- For $k = 1$: Perform operation 1. Then, $(x, y, c) = (4, 0, 1)$.
- For $k = 2$: Perform operation 1. Then, $(x, y, c) = (4, 0, 1)$.
- For $k = 3$: Perform operation 2. Then, $(x, y, c) = (4, 1, 2)$.
- For $k = 4$: Perform operation 2. Then, $(x, y, c) = (4, 2, 3)$.
- For $k = 5$: Perform operation 2. Then, $(x, y, c) = (4, 5, 4)$.

c cannot be made greater than 4 no matter how you perform the operations, so output 4.

Sample Input 2

6
6 5 4 3 2 1

Sample Output 2

2

Sample Input 3

9
3 6 5 2 7 8 9 1 4

Sample Output 3

7

G - Restricted Permutation

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 550 points

Problem Statement

You are given an integer N and a string S of length N consisting of o and x.

Find the number, modulo 998244353, of permutations $P = (P_1, P_2, \dots, P_N)$ of $(1, 2, \dots, N)$ satisfying the following condition.

- For $k = 1, 2, \dots, N$, the following two are equivalent.
 - $S_k = \text{o}$
 - P contains a permutation of $(1, 2, \dots, k)$ as a contiguous subsequence.

Constraints

- $1 \leq N \leq 2000$
- S_i is a string of length N consisting of o and x.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $S$ 
```

Output

Output the answer.

Sample Input 1

```
3  
oxo
```

Sample Output 1

2

$P = (1, 3, 2), (2, 3, 1)$ satisfy the condition.

Sample Input 2

7
XXXXXXX

Sample Output 2

0

There is no P satisfying the condition.

Sample Input 3

15
OXXOXOXXXXOXXO

Sample Output 3

1627648

Hints: AtCoder Beginner Contest 468

Taken from the official editorial, not written by us. Read one hint at a time.

A. Maximal Value *(editorial, first line)*

Count the number of indices i satisfying the condition.

B. Corridor Watch *(editorial, first line)*

A cell is not watched if and only if there is no guardman on a cell distant by at most D .

C. Between P and Q *(editorial, first line)*

There are $N!$ permutations of $(1, 2, \dots, N)$.

D. Pre-Palindrome *(editorial, first line)*

First, we will count the number of odd-length good strings.

E. Sum of Average *(editorial, first line)*

For an integer n , let $H_n = \sum_{k=1}^n \frac{1}{k}$.

F. Chmax *(editorial, first line)*

First of all, consider the DP (Dynamic Programming) where $d[i][p][q]$ is defined as the maximum value of c when the operations up to $k = i$ have finished, where $x = p$ and $y = q$.

G. Restricted Permutation *(editorial, first line)*

First of all, the permutation P always contain permutations of (1) and $(1, 2, \dots, N)$ as a subarray, so the answer is 0 if $S_1 = x$ or $S_N = x$.

Source: <https://atcoder.jp/contests/abc468/editorial>

A - Obesity

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

The value calculated by the following formula is called BMI[kg/m^2]. ($\div$ denotes division.)

- $\text{Weight}[\text{kg}] \div \text{Height}[\text{m}] \div \text{Height}[\text{m}]$

In Japan, a person whose BMI is $25 \text{ kg}/\text{m}^2$ or more is considered obese.

Determine whether a person with height $H[\text{cm}]$ and weight $W[\text{kg}]$ is considered obese in Japan.

Constraints

- $1 \leq H \leq 300$
- $1 \leq W \leq 300$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

H W

Output

Output Yes on one line if a person with height $H[\text{cm}]$ and weight $W[\text{kg}]$ is considered obese in Japan, and No otherwise.

Sample Input 1

180 60

Sample Output 1

No

Since the height is $180\text{ cm} = 1.8\text{ m}$, the BMI is $60\text{ kg} \div 1.8\text{ m} \div 1.8\text{ m} = 18.5\dots\text{ kg/m}^2$, so this person is not considered obese.

Sample Input 2

182 188

Sample Output 2

Yes

Sample Input 3

180 81

Sample Output 3

Yes

B - Keep the Change

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

Takahashi made purchases at N stores. Initially, he had 10000 yen.

At the i -th store, he bought an item worth A_i yen and paid B_i yen. Here, $A_i \leq B_i$ holds. If $S_i = \text{keep}$, he did not receive the change, and if $S_i = \text{take}$, he received the change.

Find the amount of money he lost compared to the case where he received the change at every store. To be precise,

- let X yen be Takahashi's final amount of money, and
- let Y yen be Takahashi's final amount of money in the case where he received the change at every store.

Find $Y - X$.

Constraints

- $1 \leq N \leq 100$
- $1 \leq A_i \leq B_i \leq 100$
- S_i is keep or take.
- N, A_i, B_i are all integers.

Input

The input is given from Standard Input in the following format:

```
N
A_1 B_1 S_1
A_2 B_2 S_2
⋮
A_N B_N S_N
```

Output

Output the amount of money Takahashi lost compared to the case where he received the change at every store.

Sample Input 1

```
3
1 2 keep
3 6 take
5 9 keep
```

Sample Output 1

```
5
```

Takahashi's actions are as follows.

- At the first store, he bought an item worth **1** yen, paid **2** yen, and did not receive the change.
- At the second store, he bought an item worth **3** yen, paid **6** yen, and received the change.
- At the third store, he bought an item worth **5** yen, paid **9** yen, and did not receive the change.

Sample Input 2

```
8
36 49 take
38 73 keep
27 85 take
65 71 take
52 86 keep
48 60 keep
37 98 keep
5 38 keep
```

Sample Output 2

```
175
```

C - Adjacent Sums (easy)

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

The problem statement of Problem C is the same as Problem E. Only the constraint shown in red differ.

You are given integer sequences $A = (A_1, A_2, \dots, A_N)$, $B = (B_1, B_2, \dots, B_{N-1})$ consisting of integers between 0 and $M - 1$, inclusive. The lengths of A and B are N and $N - 1$, respectively.

You can perform the following operation on A any number of times.

- Choose an integer i with $1 \leq i \leq N$, and add 1 to A_i .

Find the minimum number of operations required to satisfy the following condition.

It can be proved that the condition can always be satisfied under the constraints of this problem.

- For $i = 1, 2, \dots, N - 1$, the remainder when $A_i + A_{i+1}$ is divided by M equals B_i .

Constraints

- $2 \leq N \leq 2 \times 10^5$
- $M = 2$
- $0 \leq A_i \leq M - 1$
- $0 \leq B_i \leq M - 1$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N M
A_1 A_2 ... A_N
B_1 B_2 ... B_{N-1}
```

Output

Output the answer on one line.

Sample Input 1

```
3 2
1 1 1
1 1
```

Sample Output 1

```
1
```

If we choose $i = 2$ for the first operation, we get $A = (1, 2, 1)$.

Since $A_1 + A_2 = 1 + 2 = 3$ and $A_2 + A_3 = 2 + 1 = 3$, the condition is satisfied.

$A = (1, 1, 1)$ does not satisfy the condition, so the answer is 1.

Sample Input 2

```
2 2
1 1
0
```

Sample Output 2

```
0
```

Sample Input 3

```
10 2
0 0 0 1 1 0 1 0 1 0
0 1 0 1 0 1 0 1 0
```

Sample Output 3

```
4
```

D - Concentric Circles

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 425 points

Problem Statement

Do there exist two circles C_1 and C_2 on the xy -plane satisfying all of the following conditions? Here, C_1 and C_2 may coincide.

- The two distinct points (P_x, P_y) and (Q_x, Q_y) lie on the circumference of C_1 .
- The two distinct points (R_x, R_y) and (S_x, S_y) lie on the circumference of C_2 .
- C_1 and C_2 have the same center.

You are given T test cases; solve each of them.

Constraints

- $1 \leq T \leq 5 \times 10^4$
- $-10^9 \leq P_x, P_y, Q_x, Q_y, R_x, R_y, S_x, S_y \leq 10^9$
- $(P_x, P_y) \neq (Q_x, Q_y)$
- $(R_x, R_y) \neq (S_x, S_y)$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
T
case1
case2
⋮
caseT
```

Each test case case _{t} is given in the following format:

```
Px Py Qx Qy Rx Ry Sx Sy
```

Output

Output T lines. The t -th line should contain the answer to the t -th test case.

For each test case, output Yes if there exist two circles C_1 and C_2 satisfying all of the conditions, and No otherwise.

Sample Input 1

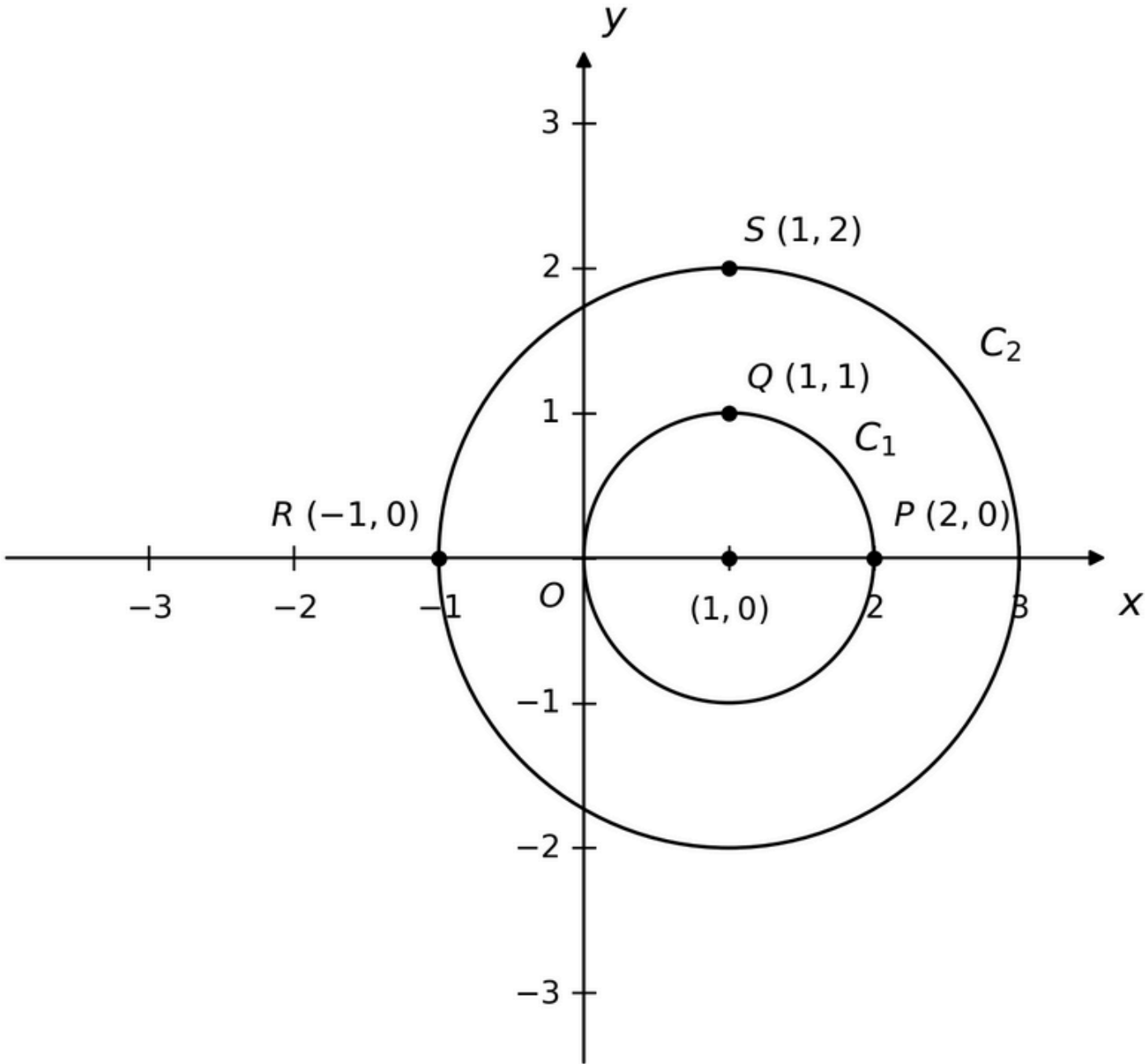
```
3
2 0 1 1 -1 0 1 2
1 0 -1 0 0 1 0 -1
4 0 3 1 2 0 1 1
```

Sample Output 1

Yes
Yes
No

Consider the first test case. As shown in the figure below, if we let C_1 be the circle with center $(1, 0)$ and radius 1, and C_2 be the circle with center $(1, 0)$ and radius 2, the conditions are satisfied.

For the second test case, letting both C_1 and C_2 be the circle with center $(0, 0)$ and radius 1 satisfies the conditions.



E - Adjacent Sums (hard)

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 450 points

Problem Statement

The problem statement of Problem E is the same as Problem C. Only the constraint shown in red differ.

You are given integer sequences $A = (A_1, A_2, \dots, A_N)$ and $B = (B_1, B_2, \dots, B_{N-1})$ consisting of integers between 0 and $M - 1$, inclusive. The lengths of A and B are N and $N - 1$, respectively.

You can perform the following operation on A any number of times.

- Choose an integer i with $1 \leq i \leq N$, and add 1 to A_i .

Find the minimum number of operations required to satisfy the following condition.

It can be proved that the condition can always be satisfied under the constraints of this problem.

- For $i = 1, 2, \dots, N - 1$, the remainder when $A_i + A_{i+1}$ is divided by M equals B_i .

Constraints

- $2 \leq N \leq 2 \times 10^5$
- $3 \leq M \leq 10^9$
- $0 \leq A_i \leq M - 1$
- $0 \leq B_i \leq M - 1$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N M
A_1 A_2 ... A_N
B_1 B_2 ... B_{N-1}
```

Output

Output the answer on one line.

Sample Input 1

```
3 10
4 6 7
5 5
```

Sample Output 1

```
5
```

If we choose $i = 2$ for the first operation, we get $A = (4, 7, 7)$.

If we choose $i = 1$ for the second operation, we get $A = (5, 7, 7)$.

If we choose $i = 1$ for the third operation, we get $A = (6, 7, 7)$.

If we choose $i = 2$ for the fourth operation, we get $A = (6, 8, 7)$.

If we choose $i = 1$ for the fifth operation, we get $A = (7, 8, 7)$.

Since $A_1 + A_2 = 7 + 8 = 15$ and $A_2 + A_3 = 8 + 7 = 15$, the condition is satisfied.

The condition cannot be satisfied with four or fewer operations, so the answer is 5.

Sample Input 2

```
2 3
1 2
2
```

Sample Output 2

```
2
```

Sample Input 3

```
10 10
0 1 2 3 4 5 6 7 8 9
9 8 7 6 5 4 3 2 1
```

Sample Output 3

```
40
```

F - Email Scheduling Optimization

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 525 points

Problem Statement

You are given length- N sequences of positive integers $A = (A_1, A_2, \dots, A_N)$ and $B = (B_1, B_2, \dots, B_N)$.

You are given Q queries.

Each query is of one of the following two types.

- 1 i x : Change A_i to x .
- 2 i x : Change B_i to x .

After processing each query, solve the following problem.

Takahashi needs to send an email to each of N companies and receive a reply from each of them. Writing the email to send to the j -th company takes A_j minutes, and the reply arrives B_j minutes after it is sent.

He starts writing emails at time 0.

He can write the N emails in any order he likes, but he cannot write two or more emails simultaneously.

Find the minimum possible time at which he finishes receiving all of the replies.

Assume that the time taken to send emails is negligible.

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq Q \leq 10^5$
- $1 \leq A_j, B_j \leq 10^9$
- In each query, $1 \leq i \leq N$.
- In each query, $1 \leq x \leq 10^9$.
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```

 $N$   $Q$ 
 $A_1$   $A_2$   $\dots$   $A_N$ 
 $B_1$   $B_2$   $\dots$   $B_N$ 
query1
query2
⋮
query $Q$ 

```

For each query query _{q} , the type of the query (1 or 2), i , and x are given in this order, separated by spaces. That is, each query is given in one of the following two formats:

```
1  $i$   $x$ 
```

```
2  $i$   $x$ 
```

Output

Output the answers in a total of Q lines. The q -th line should contain the answer to the problem after processing the q -th query.

Sample Input 1

```

3 3
4 6 7
4 6 7
1 2 1
2 3 7
2 3 1

```

Sample Output 1

```
16
16
13
```

After processing the first query, $A = (4, 1, 7)$ and $B = (4, 6, 7)$.

If Takahashi starts writing the email to send to company 3 at time 0, finishes writing it at time 7, and sends it to company 3, he receives the reply at time 14.

If he starts writing the email to send to company 2 at time 7, finishes writing it at time 8, and sends it to company 2, he receives the reply at time 14.

If he starts writing the email to send to company 1 at time 8, finishes writing it at time 12, and sends it to company 1, he receives the reply at time 16.

G - Many Sweets Problem

Time Limit: 3 sec / Memory Limit: 1024 MiB

Score : 625 points

Problem Statement

There is a length- N sequence of positive integers $A = (A_1, A_2, \dots, A_N)$. Process the following query Q times.

- `c x l r k`: Update the value of A_c to x . Then, solve the following sub-problem and output the answer.

There are $r - l + 1$ sweets. The deliciousness of the i -th sweet is A_{l+i-1} .

You decide to eat sweets until the total deliciousness of the sweets eaten becomes k or more.

What is the minimum possible number of sweets you eat, if you optimally choose which sweets to eat? Output the answer.

If the total deliciousness cannot become k or more no matter how you choose the sweets to eat, output -1 instead.

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq Q \leq 10^5$
- $1 \leq A_i \leq 10^9$
- $1 \leq c \leq N$
- $1 \leq x \leq 10^9$
- $1 \leq l \leq r \leq N$
- $1 \leq k \leq 10^{15}$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```

 $N$   $Q$ 
 $A_1$   $A_2$   $\dots$   $A_N$ 
query1
query2
⋮
query $Q$ 

```

Each query query _{q} is given in the following format:

```

 $c$   $x$   $l$   $r$   $k$ 

```

Output

Output Q lines. The q -th line should contain the answer to the q -th query.

Sample Input 1

```

7 5
8 2 4 1 7 3 6
1 1 4 7 9
5 2 1 3 8
6 4 1 5 9
6 5 3 5 1
7 9 4 6 4

```

Sample Output 1

```

2
-1
4
1
1

```

We explain the first query.

First, update A_1 to 1. Then, $A = (1, 2, 4, 1, 7, 3, 6)$. Now, solve the sub-problem.

In the sub-problem, there are four sweets, with deliciousness 1, 7, 3, 6 in order.

To minimize the number of sweets eaten so that the total deliciousness becomes $k = 9$ or more, it is optimal to eat the second and fourth sweets. Thus, the answer to the sub-problem is 2.

Sample Input 2

```
15 10
993115119 576136368 21553212 219853538 853822501 687675302 281611653 844033520 423210108 33
9630584 780395612 207907746 285523486 359061085 14767613
6 13801767 1 3 667406485
7 672229269 5 7 855399219
11 2367096 1 10 4016308479
6 5951398 8 8 413598120
6 196639646 5 11 2483790193
14 105322777 6 7 610670157
7 416730828 2 14 1236755516
9 838827476 5 14 4350335977
1 894919681 3 7 1132967029
6 932707551 6 15 1694677398
```

Sample Output 2

```
1
2
6
1
4
1
2
-1
2
2
```

Hints: AtCoder Beginner Contest 467

Taken from the official editorial, not written by us. Read one hint at a time.

A. Obesity *(editorial, first line)*

$H[\text{cm}]$ can be converted to meters by $\frac{H}{100}[\text{m}]$.

B. Keep the Change *(editorial, first line)*

For each store, compute how much yen he lost compared to the case receiving changes.

C. Adjacent Sums (easy) *(editorial, first line)*

Let $(A'_1, A'_2, \dots, A'_N)$ be the resulting A .

D. Concentric Circles *(editorial, first line)*

The key approach to this problem is not the circle itself, but where the center of the circle can exist.

Source: <https://atcoder.jp/contests/abc467/editorial>

E. Adjacent Sums (hard) *(editorial, first line)*

Let $(A'_1, A'_2, \dots, A'_N)$ be the resulting A .

F. Email Scheduling Optimization *(editorial, first line)*

It is optimal to write emails in descending order of B .

G. Many Sweets Problem *(editorial, first line)*

This problem can be solved with a data structure called merge-sort tree or wavelet matrix.

A - Compromise

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

There are N choices.

You will choose one of these choices. If you choose the i -th choice, your happiness will be X_i .

If your happiness will be negative no matter which choice you choose, output Yes; otherwise, output No.

Constraints

- $2 \leq N \leq 10$
- $-100 \leq X_i \leq 100$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $X_1$   $X_2$  ...  $X_N$ 
```

Output

Output Yes or No according to the instructions in the problem statement.

Sample Input 1

```
4  
2 0 -1 2
```

Sample Output 1

```
No
```

If you choose choice 1, 2, or 4, the happiness will not be negative.

Sample Input 2

```
3
-5 -2 -1
```

Sample Output 2

```
Yes
```

Sample Input 3

```
4
0 -2 0 -1
```

Sample Output 3

```
No
```

B - Representative Balls

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

There are N balls.

The color of the i -th ball is C_i , and its size is S_i . Here, the colors are represented by integers $1, 2, \dots, M$.

For $k = 1, 2, \dots, M$, output the maximum size of a ball of color k , or -1 if there is no ball of color k .

Constraints

- $1 \leq N, M \leq 100$
- $1 \leq C_i \leq M$
- $1 \leq S_i \leq 100$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   $M$   
 $C_1$   $S_1$   
 $C_2$   $S_2$   
 $\vdots$   
 $C_N$   $S_N$ 
```

Output

In the order $k = 1, 2, \dots, M$, output the maximum size of a ball of color k if such a ball exists, or -1 otherwise, separated by spaces.

Sample Input 1

```
4 5
1 3
2 10
1 7
4 9
```

Sample Output 1

```
7 10 -1 9 -1
```

The maximum size of a ball of color 1 is 7, that of a ball of color 2 is 10, and that of a ball of color 4 is 9.

There is no ball of color 3 or color 5.

Sample Input 2

```
5 5
2 6
5 12
5 2
5 9
2 7
```

Sample Output 2

```
-1 7 -1 -1 12
```

C - Count Close Pairs

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

This is an **interactive problem** (in which your program and the judge program communicate via Standard Input and Output).

On a number line, points $1, 2, \dots, N$ are arranged from left to right **in that order**.

Initially, you are given only the integer N .

Then, you can ask the judge the following question at most $2N$ times.

- Choose integers i and j satisfying $1 \leq i < j \leq N$, and ask whether the distance between points i and j is at most 1.

Output the number of pairs of points whose distance is at most 1, that is, the number of pairs of integers (i, j) satisfying $1 \leq i < j \leq N$ such that the distance between points i and j is at most 1.

Constraints

- $2 \leq N \leq 10^3$
- N is an integer.

Interaction

First, receive the integer N representing the number of points from Standard Input:

N

Then, you can repeat the question described in the problem statement to the judge at most $2N$ times.

Output the question to Standard Output in the following format, where i and j must be integers satisfying $1 \leq i < j \leq N$:

? i j

As a response to this, one of the following will be given from Standard Input:

Yes

No

Here, Yes indicates that the distance between points i and j is at most 1, and No indicates that the distance between points i and j is greater than 1.

Once you have found the answer X to the problem, output your answer in the following format, and then terminate the program immediately:

! X

Notes

- Each time you output something, insert a newline at the end and flush Standard Output. Otherwise, the judge result may be **TLE**.
- The judge result is indeterminate if you output something invalid during the interaction, or if the program terminates prematurely.
- Terminate the program immediately after outputting your answer. Otherwise, the judge result is indeterminate.
- The arrangement of the points and the answer are fixed at the start of the interaction between you and the judge, and do not change depending on, for example, the questions you make.

Sample Interaction

Below is an example of interaction where $N = 3$ and the coordinates of points 1, 2, 3 are 0, 0.7, 1.5, respectively. (The coordinates of points 1, 2, 3 are not given as input.)

Input	Output	Explanation
3		N is given.
	? 1 2	Ask the judge whether the distance between points 1 and 2 is at most 1.
Yes		Obtain the response from the judge that the distance between points 1 and 2 is at most 1.
	? 1 3	Ask the judge whether the distance between points 1 and 3 is at most 1.
No		Obtain the response from the judge that the distance between points 1 and 3 is not at most 1.
	? 2 3	Ask the judge whether the distance between points 2 and 3 is at most 1.
Yes		Obtain the response from the judge that the distance between points 2 and 3 is at most 1.
	! 2	Answer 2 as the number of pairs satisfying the condition.

The number of questions is 3, which is in particular at most $2N = 6$.
Since the answered number of pairs is also correct, the program will be judged as correct if it terminates immediately after this.

D - Placing Rooks

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 400 points

Problem Statement

There is a grid with N rows and N columns.

Initially, nothing is placed on the grid.

Starting from this state, Takahashi performs M operations on the grid in order. The i -th operation ($1 \leq i \leq M$) is as follows.

- Remove all pieces placed on the cells in the R_i -th row from the top.
- Next, remove all pieces placed on the cells in the C_i -th column from the left.
- Finally, place a piece on the cell at the R_i -th row from the top and the C_i -th column from the left.

Output the number of pieces placed on the grid after the M operations.

Constraints

- $1 \leq N \leq 3 \times 10^5$
- $1 \leq M \leq 3 \times 10^5$
- $1 \leq R_i \leq N$
- $1 \leq C_i \leq N$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $N$   $M$   
 $R_1$   $C_1$   
 $R_2$   $C_2$   
 $\vdots$   
 $R_M$   $C_M$ 
```

Output

Output the number of pieces placed on the grid after the M operations.

Sample Input 1

```
3 6
1 1
1 2
3 3
3 2
1 3
1 3
```

Sample Output 1

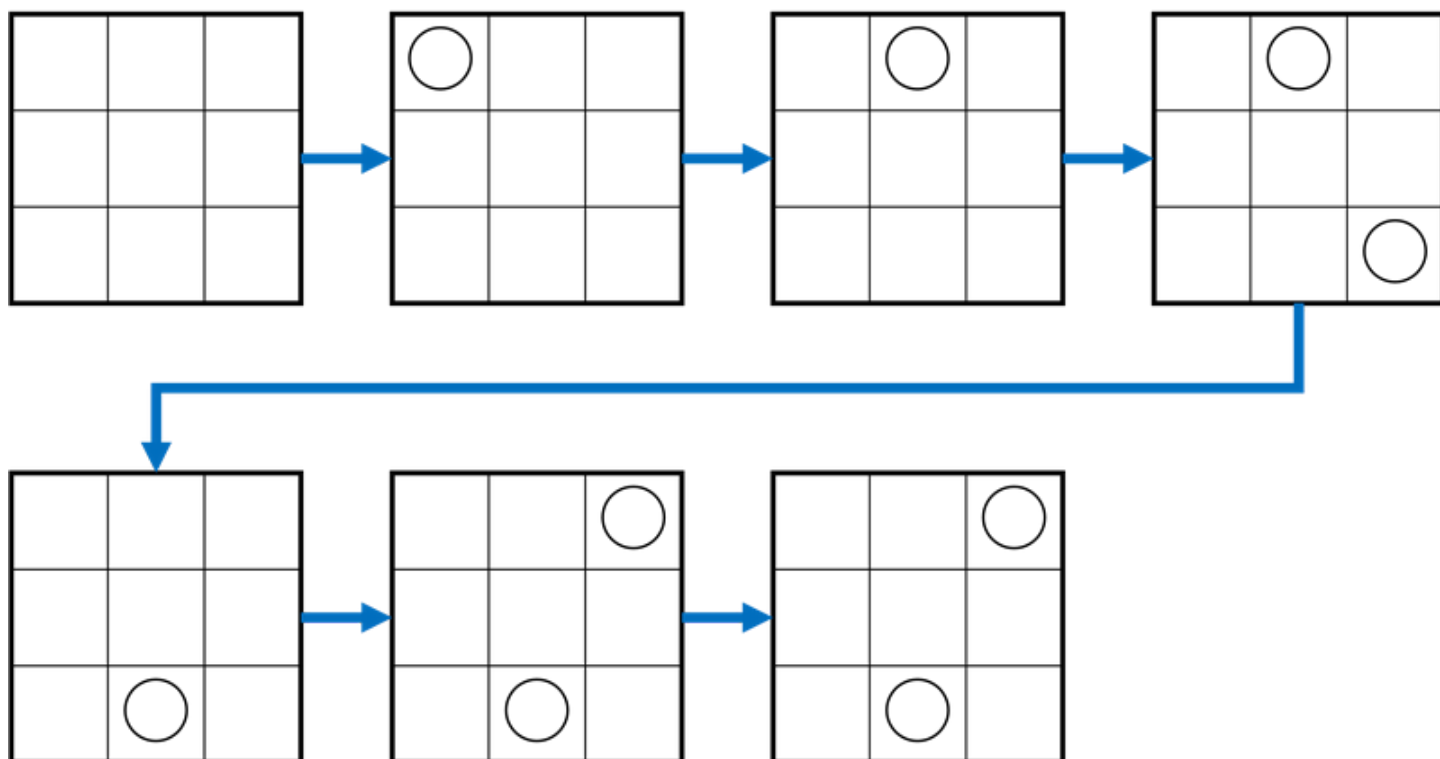
```
2
```

Initially, nothing is placed on the grid with three rows and three columns, and pieces are removed and placed by each operation as follows.

Below, the cell at the i -th row from the top and the j -th column from the left is denoted as cell (i, j) .

- In the first operation, a piece is placed on cell $(1, 1)$.
- In the second operation, the piece is removed from cell $(1, 1)$, and a piece is placed on cell $(1, 2)$.
- In the third operation, a piece is placed on cell $(3, 3)$.
- In the fourth operation, the pieces are removed from cell $(1, 2)$ and cell $(3, 3)$, and a piece is placed on cell $(3, 2)$.
- In the fifth operation, a piece is placed on cell $(1, 3)$.
- In the sixth operation, the piece is removed from cell $(1, 3)$, and a piece is placed on cell $(1, 3)$ again.

In the final state, there is one piece each on cell $(1, 3)$ and cell $(3, 2)$, so output 2.



Sample Input 2

```
2 3  
1 2  
2 1  
1 1
```

Sample Output 2

```
1
```

E - Range Flip

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 450 points

Problem Statement

There are N cards lined up. The cards are numbered $1, 2, \dots, N$.

On the front side of card i , an integer A_i is written, and on the back side, an integer B_i is written. Initially, every card is facing front side up.

You can perform the following operation at most K times.

- Choose integers l and r satisfying $1 \leq l \leq r \leq N$. For each integer i satisfying $l \leq i \leq r$, flip card i . Here, flipping a card means turning the side that was facing down before the operation to face up.

After finishing the operations, find the maximum possible value of the sum of the numbers written on the side of the cards that is facing up.

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq K \leq 10$
- $1 \leq A_i, B_i \leq 10^9$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N K
A_1 B_1
A_2 B_2
⋮
A_N B_N
```

Output

Output the answer.

Sample Input 1

```
7 2
2 1
6 9
3 5
9 2
4 8
7 4
5 6
```

Sample Output 1

```
45
```

If you choose $l = 2, r = 5$ in the first operation and $l = 4, r = 4$ in the second operation, the numbers written on the side facing up, in order of the card numbers, become 2, 9, 5, 9, 8, 7, 5, and their sum is 45.

Sample Input 2

```
5 6
9 6
3 2
8 1
7 5
8 4
```

Sample Output 2

```
35
```

It is fine if you perform no operation.

Sample Input 3

```
9 1
2 7
9 4
1 1
6 1
3 4
8 9
1 2
7 5
3 9
```

Sample Output 3

47

F - Many Mod Calculation

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 500 points

Problem Statement

You are given integers N , X and a length- N sequence of positive integers $A = (A_1, A_2, \dots, A_N)$.

For a non-negative integer x , define $f(x) = (\dots((x \bmod A_1) \bmod A_2) \dots) \bmod A_N$.

Find the number of integers x between 1 and X , inclusive, such that $f(x) = 0$.

You are given T test cases; solve each of them.

Constraints

- $1 \leq T \leq 2 \times 10^5$
- $1 \leq N \leq 2 \times 10^5$
- The sum of N over all test cases is at most 2×10^5 .
- $1 \leq X \leq 10^{18}$
- $1 \leq A_i \leq 10^{18}$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
T
case1
case2
⋮
caseT
```

Each test case is given in the following format:

```
N X
A1 A2 ... AN
```

Output

Output the answers for the test cases in order, separated by newlines.

Sample Input 1

```
4
3 7
5 2 3
9 31415
9 9 8 2 4 4 3 5 3
1 10000000000000000000
1
9 20260405
3141 5926 5358 9793 2384 6264 3383 2795 288
```

Sample Output 1

```
4
17452
10000000000000000000
77403
```

Consider the first test case.

For example, when $x = 7$, $f(7) = (((7 \bmod 5) \bmod 2) \bmod 3) = (2 \bmod 2) \bmod 3 = 0 \bmod 3 = 0$.

There are four integers x between 1 and 7 such that $f(x) = 0$: $x = 2, 4, 5, 7$.

G - Segment Sum Constraints

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 600 points

Problem Statement

You are given M triples of integers (L_i, R_i, S_i) .

Consider tuples $A = (A_1, A_2, \dots, A_N)$ of N **positive** integers satisfying all of the following conditions.

- The sum of $A_{L_i}, A_{L_i+1}, \dots, A_{R_i}$ is S_i .

If there are infinitely many tuples satisfying the conditions, output `Infinity`; otherwise, output the number of such tuples, modulo 998244353.

Constraints

- $1 \leq N \leq 8$
- $1 \leq M \leq 36$
- $1 \leq L_i \leq R_i \leq N$
- $1 \leq S_i \leq 10^9$
- All (L_i, R_i) are distinct.
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N M
L1 R1 S1
L2 R2 S2
⋮
LM RM SM
```

Output

If there are infinitely many tuples satisfying the conditions, output `Infinity`; otherwise, output the number of such tuples, modulo 998244353.

Sample Input 1

```
3 2
1 2 7
2 3 10
```

Sample Output 1

```
6
```

We have the following two conditions.

- $A_1 + A_2 = 7$
- $A_2 + A_3 = 10$

The six tuples $A = (1, 6, 4), (2, 5, 5), (3, 4, 6), (4, 3, 7), (5, 2, 8), (6, 1, 9)$ satisfy the conditions, so output 6 modulo 998244353, that is, output 6.

Sample Input 2

```
2 1
1 1 10
```

Sample Output 2

```
Infinity
```

There are infinitely many tuples A satisfying the conditions.

Sample Input 3

```
2 2
1 1 10
1 2 1
```

Sample Output 3

```
0
```

There is no tuple A satisfying the conditions.

Hints: AtCoder Beginner Contest 466

Taken from the official editorial, not written by us. Read one hint at a time.

A. Compromise *(editorial, first line)*

Inspect the element one by one, and check if there is i with $X_i \geq 0$.

B. Representative Balls *(editorial, first line)*

For each color, prepare a variable that manages the maximum size.

C. Count Close Pairs *(editorial, first line)*

This problem can be solved with the sliding window technique.

D. Placing Rooks *(editorial, first line)*

We introduce two solutions.

E. Range Flip *(editorial, first line)*

We will call cards $l, l + 1, \dots, r - 1$ simply interval $[l, r)$.

F. Many Mod Calculation *(editorial, first line)*

Let us consider what happens when $0, 1, \dots, X - 1$ are replaced by the remainders when they were divided by M .

G. Segment Sum Constraints *(editorial, first line)*

First, replace S_i in each condition with $S_i - (R_i - L_i + 1)$ in order to mandate that A is a non-negative integer sequence.

Source: <https://atcoder.jp/contests/abc466/editorial>

A - Supermajority

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

You are given positive integers A and B .

If $A > B \times \frac{2}{3}$, output Yes; otherwise, output No.

Constraints

- $1 \leq A, B \leq 1000$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
A B
```

Output

Output the answer.

Sample Input 1

```
316 465
```

Sample Output 1

```
Yes
```

Since $316 > 465 \times \frac{2}{3}$, output Yes.

Sample Input 2

101 248

Sample Output 2

No

Since $101 > 248 \times \frac{2}{3}$ does not hold, output No.

Sample Input 3

666 999

Sample Output 3

No

Since $666 > 999 \times \frac{2}{3}$ does not hold, output No.

B - Parking 2

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 200 points

Problem Statement

There is a parking lot. The fee for parking in this lot is as follows:

- During the period from exactly L o'clock to exactly R o'clock, a fee of X is charged for each hour parked.
- During the period not covered above, a fee of Y is charged for each hour parked.

If a car is parked in this lot from exactly A o'clock to exactly B o'clock without crossing midnight, how much is the fee?

Constraints

- $1 \leq X, Y \leq 1000$
- $1 \leq L < R \leq 23$
- $1 \leq A < B \leq 23$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
 $X$   $Y$   $L$   $R$   $A$   $B$ 
```

Output

Output the answer.

Sample Input 1

```
700 300 9 17 7 21
```

Sample Output 1

7400

If a car is parked from 7 o'clock to 21 o'clock, the fee is as follows.

- For the 2 hours from 7 o'clock to 9 o'clock, a fee of $300 \times 2 = 600$ is charged.
- For the 8 hours from 9 o'clock to 17 o'clock, a fee of $700 \times 8 = 5600$ is charged.
- For the 4 hours from 17 o'clock to 21 o'clock, a fee of $300 \times 4 = 1200$ is charged.

The answer is $600 + 5600 + 1200 = 7400$.

Sample Input 2

600 500 9 17 17 20

Sample Output 2

1500

If a car is parked from 17 o'clock to 20 o'clock, the fee is as follows.

- For the 3 hours from 17 o'clock to 20 o'clock, a fee of $500 \times 3 = 1500$ is charged.

The answer is 1500.

Sample Input 3

900 200 12 14 11 13

Sample Output 3

1100

C - Reverse Permutation

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 300 points

Problem Statement

You are given an integer N and a string S of length N consisting of o and x.

There is an integer sequence $A = (A_1, A_2, \dots, A_N)$ of length N . Initially, $A = (1, 2, \dots, N)$.

Perform the following operation on A for $k = 1, 2, \dots, N$ in this order.

- If $S_k = \text{o}$, reverse the first k terms of A . Specifically, replace A with $(A_k, A_{k-1}, \dots, A_1, A_{k+1}, A_{k+2}, \dots, A_N)$.
- If $S_k = \text{x}$, do nothing.

Find A after all the operations are completed.

Constraints

- $2 \leq N \leq 5 \times 10^5$
- N is an integer.
- S is a string of length N consisting of o and x.

Input

The input is given from Standard Input in the following format:

```
 $N$   
 $S$ 
```

Output

Output the elements of A after all the operations are completed, separated by spaces.

Sample Input 1

```
5  
ooxoo
```

Sample Output 1

```
5 2 1 3 4
```

A changes as follows with each operation:

- For $k = 1$: reverse the first 1 term of A . A becomes $(1, 2, 3, 4, 5)$.
- For $k = 2$: reverse the first 2 terms of A . A becomes $(2, 1, 3, 4, 5)$.
- For $k = 3$: do nothing.
- For $k = 4$: reverse the first 4 terms of A . A becomes $(4, 3, 1, 2, 5)$.
- For $k = 5$: reverse the first 5 terms of A . A becomes $(5, 2, 1, 3, 4)$.

After all the operations are completed, A is $A = (5, 2, 1, 3, 4)$.

Sample Input 2

```
7  
0000000
```

Sample Output 2

```
7 5 3 1 2 4 6
```

Sample Input 3

```
15  
X00X0X0X0X0XX00
```

Sample Output 3

```
15 11 10 7 6 3 1 2 4 5 8 9 12 13 14
```

D - X to Y

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 400 points

Problem Statement

You are given integers X , Y , and an integer K that is at least 2.

There is a variable x , which is initially $x = X$. You can perform the following operation on x zero or more times:

- Choose an integer y satisfying $\left\lfloor \frac{x}{K} \right\rfloor = y$ or $\left\lfloor \frac{y}{K} \right\rfloor = x$, and replace the value of x with y .

Here, $\lfloor z \rfloor$ is defined as the greatest integer not exceeding z , for a real number z .

Find the minimum number of operations required to make $x = Y$. Under the given constraints, it can be proved that there always exists a way to make $x = Y$ in a finite number of operations.

You are given T test cases; solve each of them.

Constraints

- $1 \leq T \leq 2 \times 10^5$
- $0 \leq X, Y \leq 10^{18}$
- $2 \leq K \leq 10^{18}$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
T
case1
case2
⋮
caseT
```

The i -th ($1 \leq i \leq T$) test case case _{i} is given in the following format:

```
X Y K
```

Output

Output the answers for the test cases in order, separated by newlines.

Sample Input 1

```
4
11 9 3
0 0 2
842 180 7
1948706013487601 48019760148910476 89014537
```

Sample Output 1

```
2
0
7
5
```

Consider the first test case.

By performing the following operations, $x = Y$ can be achieved in two operations:

- Choose $y = 3$. Since $\left\lfloor \frac{x}{K} \right\rfloor = \left\lfloor \frac{11}{3} \right\rfloor = 3$, this choice is valid. Then, replace the value of x with 3.
- Choose $y = 9$. Since $\left\lfloor \frac{y}{K} \right\rfloor = \left\lfloor \frac{9}{3} \right\rfloor = 3$, this choice is valid. Then, replace the value of x with 9.

$x = Y$ cannot be achieved in fewer than two operations, so output 2 on the first line.

E - Digit Circus

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 450 points

Problem Statement

Find, modulo 998244353, the number of integers x with $1 \leq x \leq N$ that satisfy **exactly one** of the following three conditions.

- x is a multiple of 3.
- The decimal representation of x contains 3.
- Exactly three different digits are used in the decimal representation of x .

Here, the decimal representation of an integer should not have unnecessary leading 0s.

Constraints

- N is an integer.
- $1 \leq N < 10^{500}$

Input

The input is given from Standard Input in the following format:

N

Output

Output the answer.

Sample Input 1

45

Sample Output 1

19

Among the integers from 1 to 45, there are 19 integers satisfying exactly one of the conditions in the problem statement, as follows.

- There are 10 integers satisfying only the first condition: 6, 9, 12, 15, 18, 21, 24, 27, 42, 45.
- There are 9 integers satisfying only the second condition: 13, 23, 31, 32, 34, 35, 37, 38, 43.
- There are no integers satisfying only the third condition.

Sample Input 2

1013

Sample Output 2

424

Below are examples of applicable integers.

- Integers satisfying only the first condition include 555 and 1011.
- Integers satisfying only the second condition include 343 and 553.
- Integers satisfying only the third condition include 1012 and 704.

Sample Input 3

2

Sample Output 3

0

There are no applicable integers.

Sample Input 4

314159265358979323846264338327950

Sample Output 4

```
658111391
```

F - Sjeltzer?

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 525 points

Problem Statement

A certain refrigerator assigns each drink a 6-digit numeric string as its ID.

This refrigerator stores N drinks. The i -th drink has ID S_i and size V_i . $S_1, \dots, S_N$ are distinct.

Answer Q queries. Each query is given in the form of 6-digit numeric strings x, y , and asks the following.

- Among the drinks stored in this refrigerator, find the sum of the sizes of those whose assigned ID s satisfies the following for every $k = 1, \dots, 6$.
 - $x_k \leq s_k \leq y_k$, where s_k, x_k, y_k denote the values represented by the k -th digits of s, x, y , respectively.

Constraints

- N is an integer.
- $1 \leq N \leq 3 \times 10^5$
- S_i is a string consisting of digits (0-9).
- $|S_i| = 6$
- $S_1, \dots, S_N$ are distinct.
- V_i is an integer.
- $1 \leq V_i \leq 10^9$
- Q is an integer.
- $1 \leq Q \leq 3 \times 10^5$
- In each query, x and y are strings consisting of digits (0-9).
- In each query, $|x| = |y| = 6$.

Input

The input is given from Standard Input in the following format:

```

 $N$ 
 $s_1$   $V_1$ 
...
 $s_N$   $V_N$ 
 $Q$ 
query1
⋮
query $Q$ 

```

Here, query _{q} is the q -th query, given in the following format:

```

 $x$   $y$ 

```

Output

Output Q lines. The q -th line should contain the answer to the q -th query.

Sample Input 1

```

5
000000 1
314159 2
161803 10
169231 5
384400 20
4
150001 269944
302010 396939
000000 999999
999000 000444

```

Sample Output 1

```
15
0
38
0
```

For the first query, the drinks satisfying the condition are the third and fourth drinks.

For the second query, no drinks satisfy the condition.

For the third query, the drinks satisfying the condition are the first, second, third, fourth, and fifth drinks.

For the fourth query, no drinks satisfy the condition.

Sample Input 2

```
8
778722 12744734
488742 359266786
955942 335004463
454854 331175626
934246 179761526
927777 657312747
652568 109900930
753279 405123850
8
201000 785589
202325 955898
310401 875947
044023 988999
111230 567897
133241 577989
101033 999499
453013 796988
```

Sample Output 2

```
515024780
109900930
0
331175626
331175626
331175626
584885376
331175626
```

G - Sum of Mex of Mod of Linear

Time Limit: 3 sec / Memory Limit: 1024 MiB

Score : 625 points

Problem Statement

You are given integers N, M, C, K and an integer sequence $A = (A_1, A_2, \dots, A_N)$ of length N .

You are given Q queries, which you should process in order. The q -th ($1 \leq q \leq Q$) query gives integers i_q, X_q , and you should solve the following problem after changing A_{i_q} to X_q .

Find the value of $\sum_{k=0}^{K-1} \text{mex}_{1 \leq i \leq N} \{(Ck + A_i) \bmod M\}$. Here, $\text{mex}_{1 \leq i \leq N} B_i$ denotes the smallest non-negative integer not contained in B , for an integer sequence $B = (B_1, B_2, \dots, B_N)$.

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $0 \leq C < M \leq 10^9$
- $1 \leq K \leq 10^9$
- $0 \leq A_i < M$
- $1 \leq Q \leq 2 \times 10^5$
- $1 \leq i_q \leq N$
- $0 \leq X_q < M$
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```

N M C K
A_1 A_2 ... A_N
Q
i_1 X_1
i_2 X_2
⋮
i_Q X_Q

```

Output

Output Q lines.

The q -th line ($1 \leq q \leq Q$) should contain the answer to the problem after processing the q -th query.

Sample Input 1

```
2 3 1 2
0 2
3
2 2
1 1
2 1
```

Sample Output 1

```
3
1
0
```

The first query sets $A = (0, 2)$. The sought value is $\text{mex}(\{0, 2\}) + \text{mex}(\{1, 0\}) = 1 + 2 = 3$.

The second query sets $A = (1, 2)$. The sought value is $\text{mex}(\{1, 2\}) + \text{mex}(\{2, 0\}) = 0 + 1 = 1$.

The third query sets $A = (1, 1)$. The sought value is $\text{mex}(\{1, 1\}) + \text{mex}(\{2, 2\}) = 0 + 0 = 0$.

Sample Input 2

```
7 9 3 19
7 0 2 8 1 3 4
7
3 5
1 6
3 2
5 5
4 7
7 8
2 4
```

Sample Output 2

```
32
44
53
37
49
37
54
```

Hints: AtCoder Beginner Contest 465

Taken from the official editorial, not written by us. Read one hint at a time.

A. Supermajority *(editorial, first line)*

Find the value $B \times \frac{2}{3}$ as a floating-point number, and compare it with A to find the answer.

B. Parking 2 *(editorial, first line)*

The fee within the h o'clock (from exactly h o'clock to exactly $(h + 1)$ o'clock) can be computed as follows: X if $L \leq h \leq R - 1$; Y otherwise For the integer h , run ...

C. Reverse Permutation *(editorial, first line)*

First, let us rephrase the procedure as follows: Let $A = ()$.; For $k = 1, 2, \dots, N$, do the following: Append k to the end of A .

D. X to Y *(editorial, first line)*

Consider the graph with edges between integers that can be replaced from one to another.

E. Digit Circus *(editorial, first line)*

We can solve the problem using the method called Digit DP (Dynamic Programming).

F. Sjeltzer? *(editorial, first line)*

x In this subproblem, the range of s of interest is those s such that $s_k \leq y_k$ for all k .

G. Sum of Mex of Mod of Linear *(editorial, first line)*

Let c_x be the number of integers k between 0 (inclusive) and K (exclusive) satisfying $Ck + x \equiv 0 \pmod{M}$.

Source: <https://atcoder.jp/contests/abc465/editorial>